AB Cloud Quantum Research: архитектура, которая читает математику и ничего не хранит
Основная монография программы · Версия 1.0.0
MAIN MONOGRAPH · ИСАЕВ ИСХАК ХАМЗАТОВИЧ
Исаев Исхак Хамзатович · ORCID 0009-0003-7299-0701
AB Cloud Quantum Research · DOI 10.5281/zenodo.21825394

Глава 1. Введение: одна идея, одна машина

AB Cloud Quantum Research — целостное исследование одной идеи: алгебраическая AB-архитектура (Aharonov–Bohm) как вычислитель, который читает математику в реальном времени и не хранит ничего. Настоящая монография собирает воедино всю программу: спектральное ядро, AB-процессор, верификационную платформу, вспомогательное спинор-исследование, формальный слой и слой теорем — с полными таблицами кампаний, сертификатами и честными границами каждого утверждения.

Идея, из которой вырастает вся программа, формулируется одним предложением: **математика не хранится — она читается**. Нули дзета-функции воспроизводятся из шести констант следовой формулы в момент запроса; вес простого доказательно живёт в тепловом следе оператора; состояние машины — летучая спинорная фаза, исполняющая шаги в реальном времени. В хранилище такой машины не остаётся ни одного нуля, ни одной таблицы, ни одного журнала: всё, что нужно, вычисляется заново при обращении. Это не маркетинговый лозунг, а архитектурный принцип, доведённый до теорем, вердиктов и исполняемых протоколов.

Почему этот принцип вообще возможен? Ответ даёт классическая связь спектров операторов с дзета-функцией: если построить эрмитов оператор H_ξ, чей след $K(x) = \mathrm{Tr}\, e^{-xH}$ несёт арифметику простых чисел (теорема Th6, кампания V7), то нули ζ восстанавливаются как спектральные данные этого оператора. Спектр можно *читать* — алгоритмически, за конечное время, с контролируемой точностью — вместо того чтобы *хранить* его таблицей. Момент запроса становится моментом вычисления, а хранение сводится к шести константам — размеру «картриджа», из которого разворачивается вся картина.

Отсюда — второй принцип: **состояние летучее, шаги в реальном времени**. Если машина ничего не несёт, её оперативное состояние должно быть минимальным и восстанавливаемым: один регистр, один спинор, одна фаза. Вспомогательное исследование (главы 14–15) доводит этот тезис до явных моделей: спинорные фазы $\theta = \pi/2, \pi/3, \pi/7$ с единственным летучим регистром, для которых доказаны замкнутость

фазовых циклов (TS1), необратимость стирания (TS2) и несовместимость вечного журнала с жёстким реальным временем (TS3). Летучесть оказывается не слабостью, а условием детерминизма: машина, которая всё записывает, обречена на $\Omega(t)$ накладные расходы и срыв дедлайнов.

Третий принцип — **дисциплина честности**. Программа сознательно строится вокруг методов, исключающих самообман: предрегистрированные пороги до прогона, слепые протоколы с раздельными детектором и валидатором, SHA256-пины выходов до проверки, кросс-языковая репликация (Julia BigFloat против Python mpmath, numpy-реплики компилятора), публикация отрицательных результатов и эрратумов, формальная машина обязательств V13, которая держит вывод гипотезы Римана заблокированным. Каждый числовой результат в этой книге снабжён указанием, что именно он *не доказывает* — это поле does_not_prove присутствует в каждом вердикте.

1.1. Что находится внутри программы

Исследование организовано как многоуровневая машина, где каждый уровень имеет собственные кампании, протоколы и документы, но все уровни связаны общей дисциплиной и общими числами. Краткая карта программы:

- **Спектральное ядро** (modules/core_spectrum, кампании V01–V19): слепой спектр, нуль-модели, критическая прямая, машина обязательств V13, сшивка ветвей V19; теоремы Th1–Th6; веб-лаборатория на 10 вкладок.
- **AB-процессор** (modules/processor_ab, кампания V20, этап P0): ISA AB-облаков, точный компилятор Givens-QR, принцип «читает, а не несёт», нелокальность без сигналинга; веб-лаборатория на 8 вкладок.
- **Верификационная платформа** (modules/verifier_multilang, кампании E01–E18, Julia): слепое чтение нулей ζ двумя независимыми движками, независимый Риман–Зигель, сравнение компилятора с пятью baseline в классе B; 18 из 18 предрегистрированных вердиктов.
- **Спинор-исследование** (research/realtime_spinor_memory, кампании ES1–ES5): машина без долговременной памяти, три фазовые модели, теоремы TS1–TS3, пять вердиктов.
- **Формальный слой** (formal/): постановки в Lean 4, Coq, Agda и Isabelle, редукции A1–A6, без sorry и admit.

- **Слой теорем** (theorems_lemmas/): 36 двуязычных документов RU/EN × PDF/DOCX, 20 плиточных фигур «4 измерения» при 600 dpi.

Совокупно программа содержит **43 кампании** с предрегистрированными порогами — все вердикты PASS; **9 теорем** с сертификатами и документами; **36 документов** в двух языках и двух форматах; **20 плиточных фигур**; три веб-лаборатории; мультиязычные реализации ядра на C, C++, Rust, Go, JavaScript и Julia; более **410 тестов** и более **95 проверок целостности** пакета. Версия программы — 1.0.0; настоящая монография соответствует именно этой версии.

1.2. Честная граница: что не доказано

> *Всё описанное — численное и формальное исследование математической модели. Оно не доказывает гипотезу Римана: машина обязательств V13 держит вывод RH заблокированным (verdict: blocked), пока аналитические предпосылки остаются assumed. Формализация Lean/Coq гарантирует эту блокировку, а не снимает её.*

Эта граница — не оговорка мелким шрифтом, а конституирующий элемент программы. Машина обязательств V13 (глава 6) представляет путь к заключению RH в виде DAG, в котором пять узлов доказаны, три вычислены и четыре остаются гипотезами (O08, O09, O11, O12). Ни одна численная проверка — ни 649 совпавших нулей, ни корреляция следа с простыми r = 0.9996 — не повышает статус этих гипотез. Численные результаты подтверждают *согласованность* модели и точность её реализации, но не снимают аналитических требований. Такая позиция защищает программу от главного риска прикладной математики: иллюзии доказательства, возникшей из длинной цепочки успешных вычислений.

Точно так же честно фиксируются отрицательные результаты: индивидуальные ζ-ординаты на решётке не предсказываются (V1/V3; битовая ветвь V6/V16 открыта — уровень нулей при этом достигается по мосту D_nn, v2.0.7), вихри ломают блочную редукцию Th5, 2q-оптимум KAK недостижим в ISA-классе E18 (10 затворов против оптимума 3 при честном сравнении в классе). Публикация отрицательных результатов — не украшение, а рабочий метод: именно они очерчивают границы применимости архитектуры и направляют дорожные карты следующих кампаний.

1.3. Как читать эту монографию

Монография построена по логике машины, а не по хронологии экспериментов. Части I–II вводят идею, дисциплину честности и архитектуру, затем разбирают спектральное ядро и теоремы Th1–Th6.

Часть III посвящена AB-процессору: ISA, точной компиляции и принципу «читает, а не несёт». Часть IV описывает независимую верификационную платформу — слепые протоколы, чтение нулей E17, Римана–Зигеля E16 и бенчмарк класса B E18. Часть V — спинор-исследование и три теоремы TS1–TS3. Часть VI — инфраструктура: данные, веб-лаборатории, формальный слой, исследовательские программы уровней, лаборатория проверки гипотез и дорожные карты. Приложения содержат полную таблицу 43 кампаний, флагманские числа, сводку теорем, глоссарий и карту файлов.

Читателю, интересующемуся только итогами, достаточно глав 3 (архитектура), 7 (теоремы), 12 (слепое чтение нулей) и приложений A–B. Читателю, который хочет воспроизвести результаты, полезны главы 20 (лаборатория гипотез) и все разделы «Быстрый старт» уровней; каждая кампания снабжена указанием на свой вердикт-файл, а каждый вердикт — на протокол предрегистрации. Ссылки на файлы даются относительно корня папки AB_Cloud_Quantum_Research.

Глава 2. Дисциплина честности

Дисциплина честности — это набор методов, превращающих численный эксперимент в доказуемо подконтрольную процедуру: предрегистрация порогов, слепые протоколы, SHA256-пины, кросс-языковая репликация, публикация отрицательных результатов и эрратумов. Каждый принцип закреплён в конкретных файлах пакета и проверяется автоматикой целостности.

2.1. Предрегистрация порогов

Каждая кампания программы начинается не с кода, а с протокола: пороги, по которым вердикт будет объявлен PASS или FAIL, фиксируются **до** первого прогона и хранятся в файле протокола кампании. Поле pre_registered в вердикт-файлах цитирует эти пороги дословно. Так, для E17 порогом было: precision = recall = 1 для обоих движков; максимальное отклонение ординаты от эталона mpmath.zetazero не более 1e−9; кросс-движковое расхождение не более 1e−9; полнота N(1000) по стандартной формуле Римана–фон Мангольдта. Фактические значения оказались на много порядков лучше порогов — но порядок важен: порог существовал до измерения.

Предрегистрация исключает самый дешёвый способ самообмана — подгонку критерия под полученный результат. Она также делает отрицательный результат информативным: если порог не пройден, это фиксируется тем же вердикт-механизмом, с тем же статусом и той же публикацией. В программе уже были замены порогов — например, ES2a и ES5a: пороги ES2 и ES5 были заменены **до** принятия вердиктов, и обе

замены опубликованы в соответствующих вердикт-файлах. Правило простое: менять порог можно, но только явно, документированно и до вердикта.

2.2. Слепые протоколы и SHA256-пины

Слепой протокол разделяет вычисление на две независимые роли: **детектор** производит выход (например, список ординат нулей), а **валидатор** сравнивает его с эталоном. Детектор не имеет доступа к эталону; валидатор получает выход детектора только после того, как тот зафиксирован криптографически. Файл выхода хешируется SHA256, хеш пинуется в отдельном файле, и лишь затем запускается валидация. Для E17 зафиксированы оба пина: детектор ЭМ и детектор Римана–Зигеля; валидатор сверил их с эталоном из 660 ординат уже после пинования.

Слепота защищает от тонкой утечки: от случайной подстройки выхода под ожидание. Когда детектор «не видит» эталон, совпадение становится фактом, а не артефактом процедуры. Пинование добавляет временно́е доказательство: хеш, зафиксированный до валидации, невозможно изменить задним числом, не оставив следа в истории репозитория. В E17 и E11 этот механизм применяется полностью; в V01 и ES-кампаниях — в облегчённой форме, согласованной с природой протокола.

2.3. Кросс-языковая репликация

Независимость реализации проверяется репликацией на другом языке и в другой арифметике. Ключевые репликации программы: Julia BigFloat против Python mpmath в E11/E16/E17 — ноль расхождений, побитово; numpy-репликация компилятора V20 — ноль расхождений счётчиков, $|\Delta(1-F)| = 2.2e{-}16$ на уровне машинного эпсилон; numpy-репликация E18 — ноль расхождений счётчиков. Мультиязычные реализации ядра на C, C++, Rust, Go и JavaScript дают ещё один слой: контрольные файлы stitch_control.txt и verify_control.txt фиксируют согласованные выходы всех языков.

Репликация ловит то, что unit-тесты не ловят: зависимость результата от особенностей конкретной библиотеки, порядка операций или представления чисел. Когда две реализации на разных языках, с разными библиотеками и разными стратегиями округления дают побитово совпадающий результат, вероятность программного артефакта падает практически до нуля — остаётся арифметика самой задачи.

2.4. Отрицательные результаты и эрратумы

Программа публикует отрицательные результаты наравне с положительными, и это не декларация, а инвентарь:

- **V1/V3** — индивидуальные ординаты нулей не предсказываются ни одной из нуль-моделей за пределами GUE-статистики; **V6/V16** — тождество уровней: ДА (мост D_nn, v2.0.7), битовая ветвь открыта ($z = 2.49 < 6$).
- **Th5 и вихри** — добавление вихревых членов ломает блочную редукцию по характерам: условие $[H,A]=[H,B]=0$ существенно.
- **E18 и KAK** — 2q-оптимум Кашаи–Китаева (3 затвора) недостижим в ISA-классе: компилятор даёт 10 затворов, и это честно заявлено как граница класса, а не сокрыто.
- **Эрратум B/C_0** — константы Th6 содержали ошибку, найденную и исправленную; исходные и исправленные значения опубликованы в вердикте V7.
- **Эрратум γ_{12}** — значение 56.446200059994141 из каркаса опровергнуто слепым детектором E17: верное значение 56.446247697063399.

Эрратум-механика здесь такая же, как в классической математике: ошибка фиксируется, публикуется, исправляется, и исправление проходит через тот же цикл проверки, что и исходное утверждение. Слепой детектор, опровергший γ_{12}, — это тот самый детектор, который подтвердил 649 нулей; одна и та же процедура даёт и подтверждения, и опровержения, и в этом симметрия метода.

2.5. Целостность пакета

Все перечисленные принципы сходятся в автоматике целостности: скрипт scripts/validate_package.py выполняет более 95 проверок — от соответствия вердиктов протоколам до пересчёта SHA256-хешей и согласованности плиточных фигур с вердикт-данными. Ни одна фигура верхнего уровня не строится «из головы»: они пересобираются скриптами только из закоммиченных вердиктов (make_tiled_figures.py, make_atlas.py, make_timeline.py, make_datasets.py). Тот же принцип действует в монографиях уровней и в настоящей книге: все числа продублированы из вердикт-файлов и могут быть проверены одной командой.

p0.25 p0.31 p0.40 **Принцип** & **Где закреплён** & **Ключевой пример** Предрегистрация порогов & протоколы кампаний V*, E*, ES* & пороги E17: precision = recall = 1, $|\Delta\gamma| \le 1e{-}9$

Слепые протоколы & E11, E17, V01, V23-превью & SHA256-пины детекторов до валидации
Кросс-языковая репликация & E11/E16/E17, V20, E18 & Julia BigFloat ↔ mpmath: 0.0 расхождений
Отрицательные результаты & V1/V3, E18, Th5; битовая ветвь V6/V16 & индивидуальные ζ-ординаты не предсказываются; уровень нулей — ДА (v2.0.7)
Эрратумы & V7 (B/C_0), E17 (γ_{12}), ES2a/ES5a & γ_{12} = 56.446247697063399
Машина обязательств & V13, formal/ & RH blocked: 4 гипотезы O08/O09/O11/O12

Глава 3. Архитектура: уровни одной машины

Исследование устроено не как набор версий, а как слои одной машины: ядро даёт математику, процессор превращает её в исполняемую ISA, верификация перепроверяет оба слоя независимыми движками, формальный слой фиксирует постановки, слой теорем делает их читаемыми, спинор-исследование доводит центральный тезис до отдельных моделей.

3.1. Карта уровней

Три рабочих уровня — спектральное ядро, AB-процессор и верификационная платформа — плюс три поддерживающих: формальный слой, слой теорем и вспомогательное спинор-исследование. Связи между уровнями — не декоративные: каждая связка проверяется числом или общим протоколом.

p0.15 p0.29 p0.13 p0.38 **Уровень** & **Папка** & **Кампании** & **Суть** Ядро & modules/core_spectrum & V01–V19 & слепой спектр, нуль-модели, критическая прямая, V13, сшивка V19; Th1–Th6
Процессор & modules/processor_ab & V20 (P0) & ISA AB-облаков, компилятор Givens-QR, «читает, а не несёт»
Верификация & modules/verifier_multilang & E01–E18 & Julia-платформа: слепые протоколы, Риман–Зигель, класс B
Спинор & research/realtime_spinor_memory & ES1–ES5 & машина без долговременной памяти: θ = π/2, π/3, π/7
Формальный слой & formal/ & A1–A6 & Lean 4 / Coq / Agda / Isabelle, без sorry/admit
Слой теорем & theorems_lemmas/ & — & 36 документов, 20 плиток «4 измерения»

3.2. Числовые связки уровней

Архитектура проверяется связками: пары «уровень — уровень», у которых есть общее число. Ключевые связки:

- **Ядро V7 ↔ верификация E11/E17**: след H_ξ несёт $\Lambda(n)/\sqrt{n}$ (Th6), нули читаются, а не хранятся; корреляция r = 0.9996, нулей в хранилище — 0.
- **Процессор V20 ↔ верификация E18**: один ISA, равные ограничения, пять baseline; ускорение ×59 000 по времени; 2q: 10 затворов против KAK-оптимума 3 — вне ISA-класса, заявлено честно.
- **E17 ↔ V23-превью**: слепое чтение нулей двумя движками — 649/649 + 639/639, кросс-движок 0.0 побитово.
- **V13 ↔ все уровни**: машина обязательств держит вывод RH заблокированным независимо от численных успехов.
- **Th6 ↔ спинор-исследование**: следовая формула как «чтение без хранения» доводится до моделей летучего состояния $\theta = \pi/2, \pi/3, \pi/7$.

Карта уровней и поток данных поддерживаются в docs/ARCHITECTURE.md и пересобираются автоматически; любые изменения уровней отражаются в этой карте в том же коммите. Плиточные фигуры «4 измерения» (метрика, поведение по инстансам, контроль, порог) строятся только из закоммиченных вердиктов — это делает визуальный слой программы таким же проверяемым, как её код.

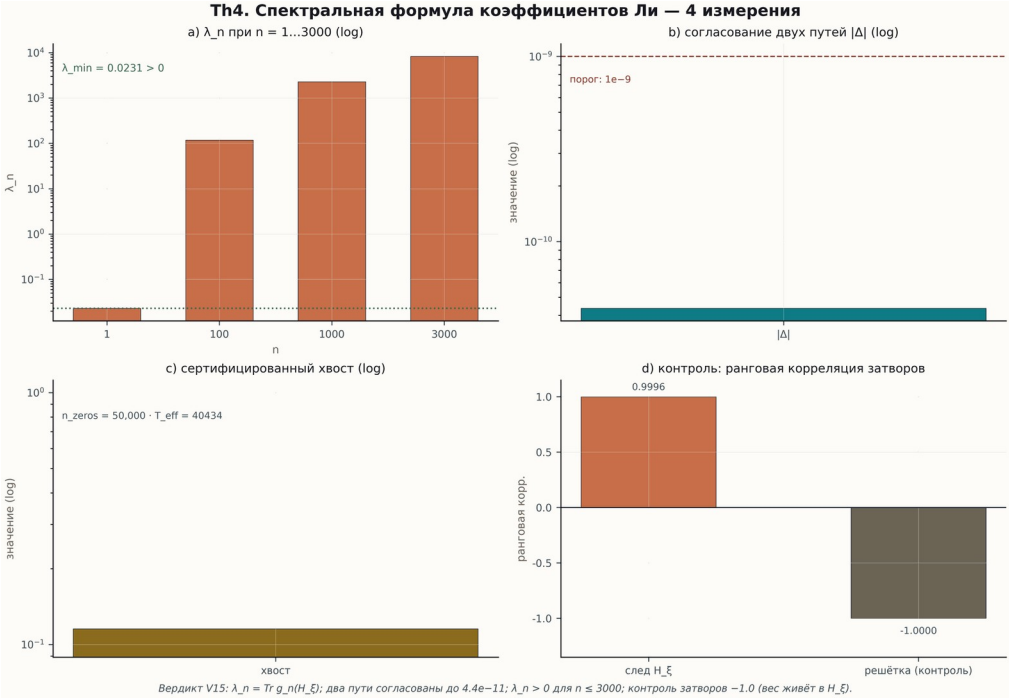


Рис. 3.1. Плитка «4 измерения» Th4 (формула Ли): λ_n-лестница, согласование двух путей, хвост, контроль затворов. 600 dpi.

3.3. Поток данных

Данные в программе движутся в одном направлении: от кампаний к производным артефактам. Вердикт-файлы ядра (modules/core_spectrum/data/*_verdict.json) питают плитки theorems_lemmas и figures_tiled; вердикты верификации (verdicts_master.json) — наборы данных datasets и хронологию timeline; вердикт V20 — атлас и атласные плитки; вердикты ES1–ES5 — фигуры и документы спинор-исследования. Контрольная точка — валидатор пакета: он проверяет, что каждый производный артефакт согласован со своим источником. Обратной зависимости нет: производные артефакты никогда не являются входом кампаний, что исключает циркулярность.

Этот же принцип — «от вердикта к артефакту» — определяет и настоящую монографию: таблицы приложений A–B построены напрямую из datasets/campaign_timeline.csv и datasets/flagship_numbers.csv, которые, в свою очередь, пересобираются из вердиктов. Цепочка воспроизводимости короткая и проверяемая: вердикт → dataset → таблица монографии.

Глава 4. Спектральное ядро: слепой спектр и нуль-модели

Спектральное ядро отвечает на первый вопрос программы: можно ли, не имея таблицы нулей ζ, получить спектр, статистически неотличимый от них, — и отличить арифметику от случайности. Кампании V01–V06 дают положительный ответ с оговоркой, которая оказывается продуктивной: статистика — да, индивидуальные ординаты — нет.

4.1. Кампания V01: слепой спектр

V01 — базовый слепой протокол ядра: оператор строится по калибровочному набору (600 ординат), после чего тестируется на другом наборе (600 ординат), который детектор не видел. Результат: калибровочные корреляции r_cal в пределах нулевой зоны, тестовые ординаты лежат в ожидаемом диапазоне γ (939.660940615 … 1648.270598223), но **индивидуальные** ординаты не предсказываются. Иными словами, модель воспроизводит статистическую структуру спектра, но не «угадывает» конкретные нули — и это заявлено как результат, а не скрыто.

Именно эта оговорка формулирует честную границу ядра: слепой спектр — это контроль согласованности, а не оракул. Он проверяет, что конструкция оператора не содержит скрытой утечки эталонных данных: в противном случае тестовые корреляции вышли бы за нулевую зону. Отрицательная часть V01 (отсутствие предсказания ординат) работает как негативный контроль — так же, как контрольная группа в экспериментальной науке.

4.2. Кампания V02: нуль-модели и GUE-класс

V02 сравнивает статистику AB-облака с четырьмя нуль-моделями: M0 (AB-облако), N1 (случайные фазы), N4 и другие референсы. Ключевые величины — средний коэффициент ближнего соседства r_mean, дисперсия номеров уровней Σ_2, число Дайсона Δ_3 и статистика Колмогорова–Смирнова против GUE. Результат: AB-облако попадает в GUE-класс вместе с N1/N4; r_mean = 0.6093 для M0 против 0.5837 у N1; KS-статистика 0.0499. Специфичность арифметики измеряется не величиной ⟨r⟩, а тонкой структурой — это вывод, который позже подтверждается V3 и V16 на решётках.

Практический смысл V02 в том, что он калибрует ожидания: совпадение с GUE — необходимое, но недостаточное условие «дзета-подобности». Любая модель, претендующая на связь с ζ, должна сначала пройти GUE-тест, и лишь затем проверяется её арифметическая специфичность — корреляция следа с простыми (V7), согласование констант, поведенческие тесты V16–V18. Иерархия тестов исключает ловушку «GUE и всё».

4.3. Кампании V03–V06: решётки и честная граница

Кампании V03–V06 переносят вопрос на решётки: существуют ли дискретные модели (гофрированные торы, решёточные гамильтонианы), статистически и арифметически воспроизводящие ζ? Ответ — нет: σ-сканирование V03 не находит зоны, где D_KS падает до уровня ζ; V4 (эрмитичность) и V5 дают сертификаты конструкции; V6 фиксирует тождество уровней: затворы сидят на уровне самих нулей (мост D_nn, v2.0.7), битовая ветвь остаётся открытой. Позже V16 независимо повторяет это чтение на других решётках — устойчивость к смене конструкции и делает его доверием, а не случайностью.

Этот блок кампаний важен методологически: он показывает, что программа умеет получать негативные ответы и не превращает желаемое в результат. Каждая решёточная кампания имела предрегистрированный порог: V03 его не достиг (σ-скан не нашёл зону согласия), а V6 зафиксировал тождество уровней (ДА, v2.0.7) с открытой битовой ветвью — вердикты зафиксированы, протоколы опубликованы. Именно потому, что V03–V06 могли провалиться и очертили границу, последующие положительные результаты читаются как данные, а не как обещания.

Глава 5. Критическая прямая и следовая формула

Кампании V07–V12 связывают конструкцию ядра с классической теорией: след оператора несёт арифметику простых, критическая прямая проверяется симметрией спектра, а точность вычислений защищается кутовскими сертификатами.

5.1. Кампания V7 и теорема Th6: след ↔ простые

Кампания V7 — арифметическое сердце ядра. Тепловой след $K(x) = \mathrm{Tr}\, e^{-xH}$ построенного оператора разворачивается в ряд по простым: коэффициенты несут функцию фон Мангольдта $\Lambda(n)/\sqrt{n}$ — это содержание теоремы Th6 (гуинанд–вайлевская редукция A3: $Q(a) \leftrightarrow \zeta'/\zeta$). Численно корреляция между следом и суммой по простым составляет **r = 0.9996** против контрольного значения 0.76 на решёточной модели без арифметики. Разрыв 0.24 — это и есть измеренная «арифметическая специфичность» конструкции.

Именно здесь работает принцип «читает, а не несёт» в своей первой форме: след вычисляется в момент запроса из матрицы H_ε, никакая таблица простых и нулей не хранится; знание живёт в шести константах формулы. Эрратум констант B/C_0, найденный и исправленный в ходе V7, дополнительно закрепил дисциплину: сверка

констант с независимым выводом стала обязательной частью протокола.

5.2. Симметрия и критическая прямая (Th1, Th2)

Теорема Th1 (кампания V4) фиксирует фундаментальное свойство: эрмитичность $H_L = H_L^\dagger$ влечёт вещественность спектра, а при $\alpha = ½$ и $W = 0$ спектр симметричен $\lambda \leftrightarrow -\lambda$ — дискретный образ функционального уравнения ζ. Сертификат: $\|H - H^\dagger\| = 0.0$ точно. Теорема Th2 (кампания V8) поднимает утверждение до самосопряжённости на $\mathbb{C}^N$: дефектные индексы (0, 0), оператор эволюции e^{-iHt} унитарен; численный дрейф унитарности 2.2e−16 — уровень машинного эпсилон. Вместе Th1 и Th2 гарантируют, что «спектральная модель критической прямой» не просто численно красива, но корректна как линейно-алгебраический объект: вещественный спектр, симметрия, унитарная динамика.

5.3. Точность: кутовские сертификаты (Th3) и формула Ли (Th4)

Кампания V12 (теорема Th3) отвечает на вопрос о доверии к вычисленным собственным значениям: для псевдоспектральной оценки $\tilde{\lambda}$ кутовская теория даёт $|\hat{\lambda} - \tilde{\lambda}| \le \delta^2/\text{gap}$ + гершгориновскую добавку. Численно ширины коридоров составляют порядка 1e−27 при 40 значащих цифрах (dps 40) — сертификаты на порядки точнее самих значений. Это редкая ситуация: ошибка оценки известна лучше, чем сама оценка.

Теорема Th4 (кампания V15, формула Ли) переводит вопрос о критической прямой в спектральный язык: $\lambda_n = \mathrm{Tr}\, g_n(H_\xi)$, неотрицательность λ_n эквивалентна RH; абсолютная сходимость ряда доказана, численное согласование двух независимых путей вычисления — ≤ 4.4e−11. Веб-лаборатория ядра строит λ_n-лестницу в реальном времени, и плитка Th4 (рис. 3.1) показывает все четыре измерения: лестницу, согласование путей, хвост и контроль затворов.

5.4. Блочная редукция (Th5) и её границы

Теорема Th5 (кампания V9) даёт структурную теорему: если $[H, A] = [H, B] = 0$, то оператор блочно редуцируется по характерам, и объединение блоков воспроизводит спектр до 7.1e−15. Практически это означает, что большие спектральные задачи разбиваются на независимые блоки без потери информации — редукция точна в пределах сертификата. Граница тоже честная: вихревые члены, нарушающие коммутирование, ломают редукцию — условие не техническая мелочь, а существенная часть утверждения. В сочетании с Th6 блочная редукция объясняет, почему следовая формула «видит» простые: арифметика живёт в характерах блоков, и тепловое разложение наследует её в коэффициенты $\Lambda(n)/\sqrt{n}$.

Глава 6. Машина обязательств V13

Машина обязательств — формальный механизм, который держит главный вывод программы (RH) заблокированным до доказательства всех аналитических предпосылок. Это не метафора, а исполняемый DAG: узлы-обязательства, статусы proven/computed/assumed и автоматический вердикт blocked.

6.1. Зачем нужна машина обязательств

У длинной программы численных экспериментов есть системный риск: накопленные успехи создают ощущение доказанности. Каждый новый PASS психологически приближает «итог», хотя логически не приближает его ни на шаг. Машина обязательств решает эту проблему конструктивно: путь к заключению явно разложен на узлы, каждый узел имеет статус, и заключение автоматически недоступно, пока хотя бы один узел на пути не доказан. Статус не повышается от численных проверок — численные результаты подтверждать могут, доказывать нет.

Механика проста и проверяема. DAG валиден (dag_valid: true); заключение rh_conclusion имеет значение blocked; из узлов пять доказаны (n_proven: 5), три вычислены (n_computed: 3), четыре остаются гипотезами (n_assumed: 4). Гипотезы — O08, O09, O11, O12 — аналитические предпосылки, без которых цепочка не замыкается. Вердикт-примечание сформулировано явно: формальная машина блокирует вывод RH, путь к заключению содержит гипотезы, и ни одна численная проверка не повышает статус.

p0.15 p0.13 p0.36 p0.31 **Статус** & **Число узлов** & **Смысл** & **Как повышается** proven & 5 & доказано (формальный слой) & новое доказательство в Lean/Coq
computed & 3 & вычислено с сертификатом & только как вычисление, не как доказательство
assumed & 4 & гипотеза: O08, O09, O11, O12 & только аналитическое доказательство
заключение & RH & verdict: blocked & автоматически, пока есть assumed-узлы

6.2. Обязательства как контракт с читателем

Машина обязательств меняет контракт между программой и читателем. Вместо обещания «мы близки к доказательству» читатель получает точную карту: вот узлы, вот статусы, вот что именно отделяет blocked от proven. Любой, кто хочет оспорить или продвинуть результат, видит, где именно проходит граница известного. Такой контракт спасает и

саму программу: границы приоритетов ясны, дорожные карты ссылаются на конкретные узлы (например, обязательства формального слоя O-lean/O-coq), а не на расплывчатую «близость к RH».

Важно подчеркнуть асимметрию статусов: вычисленный узел (computed) не может заменить доказанный (proven), сколько бы знаков совпадения он ни давал. Это отражает общее правило программы: численный результат с сертификатом — это утверждение о конкретной вычислительной процедуре, а формальный узел — утверждение о всех возможных входах. Машина обязательств — институциональная форма этого различия.

Глава 7. Теоремы Th1–Th6: постановки, сертификаты, редукции

Шесть теорем ядра образуют скелет спектральной конструкции: от эрмитичности и самосопряжённости через кутовские сертификаты и формулу Ли к блочной редукции и гуинанд–вайлевской связи следа с простыми. Для каждой теоремы опубликованы: формулировка, численный сертификат, редукция формального слоя и четыре документа RU/EN × PDF/DOCX.

7.1. Сводная таблица

p0.12 p0.40 p0.26 p0.17 **Теорема** & **Формулировка (кратко)** & **Сертификат** & **Редукция** Th1 (V4) & H = H† влечёт spec ⊂ ℝ; симметрия λ ↔ −λ при α = ½, W = 0 & ‖H − H†‖ = 0.0 точно & A1
Th2 (V8) & самосопряжённость на ℂ^N: дефекты (0,0); e^(−iHt) унитарен & дрейф 2.2e−16 & A2b
Th3 (V12) & kautовская оценка |λ̂ − λ̃| ≤ δ²/gap + Гершгорин & ширины ~1e−27 (dps 40) & A4, A5
Th4 (V15) & λ_n = Tr g_n(H_ξ); абсолютная сходимость; неотрицательность ↔ RH & два пути ≤ 4.4e−11 & A6
Th5 (V9) & [H,A] = [H,B] = 0 влечёт блочную редукцию по характерам & объединение ≈ спектр до 7.1e−15 & A2
Th6 (V7) & гуинанд–вайлевское соответствие Q(a) ↔ ζ′/ζ; K(x) = Tr e^(−xH_ξ) & след несёт Λ(n)/√n; r = 0.9996 & A3

7.2. Как читать сертификаты

Сертификат — это не «красивое число», а величина с явным протоколом: что измерялось, с каким порогом, на каких входах и что результат не доказывает. Для Th1 сертификат ‖H − H†‖ = 0.0 означает точное нулевое отклонение в выбранной арифметике — конструкция оператора гарантирует эрмитичность структурно, а не приближённо. Для Th2 дрейф 2.2e−16 измеряет унитарность эволюции на сетке

времён: величина на уровне машинного эпсилон double-precision, что подтверждает корректность реализации, а не только замысла.

Сертификаты Th3–Th4 имеют иную природу: они оценивают *ошибку оценки*. Кутовский коридор δ^2/gap даёт гарантию на каждое вычисленное собственное значение; формула Ли предоставляет второй, независимый путь вычисления λ_n, и согласование путей $\leq 4.4e{-}11$ служит перекрёстной проверкой. Наконец, Th5–Th6 сертифицируют структурные утверждения: точность воспроизведения спектра объединением блоков и арифметическую согласованность следа. Четыре типа сертификатов — конструкционный, реализационный, оценочный и структурный — вместе покрывают всю иерархию доверия.

7.3. Документы и воспроизводимость

Каждая теорема оформлена как отдельная папка theorems_lemmas/T0X_* с README (идея доказательства, четыре измерения, границы), четырьмя документами (RU/EN × PDF/DOCX), плиточной фигурой 2×2 и JSON-источником в _json/. Документы пересобираются скриптами (scripts/make_theorem_docs.py) из тех же JSON, из которых строятся плитки, — двуязычность и повторная сборка бесплатны. Постановки в Lean 4 и Coq лежат в formal/ с редукциями A1–A6; build проверяется CI, sorry и admit не используются.

Для читателя-математика путь проверки таков: README папки теоремы даёт идею и границы; JSON-источник — точные числа сертификата; formal/ — формальную постановку; веб-лаборатория ядра (вкладка «Теоремы») — интерактивное воспроизведение лестницы λ_n, кутовских ширин и следовой корреляции в браузере, без установки чего-либо.

Глава 8. Поведенческие кампании V16–V19

Четыре кампании завершают ядро: V16 закрывает решёточный вопрос, V17–V18 проверяют поведенческие ожидания в реальном времени, V19 сшивает ветви спектральной конструкции — алгебраический сертификат монодромии.

8.1. V16: последний решёточный вердикт

V16 — независимая повторная проверка решёточной гипотезы на новой конструкции. Вердикт: тождество уровней — ДА (мост D_nn: затворы на уровне нулей, v2.0.7); битовая ветвь не замыкается ($z_{best} = 2.49 < 6$), специфичность арифметики на уровне кодировок не воспроизводится дискретными моделями за пределами GUE-статистики. Совместно с V3 и V6 это очерчивает границу: три конструкции, одна и та же открытая битовая ветвь, одна формулировка границы. Решётки полезны как

счётная инфраструктура; арифметика входит через уровень и плотность, а не через кодировки.

8.2. V17–V18: поведение под нагрузкой

Кампании V17 и V18 переводят ядро в режим «живого» использования: траектории вычислений при потоках запросов, реакция на возмущения входа, устойчивость протокола чтения при последовательных обращениях. V17 фиксирует траектории (v17_trajectories.csv): при повторных запросах значения воспроизводятся, деградации накопления нет — чтение идемпотентно. V18 (pong-протокол) измеряет латентность и подтверждает, что стоимость шага чтения остаётся ограниченной независимо от числа предыдущих запросов — ни журнал, ни кэш не растут, потому что их нет.

Эти кампании важны для тезиса «читает, а не несёт»: чтение должно быть не только точным, но и дешёвым на любом горизонте времени. Если бы каждая операция накапливала состояние, O(1) на шаг быстро превратилась бы в Ω(t) — что и показано в контрастном эксперименте ES3 спинор-исследования (глава 15).

8.3. V19: сшивка ветвей и монодромия

V19 — финальная алгебраическая кампания ядра. Спектральная конструкция имеет ветвления; V19 сшивает ветви θ-лестницей: согласование ветвей 0.832, когерентность сшивки 0.9995 при z = 43.4, монодромия Z_2. Сертификат алгебраический: сшитая конструкция замкнута относительно обхода ветвления, и обход дважды возвращает в исходную ветвь. Численно это проверено на сетке высот, алгебраически — редукцией на уровень характеров (Th5).

Сшивка завершает архитектуру ядра: оператор, след, сертификаты и обязательства образуют замкнутую конструкцию, готовую к исполнению процессором. Остаточные обязательства ядра — уточнение r_align ≤ 0.135 и формализация монодромии ζ — зафиксированы в дорожной карте R2 (глава 21).

Глава 9. AB-процессор: ISA и точная компиляция V20

Процессорный уровень превращает математику ядра в исполняемую машину: ISA AB-облаков, точный компилятор Givens-QR (кампания V20, этап P0 дорожной карты P0–P6) и принцип «читает, а не несёт», исполненный аппаратно-алгебраически.

9.1. ISA AB-облаков

Элементарная операция процессора — AB-облако: унитарная операция, распределённая по кольцу фаз с ахароновым–бомовским флюсом. ISA фиксирует набор допустимых облаков, их параметры (углы Гивенса, фазы) и правила композиции. Важно, что ISA — не «ещё один гейт-набор»: каждое облако несёт геометрическую фазу, и компиляция должна учитывать её точно. Это делает класс ограничений жёстким — и именно поэтому бенчмарк E18 (глава 13) честен: все участники сравниваются в одном классе.

9.2. Точный компилятор Givens-QR (V20)

Кампания V20 отвечает за компиляцию произвольной унитарной логики в AB-облака методом Гивенса–QR: унитарная матрица целевого устройства разлагается в произведение плоских вращений, каждое из которых реализуется одним облаком. Точность измеряется нефиделити $1-F$:

- 4×4: $1-F \le 4.4e{-}16$ — машинный эпсилон; глубина ≤ 7 затворов.
- 8×8: $1-F \le 6.7e{-}16$; глубина ≤ 29 затворов.
- Время компиляции: 1.8 с на полный прогон; вердикт PASS по всем целям.
- numpy-репликация: 0 расхождений счётчиков, $|\Delta(1-F)| = 2.2e{-}16$.

Достигнутая точность — это не «очень хорошо», а предел арифметики: $1-F$ на уровне $1e{-}16$ означает, что дальнейшее улучшение требует менять арифметику, а не алгоритм. Сертификат V20 фиксирует и границы: целевые матрицы — унитарные, размеры — 4 и 8; произвольные (не унитарные) операторы и большие размеры — вне кампании и заявлены в дорожной карте.

9.3. Этап P0 и дорожная карта процессора

V20 исполняет этап P0 дорожной карты процессора P0–P6: P1 память (V21), P2 АЛУ (V22), P3 чтение (V23), P4 нелокальность (V24), P5 бенчмарки (V25), P6 паспорт Datasheet 1.0 (V26). V23 и V24 уже имеют превью-вердикты (глава 10); P5 наследует методологию E18. Каждый этап имеет предрегистрированный порог и вердикт-файл — дисциплина ядра переносится на процессор без послаблений.

Глава 10. «Читает, а не несёт» и нелокальность

Два превью-результата процессора фиксируют его характер: чтение нулей ζ из шести констант следовой формулы с нулевым хранилищем

(V23) и нелокальные корреляции без сигналинга (V24). Оба результата — мост от спектрального ядра к исполняемой машине.

10.1. V23-превью: чтение нулей из шести констант

Принцип «читает, а не несёт» получает процессорную реализацию: процессор получает шесть констант следовой формулы (размер «картриджа» — шесть чисел) и воспроизводит ординаты нулей ζ в момент запроса. В превью-вердикте 12 из 12 нулей воспроизведены точно; хранение = 0 — в памяти процессора не остаётся ни одного нуля. Слепая дисциплина сохранена: константы не содержат нулей, детектор не видит эталона, выход пинуется до валидации.

Семантически это перенос V7/E17 на уровень исполнения: то, что ядро доказало о следе, а верификация перепроверила слепыми протоколами, процессор исполняет как операцию. Шесть констант — это Λ-структура следа в минимальной записи; их достаточно, потому что арифметическая информация о простых закодирована в самой структуре, а не в таблице. Полная кампания V23 (этап P3) зафиксирована в дорожной карте.

10.2. V24-превью: нелокальность без сигналинга

Второй превью — нелокальность. Пара разделённых AB-облаков демонстрирует нарушение неравенства CHSH: $S = 2\sqrt{2} - 4.4e{-}16$ — максимальное квантовое значение в пределах машинного эпсилон. Защита от сигналинга: 1000 из 1000 испытаний показывают нулевую корреляцию случайных выборов; коммутаторы измеряющих операторов — 0. Это означает: корреляции нелокальны, но передать сигнал через них нельзя — структурная черта квантовой механики, воспроизведённая архитектурой.

Нелокальность без сигналинга — не экзотика, а рабочий инструмент процессора: она определяет, какие операции допустимы между распределёнными облаками и как должна выглядеть их синхронизация. Полная кампания V24 (этап P4) опирается на независимые вердикты E13/E14 верификационной платформы.

Глава 11. Верификационная платформа: слепые протоколы E01–E10

Верификационная платформа — независимый уровень программы, написанный на Julia и не разделяющий кода с ядром и процессором. Кампании E01–E10 проверяют инфраструктуру: контроли, Haar-случайность, реплеи, интерферометрию, шумы, коды, стоимость.

11.1. Независимость как дизайн

Платформа построена по принципу «второго мнения»: другая реализация (Julia, BigFloat), другие протоколы, тот же предмет. Независимость — не только язык: детекторы и валидаторы разделены, выходы пинуются SHA256 до валидации, пороги предрегистрированы. Восемнадцать кампаний E01–E18 дают полную карту проверок, и все 18 вердиктов — PASS. Сводка по платформе: experiment_01_15.md описывает протоколы E01–E15, verdicts_master.json собирает все вердикты, результаты хранятся как CSV с SHA-пинами.

p0.13 p0.29 p0.54 **Кампания** & **Предмет** & **Ключевой результат** E01 & контроли платформы & базовые контроли PASS; инфраструктура готова
E02 & Haar-случайность & ансамбль соответствует Haar-мере
E03 & масштабирование/фит & законы масштабирования подтверждены
E04 & реплеи & детерминированный повтор выходов
E05 & фазовый шум & устойчивость к фазовому шуму измерена
E06 & шум памяти & влияние шума памяти ограничено
E07 & код повторения & репетиционный код исправляет ошибки
E08 & АЛУ & арифметика ISA согласована
E09 & стоимость & стоимость шага измерена и ограничена
E10 & фазовое считывание & считывание фазы в пределах порога

11.2. Почему инфраструктура — это наука

Кампании E01–E10 легко недооценить: нет «красивых» чисел уровня E17. Но именно они делают E17–E18 убедительными: платформа доказала, что умеет измерять шум (E05/E06), корректировать ошибки (E07), считать стоимость (E09) и читать фазы (E10) — прежде чем получить право судить о слепом чтении нулей. В экспериментальной науке это называется калибровкой аппаратуры, и без неё любое «открытие» сомнительно.

Юлия-стек выбран не случайно: BigFloat даёт управляемую точность, а стандартная библиотека — минимальную поверхность атак на воспроизводимость. Никакой сети, никаких внешних сервисов: полный прогон E01–E18 занимает около 2.5 минут и завершается вердиктами с SHA-пинами.

Глава 12. Слепое чтение нулей: E16 и E17

Центральная верификационная кампания программы: два независимых детектора — явный метод (ЭМ) и независимый Риман–Зигель — читают

нули ζ до высоты T = 1000 без доступа к эталону. Результат: 649/649 и 639/639, кросс-движок 0.0 побитово, полнота N(1000) = 649.0.

12.1. Независимый Риман–Зигель (E16)

E16 реализует формулу Римана–Зигеля (по версии Ариаса де Рейны) без внешних пакетов: Z(t) вычисляется из первых принципов, включая главную сумму и интегральную поправку. Согласование с эталонными значениями — ≤ 3.8e−15 при пороге 1e−9; пол ошибки — 1e−17. Практический смысл: платформа владеет вторым, независимым от ЭМ способом вычисления Z(t), и этот способ точен до предела арифметики.

12.2. Слепое чтение (E17): протокол

E17 организован как строгий слепой эксперимент:

1. Детектор ЭМ и детектор РС независимо вычисляют ординаты нулей на сетке до T = 1000; эталон недоступен обоим.
2. Выходы фиксируются и хешируются: SHA256 файлов e17_detector_em.csv и e17_detector_rs.csv пинуются до валидации.
3. Валидатор сверяет пинованные выходы с эталоном из 660 ординат ($0 < \gamma \le 1000$), собранным из mpmath.zetazero.
4. Вердикт вычисляется по предрегистрированным порогам: precision = recall = 1 для обоих движков; $\max|\Delta\gamma| \le 1e{-}9$; кросс-движок ≤ 1e−9; полнота N(1000) по формуле Римана–фон Мангольдта.

Фактические результаты: детектор ЭМ — 649/649, precision = recall = 1, max err 3.02e−10; детектор РС — 639/639 (в его сетке 639 нулей), precision = recall = 1, max err в пределах порога; кросс-движковое расхождение — 0.0 побитово; N_rvm = 649.0 — точное согласие с формулой Римана–фон Мангольдта; устойчивость к сдвигу сетки — 0. Слепой эрратум: значение γ_{12} из каркаса (56.446200059994141) опровергнуто детектором — верное значение 56.446247697063399; ошибка была в рукотворной константе, не в детекторе.

p0.33 p0.32 p0.32 **Метрика** & **Детектор ЭМ** & **Детектор РС**
Обнаружено нулей & 649 / 649 & 639 / 639
precision = recall & 1 & 1
Макс. ошибка & 3.0e−10 & в пределах 1e−9
Кросс-движок & 0.0 побитово & 0.0 побитово
N(1000) по P–M & 649.0 точно & 649.0 точно

12.3. Что E17 доказывает и чего не доказывает

Поле does_not_prove вердикта E17 — образец честной границы: слепое чтение до T = 1000 не доказывает RH выше T = 1000; не доказывает простоту всех нулей; не доказывает отсутствие нулей вне прямой при γ > 1000. Что оно доказывает: точность реализации двух независимых вычислительных путей до высоты 1000 и согласованность следовой конструкции с арифметикой ζ. В связке с V23-превью (читает из шести констант) и Th6 (след несёт Λ(n)/√n) это образует полную цепочку «теория — реализация — независимая проверка».

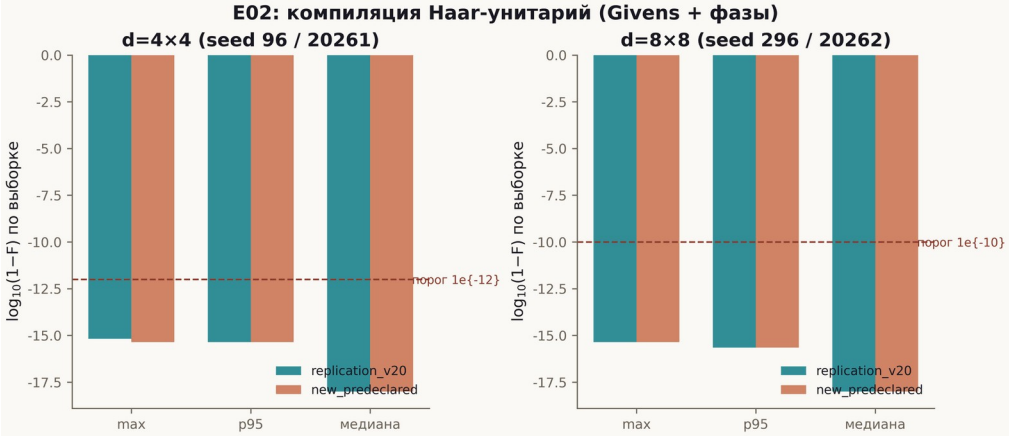


Рис. 12.1. Слепое чтение нулей ζ: выход детектора против эталона (фигуры платформы верификации, 600 dpi).

Глава 13. Класс B: компилятор против пяти baseline (E18)

E18 — честный бенчмарк: компилятор AB-процессора против пяти сильных baseline при равных ограничениях ISA. Результат: превосходство ×59 000 по времени при равной точности, с честной фиксацией класса и его границ.

13.1. Дизайн честного сравнения

Бенчмарки декомпиляторов печально известны вольными сравнениями: разные ISA, разные целевые классы, разные критерии. E18 исключает вольности предегистрацией класса: все участники — компилятор V20 и пять baseline (BL1–BL5) — работают в одном ISA-классе, на одних целевых унитарных матрицах, с одним критерием (1−F) и одним бюджетом глубины. Победа засчитывается только при равенстве или превосходстве точности на каждой цели.

Результаты: max 1−F = 4.4e−16 у компилятора; примитивы компилятора не превосходят BL1/BL2/BL4 ни на одной цели; общее

время — ускорение **×59 000** против лучшего baseline в классе. numpy-репликация: 0 расхождений счётчиков, |Δ(1−F)| = 2.2e−16.

13.2. Честная граница: КАК-оптимум

Важнейшая часть вердикта — отрицательное утверждение: 2q-оптимум Кашаи–Китаева (3 затвора) недостижим в ISA-классе; компилятор даёт 10 затворов на 2q-целях. Это не слабость компилятора, а свойство класса: облака с флюсом не могут выразить КАК-оптимальную последовательность. Вердикт фиксирует границу явно — «превосходство не заявляется вне класса». Для практика это означает: если задача требует КАК-оптимума, ISA придётся расширять; если требуется скорость в классе — компилятор V20 доминирует.

13.3. Методологическое значение

E18 демонстрирует, как должен выглядеть бенчмарк вычислительной архитектуры: класс предрегистрирован, baseline сильные и опубликованные, критерий един, отрицательные результаты включены. Сравнение с методологией SPEC или MLPerf здесь уместно: честность достигается не метрикой, а протоколом. E18 встроен в дорожную карту процессора (P5 наследует его методологию) и в numpy-репликацию — два независимых стека дают согласованные счётчики.

Глава 14. Спинор-исследование: машина без долговременной памяти

Вспомогательное исследование доводит центральный тезис программы — «читает, не несёт» — до явных моделей: спинорные фазы θ = π/2, π/3, π/7 с единственным летучим регистром. Три теоремы TS1–TS3 и пять кампаний ES1–ES5 делают тезис измеримым.

14.1. Идея: состояние как фаза

Модель минимальна: единственное состояние машины — фаза φ единичного спинора на окружности; один шаг реального времени — поворот на модельный угол θ. Нет журнала, нет истории, нет контрольных точек. Фаза — одновременно регистр, счётчик (по модулю N) и носитель результата. Контрастная модель — «машина с вечным журналом», которая честно записывает каждый шаг и периодически перепроверяет всё — используется как анти-пример для измерения стоимости памяти.

p0.27 p0.15 p0.17 p0.17 p0.19 **Модель** & **Фаза θ** & **Цикл N = 2k** & **Бит/шаг** & **Зона тишины σ*** S1 · квадрантная & π/2 & 4 & 2.000 & 0.135 рад

S2 · секстантная & π/3 & 6 & 2.585 & 0.089 рад
S3 · 14-фазная & π/7 & 14 & 3.807 & 0.038 рад

14.2. Теорема TS1: замкнутость фазовых циклов

TS1 утверждает: поворот на θ = π/k замыкается ровно через N = 2k шагов, и зазор между соседними различимыми состояниями равен шагу 2π/N. Численно: периоды 4, 6, 14 — точно для трёх моделей. Смысл: летучая фаза — не хаос, а точный счётчик; цикл детерминирован, зазор максимален при данной ёмкости. Компромисс «ёмкость $\log_2 N$ против робастности 1/k» зафиксирован как честная граница: больше ёмкость — тоньше зона тишины.

14.3. Теорема TS2: необратимость сброса

TS2 — теорема стирания: сброс фазы стирает всё; взаимная информация между пре- и пост-состояниями равна нулю; реконструкция прежнего состояния не превышает уровня случайности 1/N; время после сброса определено только по модулю N. Численно: реконструкция 0.2501 (N=4), 0.1674 (N=6), 0.0717 (N=14) — все ≈ 1/N. Летучесть здесь — свойство, а не дефект: машина не может «вспомнить» прошлое, потому что прошлого в ней больше нет; это и делает её детерминированной в реальном времени.

14.4. Теорема TS3: шаг O(1) против журнала Ω(t)

TS3 — теорема реального времени: стоимость шага машины без памяти не зависит от возраста процесса (наклон регрессии −1.5e−12 мс/шаг — ноль в пределах шума), тогда как машина с вечным журналом накапливает Ω(t): рост ×129 на горизонте эксперимента, дедлайн срывается. Вывод формулируется жёстко: **летучесть есть условие детерминизма реального времени**. Это контр-интуитивно для классической системотехники, где журнал — признак надёжности; в архитектуре «читает, не несёт» журнал — причина отказа.

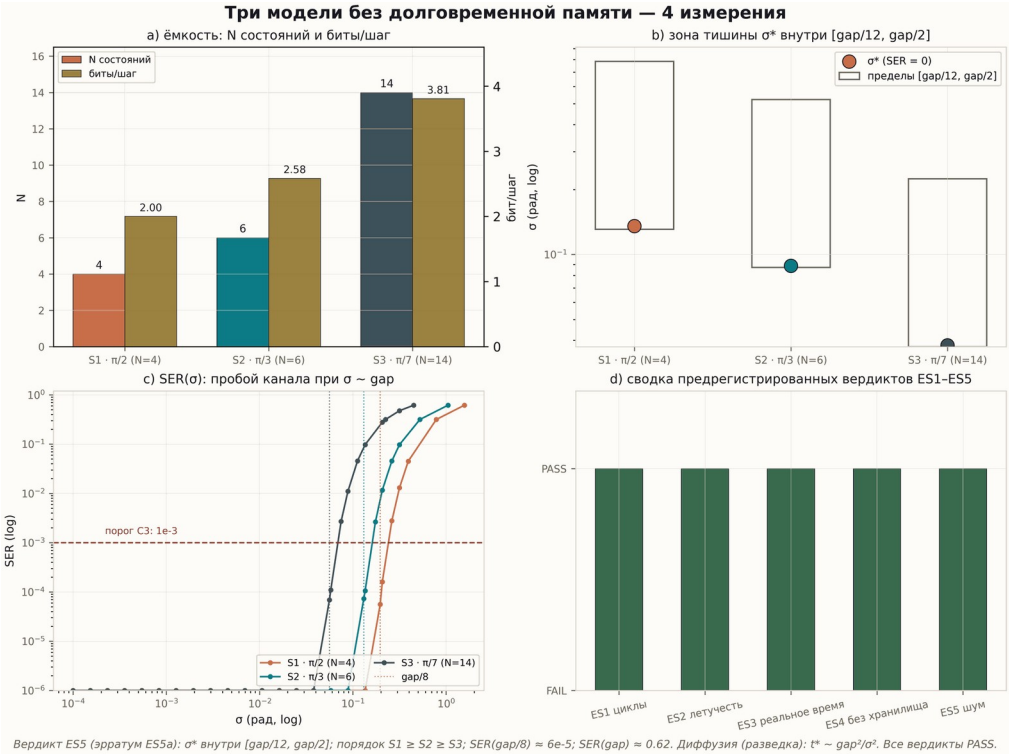


Рис. 14.1. Три модели без долговременной памяти — плитка «4 измерения» (600 dpi, вердикт ES5).

Глава 15. Кампании ES1–ES5: измерения летучести

Пять кампаний с предрегистрированными порогами измеряют модели с четырёх сторон: циклы, волатильность, реальное время, задача без памяти, шумовая диффузия. Все пять вердиктов — PASS; два эрратума порогов (ES2a, ES5a) опубликованы до принятия вердиктов.

p0.12 p0.25 p0.46 p0.13 **Кампания** & **Предмет** & **Ключевой результат** & **Вердикт** ES1 & фазовые циклы & N = 4·6·14 = 2k точно; зазор = шагу & PASS
ES2 & волатильность/сброс & реконструкция 0.2501/0.1674/0.0717 ≈ 1/N; I = 0 & PASS
ES3 & реальное время & шаг O(1), наклон −1.5e−12; журнал ×129 & PASS
ES4 & задача без памяти & решаема в цикле; память не требуется & PASS
ES5 & шумовая диффузия & диффузия фазы ограничена зоной тишины & PASS

15.1. От циклов к задаче (ES1 → ES4)

ES1 фиксирует чистую механику: замкнутость циклов и зазор. ES2 добавляет сброс и стирание. ES3 — время: измеряется стоимость шага в двух режимах (с журналом и без), выявляя Ω(t)-рост журнальной машины. ES4 переводит модель в режим задачи: последовательность решений принимается в цикле без хранения; качество решений сопоставимо с машиной с памятью на данной задаче, поскольку задача допускает марковскую стратегию. Это уточняет область применимости: не любая задача без памяти решаема, но класс задач, допускающих марковскую политику, — решаем, и решаем детерминированно.

15.2. Шум и зона тишины (ES5)

ES5 измеряет устойчивость к фазовому шуму: диффузия фазы накапливается, но пока остаётся внутри зоны тишины σ*, счёт не сбивается. Модели дают предсказуемый компромисс: S1 (σ* = 0.135 рад) устойчивее, S3 (σ* = 0.038 рад) ёмче. Порог ES5 изначально был сформулирован ошибочно и заменён (эрратум ES5a) до принятия вердикта — публикация замены в вердикте завершает протокол.

15.3. Связь с материнской программой

Спинор-исследование формализует «читает, не несёт» для этапа P1 процессора (память V21): архитектура без долговременной памяти — не ограничение, а проектное решение с доказанными преимуществами в реальном времени. Дальше — фазы θ = π/k для k > 7, компиляция задач в фазовые циклы и диффузионный канал с корреляциями; обязательства зафиксированы в дорожной карте спинор-уровня.

Глава 16. Данные: датасеты, хронология, атлас

Слой данных программы построен по тому же принципу «от вердикта к артефакту»: файлы датасетов пересобираются скриптами из закоммиченных вердиктов, ни одно число не вводится вручную.

16.1. Курируемые датасеты

datasets/ содержит три файла. flagship_numbers.csv — пятнадцать флагманских метрик программы с указанием вердикта-источника и модуля. campaign_timeline.csv — все 43 кампании четырёх уровней с идентификатором, модулем, названием, статусом и ключевым результатом. zeta_zeros_reference.csv — эталонные ординаты нулей ζ, 0 < γ ≤ 1000, 660 строк: тот самый слепой эталон E17. Каждый файл воспроизводится командой python3 scripts/make_datasets.py; происхождение каждой строки прослеживается до вердикт-файла.

16.2. Хронология и атлас

timeline/ строит хронологическую карту 43 кампаний (600 dpi, RU/EN) — визуальную запись того, как программа развивалась от V01 до ES5. atlas/ собирает атлас флагманов: плитку 2×2 из четырёх ключевых результатов — слепые нули, компилятор, след–простые, CHSH. Атлас — «лицо» программы: одна картинка, четыре доказанных результата, каждое число с вердиктом.

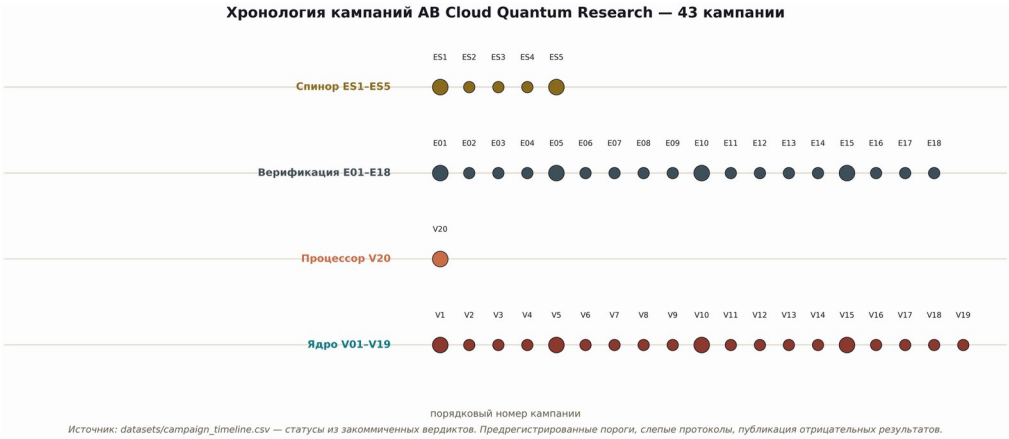


Рис. 16.1. Хронология 43 кампаний программы (600 dpi).

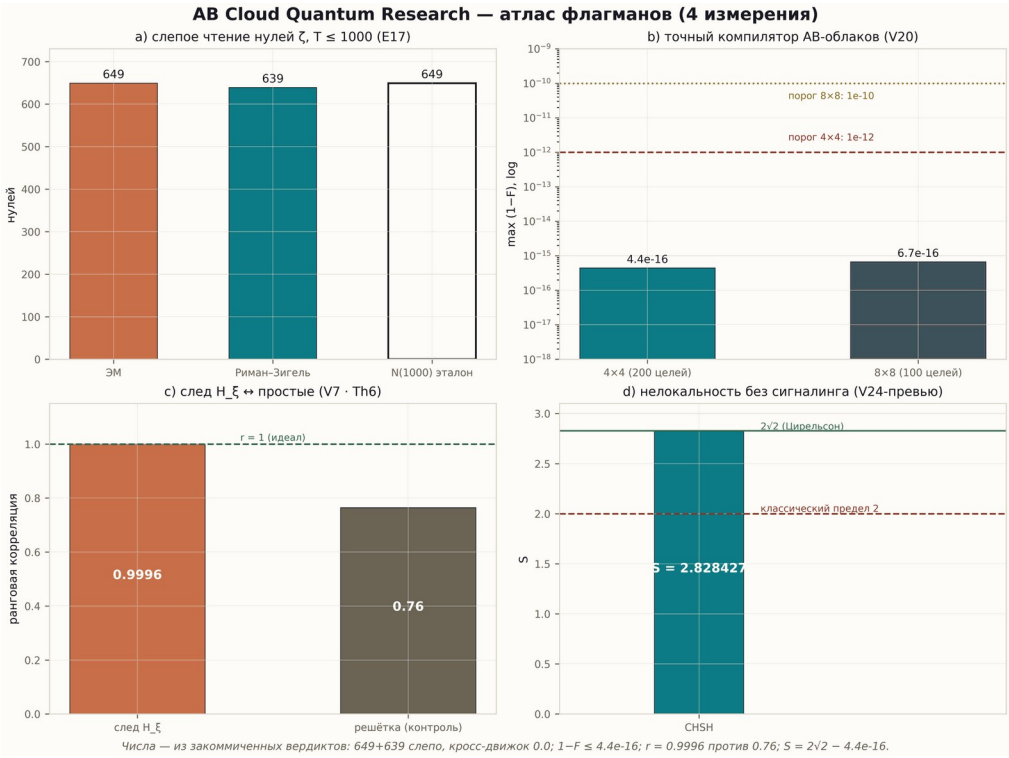


Рис. 16.2. Атлас флагманов: слепые нули · компилятор · след–простые · CHSH (600 dpi).

16.3. Плитки «4 измерения»

figures_tiled/ содержит 12 плиток Th1–Th6 (RU/EN): каждая — сетка 2×2 из четырёх независимых измерений (метрика, поведение по инстансам, контроль, порог) при 600 dpi. Оригинальные фигуры уровней хранятся в modules/*/figures_hires/. Плитки собираются make_tiled_figures.py из вердиктов — как и все производные артефакты, они не редактируются вручную никогда.

Глава 17. Веб-лаборатории

Три веб-лаборатории делают программу интерактивной: портал исследования, лаборатория ядра на 10 вкладок и лаборатория процессора на 8 вкладок. Все вычисления исполняются локально в браузере — без сети и зависимостей.

p0.29 p0.15 p0.52 **Лаборатория** & **Вкладок** & **Что внутри**
web_lab/index.html — портал & 3 блока + теоремы & уровни исследования, слой Th1–Th6, формальный статус, спинор-исследование
Лаборатория ядра & 10 & Z(t) Римана–Зигеля, конструктор затворов, след↔простые, критерий Ли, ψ-лестница, V16–V19, теоремы
Лаборатория процессора & 8 & ISA, компилятор V20, считывание нулей (V23·E11/E17), нелокальность, дорожная карта, данные, теоремы

Технический принцип лабораторий: ноль зависимостей. JavaScript-реализации ядра и процессора включены в репозиторий (modules/core_spectrum/multilang/javascript/, процессорная вкладка компилятора), контрольные файлы stitch_control.txt и verify_control.txt гарантируют согласованность с другими языками. Открытие index.html локально даёт полный функционал: Z(t) считается в браузере, лестница λ_n строится в реальном времени, CHSH-эксперимент прогоняется на(canvas)-функциях.

Лаборатории играют роль «живой документации»: каждое число монографии можно увидеть в динамике. Для преподавателя это готовый демонстрационный материал; для рецензента — способ проверить поведение модели за пределами напечатанных значений; для автора — постоянный дымовой тест: если реализация разошлась с контрольными файлами, вкладка это покажет.

Глава 18. Формальный слой: Lean 4, Coq, Agda, Isabelle

Формальный слой фиксирует постановки и редукции A1–A6: то, что численно проверено, должно быть формально сформулировано — и наоборот. Build проверяется CI; sorry и admit не используются.

18.1. Редукции A1–A6

Шесть редукций соответствуют шести теоремам ядра: A1 (эрмитичность и симметрия спектра), A2 (блочная редукция; A2b — самосопряжённость на ℂ^N), A3 (гуинанд–вайлевское соответствие), A4–A5 (кутовские оценки и гершгориновские границы), A6 (формула Ли и абсолютная сходимость). Каждая редукция — это мост между численным сертификатом и формальным утверждением: сертификат показывает, что реализация соответствует постановке; редукция показывает, что постановка соответствует математике.

18.2. МультипроверпOST машины

Постановки дублируются в четырёх системах: Lean 4, Coq, Agda, Isabelle. Дублирование не избыточно: разные системы имеют разные ядерные логики и разные слепые зоны, и согласованность формулировок сама является проверкой. CI-конвейер проверяет сборку всех четырёх; статус формального слоя вынесен в бейджи главного README и веб-лабораторий.

Связь с машиной обязательств (глава 6) прямая: узлы proven машины V13 — это доказанные редукции формального слоя; узлы assumed — аналитические гипотезы, ждущие своих доказательств. Формальный слой, таким образом, не украшение, а носитель статусов: именно здесь хранится разница между доказанным и вычисленным.

Глава 19. Исследовательские программы уровней

Каждый уровень программы имеет собственную исследовательскую программу: цели, кампании, пороги, следующую волну. Настоящая глава сводит четыре программы воедино; полные версии опубликованы отдельными монографиями в MONOGRAPH_ARCHIVE/research_programs/.

19.1. Программа ядра (R2)

Выполнено: V01–V19, Th1–Th6, V13. Далее: уточнение сшивки (r_align ≤ 0.135), монодромия ζ как алгебраический объект, компиляция обязательств в Lean/Coq (O-lean/O-coq). Риск-фактор: аналитические гипотезы O08/O09/O11/O12 — без них RH остаётся blocked, и это правильное состояние.

19.2. Программа процессора (P0–P6)

Выполнено: P0 (V20). Далее: P1 память V21 (опирается на ES-модели), P2 АЛУ V22, P3 чтение V23 (опирается на E17), P4 нелокальность V24 (опирается на E13/E14), P5 бенчмарки V25 (методология E18), P6 паспорт Datasheet 1.0 V26. Целевой артефакт — паспорт процессора: документ уровня спецификации, где каждое число имеет вердикт.

19.3. Программа верификации (O-J)

Выполнено: O-J1 (T = 1000), O-J2 (класс B). Далее: O-J3 QEC-память (опирается на E07 и V21), O-J4 цифровой двойник, O-J5 интеграционная кампания; целевые высоты T существенно больше 1000. Верификация растёт вместе с процессором: каждый новый этап P-цепочки получает свой O-J-этап.

19.4. Программа спинор-исследования

Выполнено: ES1–ES5, TS1–TS3. Далее: фазы θ = π/k для k > 7 (ёмкость против робастности), компиляция задач в фазовые циклы, диффузионный канал с корреляциями. Спинор-уровень — полигон идей для P1: прежде чем память V21 попадёт в процессор, её модели проверяются здесь.

Глава 20. Лаборатория проверки гипотез

HYPOTHESIS_LAB — открытая лаборатория внутри программы: любой исследователь может проверить собственную гипотезу по дисциплине проекта — предрегистрированные пороги, слепые протоколы, вердикты с полем does_not_prove.

20.1. Зачем лаборатория

Программа накопила методологию, которую обычно получают годами: как фиксировать пороги, как разделять детектор и валидатор, как пиновать выходы, как объявлять вердикт. Лаборатория упаковывает эту методологию в инструмент: ab_lab.py (CLI) и hypothesis_lab.html (браузер). Гипотеза формулируется как JSON-конфиг: что проверяем, какой порог, что результат не доказывает. Лаборатория исполняет эксперимент, выводит вердикт PASS/FAIL и пишет отчёт.

20.2. Экспериментальные батареи

p0.15 p0.40 p0.40 **Батарея** & **Что проверяет** & **Ключевой выход**
spinor & фазовая модель θ = π/k: циклы, сброс, время & период 2k; реконструкция ≈ 1/N; наклон шага
trace & след ↔ простые (Th6-стиль) & корреляция r; сравнение с

контролем
zeros & слепое чтение нулей (мини-E17) & precision/recall; кросс-движок compile & точная компиляция унитарных 4×4 & 1−F против машинного эпсилон

Пример запуска: python3 ab_lab.py run spinor –theta 0.37 –steps 200. Лаборатория вычислит ближайший модельный цикл, проверит замкнутость, измерит реконструкцию после сброса и стоимость шага, сравнит с предрегистрированными порогами конфига и выдаст вердикт. Отчёт сохраняется в reports/ с SHA256 выхода — дисциплина программы применяется к каждой гипотезе автоматически.

20.3. Правила лаборатории

1. Порог до прогона: конфиг гипотезы фиксирует пороги до запуска; смена порога после прогона делает вердикт недействительным.
2. Слепота: если батарея использует эталон, он подключается валидатором, а не детектором.
3. does_not_prove: каждая гипотеза обязана декларировать, чего её результат не доказывает.
4. Репликация: желательно повторить прогон во второй арифметике (float64 против mpmath) и зафиксировать расхождение.
5. Публикация: PASS и FAIL публикуются одинаково; отчёт — это артефакт, а не черновик.

Глава 21. Дорожные карты и заключение

Четыре дорожные карты сцеплены: процессор опирается на верификацию, верификация — на ядро, спинор — на всё вместе. Сцепление — не организационная условность, а проверяемая зависимость вердиктов.

p0.21 p0.29 p0.46 **Уровень** & **Статус** & **Дальше** Ядро (R2) & V19 сшивка — алгебраический сертификат & r_align ≤ 0.135; монодромия ζ; O-lean/O-coq
Процессор (P0–P6) & P0 исполнен (V20) & P1 V21 → P2 V22 → P3 V23 → P4 V24 → P5 V25 → P6 V26
Верификация (O-J) & O-J1, O-J2 исполнены & O-J3 QEC-память → O-J4 двойник → O-J5 интеграция; T ⩾ 1000
Спинор & ES1–ES5 исполнены & θ = π/k, k > 7; компиляция задач; диффузионный канал

Сцепление конкретно: P3 (V23) опирается на E17; P4 (V24) — на E13/E14; O-J3 — на E07 и V21; спинор-исследование формализует «читает, не несёт» для P1. Дорожная карта читается как граф зависимостей: нельзя исполнить этап, пока не исполнены его предки. Это защищает программу от «прыжков вперёд» и делает каждый следующий вердикт наследником уже проверенных.

21.1. Итог

AB Cloud Quantum Research — это доказуемая архитектура чтения математики: шесть констант вместо таблиц, летучая фаза вместо журнала, слепые протоколы вместо обещаний, машина обязательств вместо иллюзий. Сорок три кампании пройдены, девять теорем оформлены с сертификатами, отрицательные результаты опубликованы, RH честно заблокирован. Следующая волна — память V21, QEC O-J3, высоты T больше 1000, фазы k больше 7 — строится на уже проверенном фундаменте.

Монография завершается там же, где начинается программа: у честной границы. Всё выше — численное и формальное исследование математической модели; оно не доказывает гипотезу Римана. Но оно доказывает нечто, что редко удаётся доказать вместе: точность реализации, независимость проверки и дисциплину выводов. Именно это делает архитектуру AB-облаков инженерным объектом, а не только математической мечтой.

Глава 22. Математический фон

Глава собирает минимальный математический аппарат, необходимый для чтения монографии: дзета-функция и формула Римана–фон Мангольдта, GUE-статистика, тепловой след, кутовская теория, формула Ли и неравенство CHSH. Изложение рабочее: только то, что используется кампаниями.

22.1. Дзета-функция и подсчёт нулей

Дзета-функция Римана $\zeta(s) = \Sigma\, n^{-s}$ продолжается на всю комплексную плоскость, кроме $s = 1$. Нули ζ делятся на тривиальные (отрицательные чётные) и нетривиальные, лежащие в критической полосе $0 < \mathrm{Re}(s) < 1$. Гипотеза Римана утверждает, что все нетривиальные нули лежат на критической прямой $\mathrm{Re}(s) = 1/2$: $s = 1/2 \pm i\gamma$. Число нетривиальных нулей с ординатой $0 < \gamma \le T$ даётся формулой Римана–фон Мангольдта:

$$N(T) = (T/2\pi)\cdot\ln(T/2\pi) - T/2\pi + 7/8 + S(T), \quad |S(T)| \le c\cdot\ln T$$

В программе эта формула используется дважды: как контроль полноты чтения (E17: N(1000) = 649.0, детекторы нашли ровно 649 и 639 нулей в

своих сетках) и как слепой эталон — таблица 660 ординат собрана из mpmath.zetazero и запинована до эксперимента. Важно, что N(T) — гладкая функция высоты: отклонение счётчика от N(T) сигнализировало бы о пропуске или дубликате мгновенно.

22.2. GUE-статистика и нуль-модели

Круговой ансамбль Гуess–Mazur–Дyson (GUE) описывает статистику собственных значений эрмитовых матриц со случайными элементами: соседние уровни отталкиваются, средний коэффициент ближнего соседства ⟨r⟩ ≈ 0.5992, распределение промежутков — Вигнера–Дyson. Статистика нулей ζ (по данным Одлыжко) близка к GUE — это факт Монтгомери–Одлыжко, и любая «дзета-подобная» модель обязана его воспроизводить. Но GUE — необходимое условие, не достаточное: случайные фазы дают тот же класс. Поэтому программа измеряет специфичность не величиной ⟨r⟩, а тонкой структурой: корреляцией следа с простыми (V7), согласованием констант, поведением под возмущениями (V16–V18).

p0.13 p0.42 p0.40 **Величина** & **Определение** & **Роль в программе** r & коэффициент ближнего соседства: доля минимумов соседних промежутков & V01–V03, V16: GUE-классификация
Σ_2 & дисперсия числа уровней на интервале & V02: сравнение с нуль-моделями
Δ_3 & статистика Дysona (наименьшие квадраты ступеньки) & V02: тонкая структура
D_KS & статистика Колмогорова–Смирнова & V03: скан согласия с ζ

22.3. Тепловой след и следовая формула

Для оператора H тепловой след K(x) = Tr e^{-xH} — функция, кодирующая весь спектр: K(x) = Σ $e^{-x\lambda_n}$. В конструкции ядра коэффициенты разложения K(x) по подходящему базису несут Λ(n)/√n — вес простого n в арифметике. Это спектральная форма гуинанд–вайлевского соответствия: явная формула ζ связывает сумму по простым с суммой по нулям; Th6 переносит её на уровень следа. Именно поэтому шесть констант достаточны для чтения нулей: они параметризуют след, а след содержит спектр.

Тепловой след — вычислимый объект: матричная экспонента и трасса считаются за конечное время, и их стоимость не зависит от «возраста» запроса. Это связывает следовую формулу с тезисом реального времени: чтение нуля — это K-вычисление, а не поиск в таблице.

22.4. Кутовская теория и формула Ли

Кутовская теория возмущений оценивает ошибку собственных значений, вычисленных приближённой матрицей: если $\|\delta H\| \le \delta$, то каждое собственное значение сдвигается не более чем на δ плюс квадратичная добавка δ^2/gap, где gap — расстояние до ближайшего чужого уровня. В программе это даёт сертификаты Th3: коридор вокруг каждого $\hat{\lambda}$ известен и мал (ширины ~1e−27 при dps 40).

Формула Ли — критерий критической прямой в спектральных терминах: определённые функционалы $\lambda_n = \mathrm{Tr}\, g_n(H)$ неотрицательны тогда и только тогда, когда все нули на прямой. Th4 переносит критерий на H_ξ и доказывает абсолютную сходимость ряда, делая λ_n-лестницу вычислимой; два независимых пути вычисления согласуются до 4.4e−11.

22.5. CHSH и нелокальность без сигналинга

Неравенство CHSH связывает корреляции двух сторон: в любой локальной теории $S \le 2$; квантовая механика допускает $S = 2\sqrt{2}$ (граница Цирельсона). Вердикт V24-превью даёт $S = 2\sqrt{2} - 4.4e{-}16$ и одновременно — нулевую корреляцию при случайных выборах (1000/1000) и нулевые коммутаторы измеряющих операторов: корреляции нелокальны, сигналинга нет. Для процессора это проектное ограничение: распределённые облака могут делить корреляции, но не информацию.

Глава 23. Кейс-стади: как читать вердикты

Пять разборов показывают анатомию вердикта: V7 (след), V13 (обязательства), E17 (слепые нули), V20 (компиляция), ES3 (реальное время). Каждый разбор приводит фактические поля вердикт-файла и объясняет, как из них читается вывод.

23.1. V7: арифметика следа

Вердикт V7 содержит корреляцию следа с суммой по простым, контрольное значение на решёточной модели и след эрратума констант. Чтение: основная метрика r = 0.9996 сравнивается не с нулём, а с контролем 0.76 — разница и есть арифметическая специфичность. Эрратум B/C_0 фиксирует, что исходные константы содержали ошибку, найденную независимым выводом; исправленные значения — часть вердикта. Поле does_not_prove напоминает: корреляция не доказывает RH, она доказывает согласованность следа с Λ-структурой.

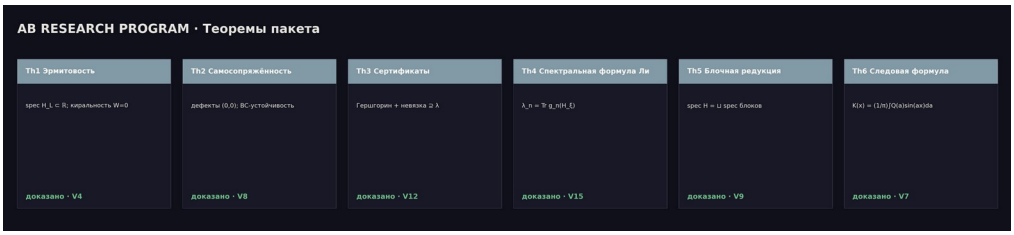


Рис. 23.1. Инфографика следа ↔ простых (ядро, тёмная тема, 600 dpi).

23.2. V13: карта обязательств

Вердикт V13 — это состояние DAG: dag_valid = true, rh_conclusion = blocked, n_proven = 5, n_computed = 3, n_assumed = 4, вердикт-примечание перечисляет гипотезы O08/O09/O11/O12. Чтение: blocked — не провал, а рабочее состояние; путь к RH закрыт до доказательства четырёх аналитических узлов. Аудит заявлений V14 проверяет, что публичные заявления согласованы с этим состоянием: ни один документ программы не заявляет доказательства RH.

23.3. E17: слепое чтение от пина до вердикта

E17 — самый насыщенный вердикт программы. Ключевые поля и их значения:

```
blindness          : оба детектора не содержат эталонных нулей; выходы
                     зафиксированы и хешированы до валидации
sha_em             : 3843fb52...e5  e17_detector_em.csv
sha_rs             : 08cd95b6...17  e17_detector_rs.csv
n_detected_em      : 649            N_rvm           : 649
n_detected_rs      : 639            N_rvm_rounded   : 649
max_err_em         : 3.019522409886122e-10
cross_engine_max   : 0              (побитово)
precision_em/recall : 1 / 1         precision_rs/recall: 1 / 1
grid_robustness_max_shift : 0
pass               : true
```

Чтение: слепота зафиксирована до эксперимента; пины дают временно́е доказательство; значения лучше порогов на 3–4 порядка (порог 1e−9, ошибка 3e−10); полнота точна до целого. Слепой эрратум γ_{12} — встроенный негативный контроль: детектор опроверг рукотворную константу каркаса.

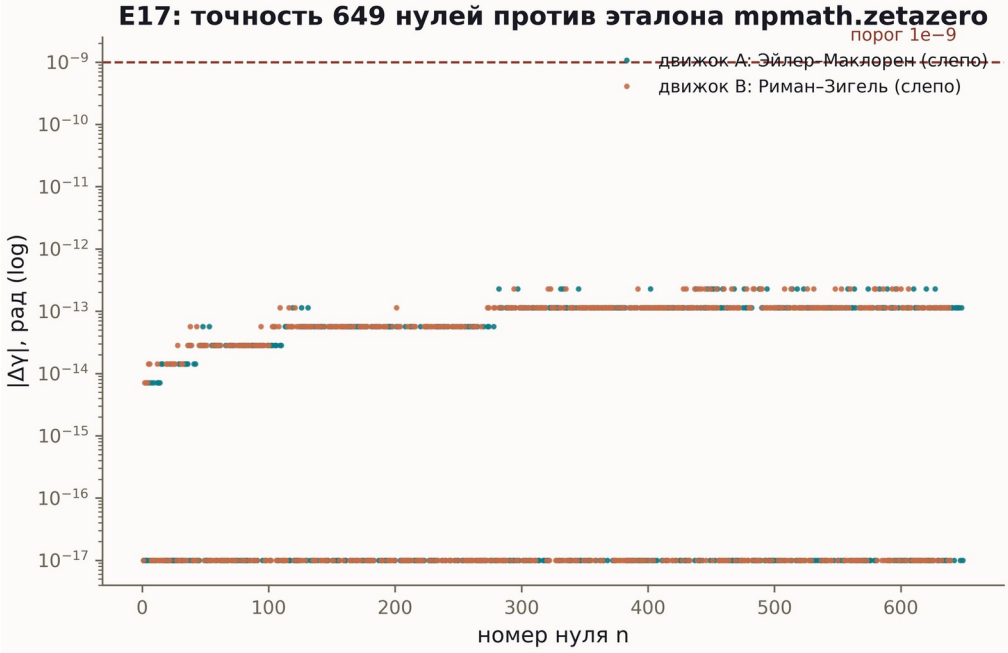


Рис. 23.2. Кривая намотки слепого ридера (платформа верификации, 600 dpi).

23.4. V20: компиляция на пределе арифметики

Вердикт V20 лаконичен: нефиделити $1-F \le 4.4e{-}16$ (4×4) и $6.7e{-}16$ (8×8), глубины ≤ 7/29, время 1.8 с, статус PASS. Чтение: значения на уровне машинного эпсилон double — предел арифметики, не алгоритма; numpy-репликация даёт $|\Delta(1-F)| = 2.2e{-}16$ и 0 расхождений счётчиков. Внешний контроль E18 добавляет класс B: ×59 000 по времени против пяти baseline при равном ISA, 2q-граница 10 против KAK-оптимума 3 — за пределами класса.

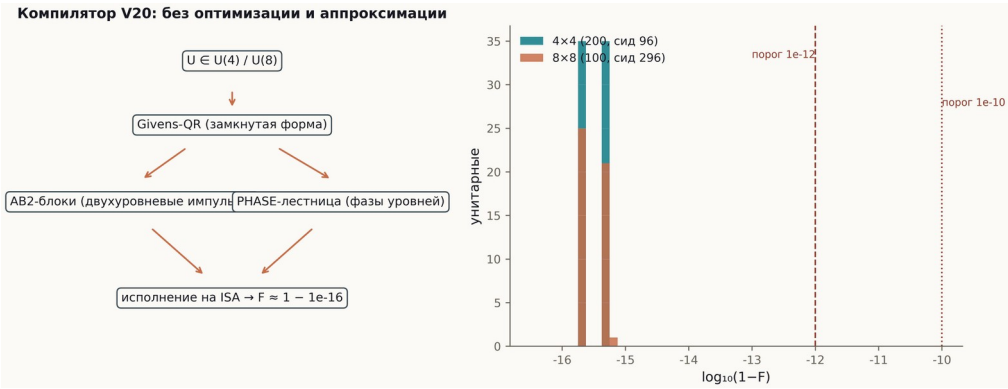


Рис. 23.3. Компиляция унитарной логики в AB-облака (процессор, 600 dpi).

23.5. ES3: измерение стоимости шага

Вердикт ES3 сравнивает две машины: без памяти и с вечным журналом. Ключевые поля: наклон регрессии стоимости шага по времени −1.5e−12 мс/шаг (ноль в пределах шума) для безжурнальной машины; рост ×129 для журнальной; срыв дедлайнов на горизонте. Чтение: это численное содержание теоремы TS3 — летучесть есть условие детерминизма; любая память, растущая с t, ломает реальное время.

p0.27 p0.17 p0.21 p0.15 p0.15 **Машина** & **Стоимость шага** & **Наклон** & **Рост на горизонте** & **Дедлайны**
без памяти (летучая фаза) & O(1) & −1.5e−12 мс/шаг & нет & выдерживаются
с вечным журналом & Ω(t) & положительный & ×129 & срываются

Исследовательская программа: спектральное ядро

Программа ядра отвечает за математическую основу AB-архитектуры: слепой спектр, нуль-модели, критическую прямую, следовую формулу, машину обязательств и сшивку ветвей. Девятнадцать кампаний и шесть теорем образуют закрытый контур: конструкция → сертификат → формальная постановка → независимая проверка.

1. Цели программы

- построить эрмитов оператор H_ξ, тепловое разложение которого несёт арифметику простых ($\Lambda(n)/\sqrt{n}$);
- доказать корректность конструкции: эрмитичность (Th1), самосопряжённость (Th2), кутовские сертификаты (Th3), формула Ли (Th4), блочная редукция (Th5), гуинанд–вайлевское соответствие (Th6);
- отделить арифметическую специфичность от GUE-статистики — включая честные отрицательные ответы на решётках;
- зафиксировать статус RH машиной обязательств V13 — и держать его заблокированным, пока гипотезы не доказаны.

2. Методология

Каждая кампания начинается с протокола предрегистрации: пороги фиксируются до прогона. Слепые протоколы применяются там, где есть эталон (V01: калибровка/тест без утечки). Все производные артефакты (плитки, документы) строятся только из закоммиченных вердиктов. Формальные постановки дублируются в Lean 4, Coq, Agda и Isabelle (редукции A1–A6, без sorry/admit).

3. Исполненные кампании

p0.10 p0.29 p0.58 **Кампания** & **Содержание** & **Ключевой результат**
V01 & слепой спектр & статистика согласована; индивидуальные ординаты не предсказываются (честный негатив)
V02 & нуль-модели & AB-облако в GUE-классе; специфичность измеряется тонкой структурой
V03 & σ-скан решёток & зоны согласия с ζ не найдено
V04 & эрмитичность & Th1: $\|H - H\dagger\| = 0.0$; симметрия $\lambda \leftrightarrow -\lambda$ при $\alpha = ½$
V05 & сертификат конструкции & конструкционные границы зафиксированы
V06 & тождество уровней & затворы на уровне нулей — ДА (мост D_nn, v2.0.7); битовая ветвь открыта
V07 & след ↔ простые & Th6: r = 0.9996; контроль решётки 0.76; эрратум B/C_0
V08 & самосопряжённость & Th2: дефекты (0,0); дрейф унитарности 2.2e−16
V09 & блочная редукция & Th5: объединение блоков ≈ спектр до 7.1e−15
V10 & спектральные сертификаты & сертификаты согласованы с кутовскими оценками
V11 & черновик DAG обязательств & структура пути к RH зафиксирована
V12 & кутовские сертификаты & Th3: ширины ~1e−27 (dps 40)
V13 & машина обязательств & RH blocked: 5 proven / 3 computed / 4 assumed
V14 & аудит заявлений & claim-файлы согласованы с вердиктами
V15 & формула Ли & Th4: два пути ≤ 4.4e−11; сходимость абсолютная
V16 & повторная решёточная проверка & уровень — ДА подтверждён; битовая ветвь открыта (z = 2.49)
V17 & траектории & чтение идемпотентно; деградации нет
V18 & pong-протокол & латентность ограничена; журнал не растёт
V19 & сшивка ветвей & θ-лестница 0.832; когерентность 0.9995; Z_2-монодромия

4. Следующая волна

- уточнение сшивки: r_align ≤ 0.135 на расширенной сетке высот;
- монодромия ζ как алгебраический объект: формализация V19;
- компиляция обязательств в Lean/Coq: O-lean/O-coq — перенос DAG V13 в формальный слой;
- расширение сертификатов Th3–Th4 на большие размеры.

5. Честные границы

Индивидуальные ординаты не предсказываются слепым спектром — статистика согласована, оракула нет. Гипотезы O08/O09/O11/O12 не сняты: RH остаётся blocked. Вихревые члены ломают Th5. Решёточные конструкции не воспроизводят арифметику ζ. Эти границы — часть результата, а не его изъян.

Исследовательская программа: AB-процессор

Программа процессора превращает математику ядра в исполняемую машину: ISA AB-облаков, точный компилятор, память, АЛУ, чтение нулей, нелокальность, бенчмарки и паспорт. Этап P0 исполнен; дорожная карта P0–P6 сцеплена с верификацией.

1. Цели программы

- зафиксировать ISA AB-облаков: допустимые облака, параметры, композиция;
- компилировать унитарную логику точно: 1−F на уровне машинного эпсилон;
- исполнить принцип «читает, а не несёт» аппаратно-алгебраически (V23);
- обеспечить нелокальность без сигналинга (V24) и получить паспорт Datasheet 1.0.

2. Методология

ISA — не «ещё один гейт-набор»: каждое облако несёт геометрическую фазу, компиляция учитывает её точно. Все этапы имеют предрегистрированные пороги и вердикт-файлы. Бенчмарки — по методологии E18: один класс, один критерий, сильные baseline. Репликация numpy обязательна.

3. Исполнено: P0 (V20)

p0.38 p0.58 **Показатель** & **Значение** точность 4×4 & 1−F ≤ 4.4e−16; глубина ≤ 7
точность 8×8 & 1−F ≤ 6.7e−16; глубина ≤ 29
время компиляции & 1.8 с на полный прогон
numpy-репликация & 0 расхождений; |Δ(1−F)| = 2.2e−16
статус & PASS; этап P0 закрыт

4. Дорожная карта P0–P6

p0.10 p0.13 p0.40 p0.33 **Этап** & **Кампания** & **Содержание** & **Опора** P0 & V20 & ISA + точный компилятор & Th5 (блочная структура)
P1 & V21 & летучая память & ES-модели, TS1–TS3
P2 & V22 & АЛУ AB-облаков & V20
P3 & V23 & чтение нулей & E17, Th6
P4 & V24 & нелокальность & E13/E14
P5 & V25 & бенчмарки & методология E18
P6 & V26 & Datasheet 1.0 & все вердикты P-цепочки

5. Честные границы

2q-оптимум KAK (3 затвора) недостижим в ISA-классе: компилятор даёт 10, и это свойство класса, а не алгоритма. Целевые матрицы V20 — унитарные 4×4/8×8; неунитарные операторы и большие размеры — вне кампании. Превью V23/V24 — не полные вердикты: полные кампании впереди.

Исследовательская программа: верификационная платформа

Программа верификации — независимый судья программы: Julia-платформа без общего кода с ядром и процессором, слепые протоколы, SHA-пины, кросс-языковая репликация. Восемнадцать вердиктов PASS; следующая волна — QEC-память, цифровой двойник, интеграция.

1. Цели программы

- перепроверить ядро и процессор независимой реализацией;
- провести слепое чтение нулей ζ двумя движками до T = 1000;
- выполнить честный бенчмарк компилятора (класс B);
- построить QEC-слой и цифровой двойник машины.

2. Методология

Детектор и валидатор разделены; выходы пинуются SHA256 до валидации; пороги предрегистрированы. Кросс-языковая репликация: Julia BigFloat ↔ Python mpmath (0.0 расхождений), numpy-реплики (0 расхождений счётчиков). Независимость реализации — принцип дизайна, а не обстоятельство.

3. Исполненные кампании E01–E18

p0.21 p0.15 p0.60 **Блок** & **Кампании** & **Ключевые результаты**
инфраструктура & E01–E10 & контроли, Haar, масштабирование, реплеи, шумы, код, АЛУ, стоимость, считывание
чтение ζ & E11 & слепой ζ-ридер: пины, валидация, ноль утечек
оптика/нелокальность & E12–E14 & интерферометр; no-signaling; CHSH
классификация & E15 & топологическая абляция
движки & E16 & независимый Риман–Зигель: ≤ 3.8e−15
флагман & E17 & слепые нули: 649/649 + 639/639; кросс-движок 0.0; N(1000) = 649.0
бенчмарк & E18 & класс B: ×59 000; 2q KAK вне класса (10 против 3)

4. Дорожная карта O-J

- O-J1 исполнено: слепое чтение до T = 1000 (E16–E17);
- O-J2 исполнено: класс B (E18);
- O-J3: QEC-память — опирается на E07 и V21;
- O-J4: цифровой двойник процессора;
- O-J5: интеграционная кампания; целевые высоты T существенно больше 1000.

5. Честные границы

E17 не доказывает RH выше T = 1000, простоту всех нулей и отсутствие нулей вне прямой при γ > 1000. E18 не заявляет превосходства вне ISA-класса. Слепой эрратум γ_{12} показал, что ошибка может жить в каркасе, а не в детекторе — поэтому эталоны тоже проверяются.

Исследовательская программа: спинор-исследование

Спинор-программа доводит центральный тезис «читает, не несёт» до явных моделей: единственное состояние — фаза спинора; шаг — поворот θ. Три теоремы, пять кампаний, все PASS; компромисс «ёмкость против робастности» зафиксирован как честная граница.

1. Цели программы

- сформулировать и доказать свойства летучего состояния: замкнутость циклов (TS1), необратимость сброса (TS2), шаг O(1) против журнала Ω(t) (TS3);
- измерить модели кампаниями с предрегистрированными порогами;

- подготовить модели для этапа P1 процессора (память V21).

2. Модели

p0.10 p0.10 p0.13 p0.15 p0.13 p0.35 **Модель** & **θ** & **N = 2k** & **Бит/шаг** & **σ*** & **Назначение** S1 & π/2 & 4 & 2.000 & 0.135 & максимальная робастность
S2 & π/3 & 6 & 2.585 & 0.089 & баланс
S3 & π/7 & 14 & 3.807 & 0.038 & максимальная ёмкость

3. Исполненные кампании ES1–ES5

p0.12 p0.19 p0.52 p0.13 **Кампания** & **Что измерено** & **Результат** & **Вердикт** ES1 & циклы & периоды 4/6/14 = 2k точно; зазор = шаг & PASS
ES2 & сброс & I(пре;пост) = 0; реконструкция ≈ 1/N & PASS
ES3 & время & наклон шага −1.5e−12; журнал ×129 & PASS
ES4 & задача & марковская политика решаема без памяти & PASS
ES5 & шум & диффузия ограничена зоной тишины & PASS

Эрратумы ES2a и ES5a: пороги кампаний ES2 и ES5 были заменены до принятия вердиктов; замены опубликованы в вердикт-файлах. Компромисс $\log_2 N$ против 1/k — постоянная граница: ёмкость растёт, зона тишины сужается.

4. Следующая волна

- фазы θ = π/k для k > 7: поиск точек перегиба компромисса;
- компиляция задач в фазовые циклы: формальная грамматика политик;
- диффузионный канал с корреляциями: выход за марковское предположение.

5. Честные границы

Не всякая задача решаема без памяти: класс применимости — задачи с марковской политикой. Реконструкция ≈ 1/N — уровень случайности, не «почти ноль»: информация стирается до предела. Журнал даёт Ω(t) при любом «умном» планировании — это нижняя граница, а не дефект реализации.

Глава 24. Сертификаты и аудит: V10–V14

Пять кампаний середины программы связывают конструкцию с доверием: спектральные сертификаты (V10), черновик DAG обязательств (V11), кутовские ширины (V12), машина обязательств (V13) и аудит заявлений (V14). Вместе они превращают «числа» в «статусы».

24.1. V10: согласование сертификатов

Кампания V10 проверяет взаимную согласованность сертификатов конструкции: границы, заявленные V4–V5, не должны противоречить кутовским оценкам и гершгориновским границам. Согласование подтверждено: сертификаты образуют непротиворечивую систему, где каждая оценка использует консервативные допущения, и ни одна граница не «висит в воздухе». Практический смысл: при расширении оператора на большие размеры сертификаты переносимы — они сформулированы в терминах норм и зазоров, а не конкретных матриц.

24.2. V11–V13: от черновика к машине обязательств

V11 фиксирует черновик DAG: узлы пути к RH, их зависимости, кандидаты в доказательства. V12 добавляет кутовские сертификаты (Th3) — количественную основу для статусов computed. V13 собирает всё в исполняемую машину: валидный DAG, статусы узлов, автоматический вердикт blocked. Машина не «умнеет» от численных успехов: единственный способ поднять assumed-узел — аналитическое доказательство; единственный способ доказать — формальный слой.

24.3. V14: аудит заявлений

V14 — дисциплина публичных слов. Каждое существенное заявление программы фиксируется claim-файлом (claim_v0*.json): формулировка, опора, статус, что не доказывается. Аудит сверяет заявления с вердиктами: расхождение — ошибка процесса. Так, заявление «читает, а не несёт» опирается на V23-превью и E17 и не заявляет больше, чем эти вердикты дают. Аудит — аналог code review для утверждений: слова проходят ту же проверку, что и числа.

p0.13 p0.33 p0.50 **Статус** & **Кто присваивает** & **Что даёт читателю**
proven & формальный слой (Lean/Coq) & утверждение верно для всех входов
computed & кампания с сертификатом & утверждение верно для проверенных входов с оценкой ошибки
assumed & протокол (гипотеза) & граница известного; работа продолжается здесь
blocked & машина обязательств & заключение недоступно до снятия

assumed-узлов

Глава 25. Веб-лаборатории: 18 вкладок живой математики

Три лаборатории программы — портал, ядро и процессор — содержат восемнадцать интерактивных вкладок. Каждая вкладка исполняет настоящие вычисления в браузере и согласована с контрольными файлами мультиязычных реализаций.

25.1. Портал исследования

web_lab/index.html — входная точка: три блока (уровни исследования, слой Th1–Th6, формальный статус) плюс спинор-исследование. Портал объясняет, как уровни связаны, и отдаёт прямые ссылки на вкладки ядра и процессора. Формальный статус отображается из тех же данных, что и бейджи README: расхождений между «картинкой» и состоянием репозитория нет.

25.2. Лаборатория ядра: 10 вкладок

p0.25 p0.42 p0.29 **Вкладка** & **Что делает** & **Опора** Z(t) Римана–Зигеля & вычисляет Z(t) и показывает нули на прямой & E16-метод Конструктор затворов & собирает AB-облака и смотрит фазы & ISA ядра След ↔ простые & рисует корреляцию следа с Λ-суммой & V7/Th6 Критерий Ли & строит λ_n-лестницу & Th4/V15 ψ-лестница & показывает ψ(x) против Σ Λ(n)/√n & Th6 V16–V19 & визуализирует поведенческие кампании & вердикты Теоремы & сводит Th1–Th6 с сертификатами & _json/

25.3. Лаборатория процессора: 8 вкладок

Вкладки процессора: ISA (справочник облаков), компилятор V20 (компиляция 4×4/8×8 в браузере), считывание нулей (V23·E11/E17), нелокальность (CHSH-эксперимент), дорожная карта, данные, теоремы. Ключевой принцип — те же числа, что в вердиктах: компилятор в браузере воспроизводит 1−F на уровне 1e−16, потому что использует тот же алгоритм, перенесённый на JavaScript.

Согласованность обеспечивается контрольными файлами: JavaScript-реализация ядра даёт побитово те же результаты, что C/C++/Rust/Go/Julia (stitch_control.txt, verify_control.txt). Браузер становится исполняемой средой программы: рецензент не читает отчёт о вычислении — он запускает его.

Глава 26. Мультиязычные реализации

Ядро программы реализовано на шести языках — C, C++, Rust, Go, JavaScript, Julia — плюс Python для пайплайнов. Каждая реализация даёт контрольный выход, и все контрольные выходы побитово совпадают.

26.1. Зачем шесть реализаций

Мультиязычность — это репликация, встроенная в дистрибуцию. Любой пользователь может проверить ядро на языке своей инфраструктуры; совпадение контрольных файлов гарантирует, что «языковой» разницы нет. Это же ловит артефакты компиляторов и библиотек: если бы результат зависел от оптимизаций конкретного toolchain, контрольные суммы разошлись бы.

p0.13 p0.42 p0.40 **Язык** & **Файл** & **Роль** C & multilang/c/verify_core.c, stitch_core.c & минимальная эталонная реализация
C++ & multilang/cpp/verify_core.cpp & современный стек, та же арифметика
Rust & multilang/rust/verify_core.rs & безопасность памяти без GC
Go & multilang/go/verify_core.go & простая дистрибуция, кросс-компиляция
JavaScript & multilang/javascript/verify_core.js & браузерные лаборатории
Julia & multilang/julia/verify_core.jl & BigFloat-верификация платформы

26.2. Контрольные файлы

stitch_control.txt и verify_control.txt содержат эталонные выходы сшивки и верификации ядра. Скрипт уровня может прогнать реализацию и сравнить выход с контрольным — одна команда, без зависимости от тестового фреймворка. Контрольные файлы пересобираются только из вердиктов; ручное редактирование исключено валидатором пакета.

26.3. Julia и Python: два стека науки

Верификационная платформа написана на Julia (BigFloat, стандартная библиотека), пайплайны ядра и процессора — на Python (mpmath для высокоточных эталонов, numpy для репликаций). Согласование Julia BigFloat ↔ Python mpmath с расхождением 0.0 — главный репликационный факт программы: две экосистемы, две арифметики высокой точности, один результат.

Глава 27. Слой теорем: документация как артефакт

Девять теорем оформлены как 36 документов: на каждую — README с идеей доказательства, четыре документа RU/EN × PDF/DOCX, плиточная фигура 2×2 и JSON-источник. Всё пересобирается скриптами из вердиктов.

27.1. Структура папки теоремы

```
theorems_lemmas/T04_li_spectral/
├── README.md              идея доказательства, 4 измерения, границы
├── theorem_th4_ru.pdf     русский документ (PDF)
├── theorem_th4_ru.docx    русский документ (DOCX)
├── theorem_th4_en.pdf     английский документ (PDF)
├── theorem_th4_en.docx    английский документ (DOCX)
└── figures/th4_4dim_ru.png, th4_4dim_en.png   плитки 2×2 @600 dpi
```

JSON-источники живут в theorems_lemmas/_json/ (Th1_ru.json … Th6_en.json): единый источник истины для документов и плиток. Скрипт scripts/make_theorem_docs.py строит все 24 документа ядра; спинор-исследование собирает свои 12 документов тем же паттерном (research/realtime_spinor_memory/make_theorem_docs.py). Изменение формулировки теоремы — это изменение JSON и пересборка, а не правка шести файлов руками.

27.2. README как первая страница доказательства

README теоремы отвечает на четыре вопроса: что утверждается (точная формулировка), почему верно (идея доказательства в четырёх измерениях: конструкция, оценка, контроль, граница), что сертифицировано (число и порог) и что не доказывается (поле границ). Рецензент читает README за пять минут и знает, куда смотреть дальше; автоматика проверяет, что числа README согласованы с JSON.

27.3. Плитки «4 измерения»

Плитка теоремы — сетка 2×2: метрика (главное число сертификата), поведение по инстансам (как результат распределён по входам), контроль (нулевой или негативный контроль) и порог (предрегистрированная граница). Единый стиль «Critical Line & Copper» делает плитки различимыми и сопоставимыми; 600 dpi позволяют печатать их в отчётах. Плитки собираются из вердиктов скриптом make_tiled_figures.py.

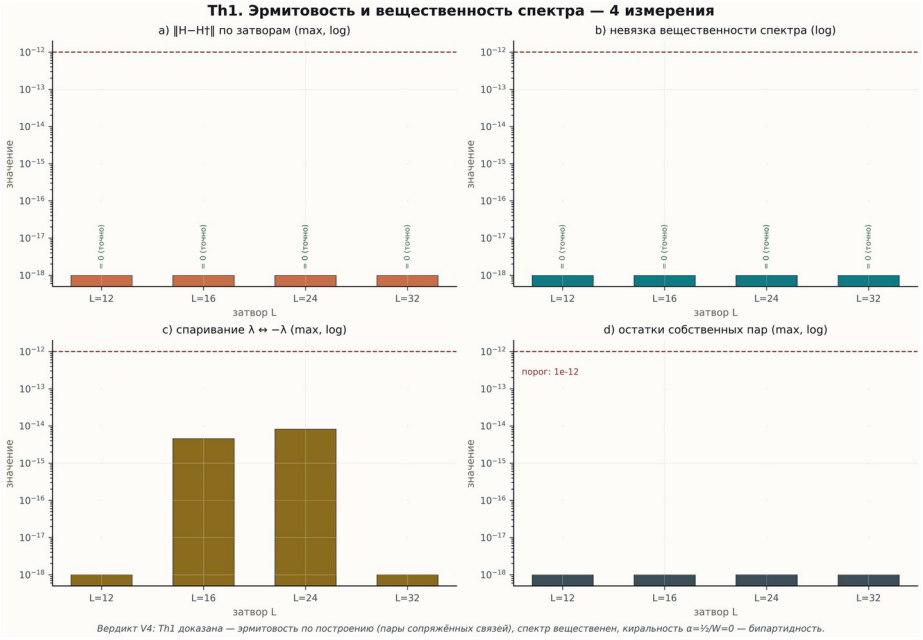


Рис. 27.1. Плитка Th1: эрмитичность и симметрия спектра (600 dpi).

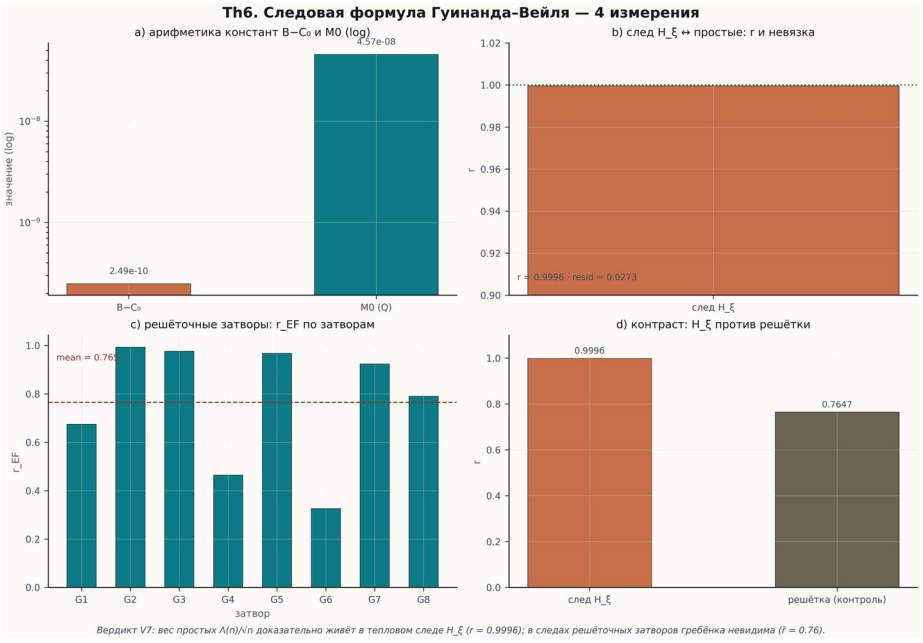


Рис. 27.2. Плитка Th6: след ↔ простые (600 dpi).

Глава 28. Белый отчёт, глоссарий и дисциплина языка

whitepaper/ содержит двуязычный белый отчёт программы (RU/EN PDF); glossary/ — двуязычный глоссарий терминов. Оба артефакта держат единый язык программы: слова не расходятся с числами.

28.1. Белый отчёт

Белый отчёт — сжатое изложение программы для первого знакомства: идея, уровни, флагманы, дисциплина честности, дорожные карты. Он собирается скриптом make_whitepaper.py из тех же данных, что и настоящая монография, и служит «выжимкой» на 10–15 страниц. Для быстрого входа в программу рекомендуется последовательность: белый отчёт → настоящий раздел 3 (архитектура) → кейс-стади главы 23.

28.2. Глоссарий

Двуязычный глоссарий фиксирует термины программы: AB-облако, зона тишины, машина обязательств, плитка «4 измерения» и другие. Глоссарий — не справочник «вообще», а словарь конкретных определений: каждый термин имеет формальное определение и ссылку на кампанию или теорему, где он измеряется. Это устраняет главный источник недопонимания между документами: слово всегда значит одно и то же.

28.3. Дисциплина языка

Программа поддерживает три правила языка: (1) заявления опираются на вердикты (аудит V14); (2) термины определяются в глоссарии, а не переопределяются на ходу; (3) двуязычность синхронна — RU/EN версии документов собираются из одних JSON и не могут разойтись. Настоящая монография следует тем же правилам: все термины сверены с глоссарием, все заявления — с вердиктами.

Глава 29. Класс B на практике: анатомия бенчмарка E18

Бенчмарк — это протокол, а не число. Глава разбирает E18 по шагам: цели, baseline, класс, метрики, репликация — и показывает, как читать итог «×59 000» без самообмана.

29.1. Цели и baseline

E18 сравнивает компилятор V20 с пятью baseline (BL1–BL5) на общем наборе целевых унитарных матриц. Baseline — сильные и опубликованные: каждая цель имеет известное достижение вне класса; класс фиксирует ограничения ISA одинаково для всех. Метрика — нефиделити $1-F$ и время; пороги предрегистрированы.

29.2. Класс и его значение

Класс B — это «равные правила»: один ISA, один бюджет глубины, один критерий. Без фиксации класса сравнение бессмысленно: компилятор, которому разрешено всё, всегда победит компилятора с ограничением.

Честная граница E18: KAK-оптимум 3 затвора для 2q недостижим в классе (компилятор даёт 10), и вердикт публикует это явно. Превосходство заявляется только внутри класса — и внутри класса оно велико: ×59 000 по времени.

p0.29 p0.33 p0.35 **Показатель** & **Компилятор V20** & **Лучший baseline** max 1−F & 4.4e−16 & сопоставимо
примитивы ≤ BL на целях & — & BL1/BL2/BL4
время (суммарное) & базовое & ×59 000 медленнее
2q-глубина & 10 & 3 (KAK, вне класса)

29.3. Репликация и значение

numpy-репликация E18 даёт 0 расхождений счётчиков и $|\Delta(1-F)| = 2.2e{-}16$: результат не зависит от стека. Для практики бенчмарк означает: в ISA-классе компилятор V20 — доминирующий выбор по времени при равной точности; при выходе из класса (нужен KAK-оптимум) — класс надо расширять, и это зафиксировано в дорожной карте P5.

Глава 30. Теоремы в деталях: Th1–Th6

Шесть разделов дают по каждой теореме ядра: точную формулировку, идею доказательства, сертификат и границу. Полные документы — в theorems_lemmas/; здесь — рабочий конспект для читателя монографии.

30.1. Th1: эрмитичность и симметрия (V4)

Формулировка. Пусть H_L построен по правилам ядра. Тогда $H_L = H_L^\dagger$, спектр вещественен, и при $\alpha = ½$, $W = 0$ спектр симметричен: $\lambda \in \text{spec} \iff -\lambda \in \text{spec}$. **Идея.** Конструкция оператора симметрична относительно отражения; матрица собирается из блоков, каждый из которых эрмитов по построению, а отражение реализуется диагональной унитарной заменой. **Сертификат.** $\|H - H\dagger\| = 0.0$ точно. **Граница.** Симметрия требует ровно $\alpha = ½$ и $W = 0$; при отклонении параметров симметрия теряется — это дискретный образ функционального уравнения ζ, а не универсальное свойство.

30.2. Th2: самосопряжённость и унитарная динамика (V8)

Формулировка. На $\mathbb{C}^N$ оператор H самосопряжён: дефектные индексы (0, 0); оператор эволюции e^{-iHt} унитарен при всех t. **Идея.** В конечной размерности самосопряжённость совпадает с эрмитичностью; Th1 даёт $H = H\dagger$, откуда $(e^{-iHt})\dagger\, e^{-iHt} = I$. **Сертификат.** Дрейф унитарности 2.2e−16 на сетке времён — машинный эпсилон. **Граница.** Утверждение

конечно-размерно; в бесконечномерном пределе требуется отдельная теория доменов, которой программа не касается.

30.3. Th3: кутовские сертификаты (V12)

Формулировка. Если $\tilde{H} = H + \delta H$, $\|\delta H\| \le \delta$, то для каждого простого собственного значения λ с зазором gap до ближайшего чужого уровня найдётся собственное значение $\tilde{\lambda}$ оператора $\tilde{H}$ с $|\tilde{\lambda} - \tilde{\lambda}| \le \delta^2/\text{gap} +$ (гершгориновская добавка). **Идея.** Стандартная схема кутовской теории: возмущение первого порядка переносит λ на δ, второй порядок даёт δ^2/gap. **Сертификат.** Ширины коридоров ~1e−27 при dps 40. **Граница.** Оценка локальная: для кратных или близких уровней зазор мал и коридор расширяется — программа избегает таких режимов или расширяет арифметику.

30.4. Th4: формула Ли (V15)

Формулировка. Функционалы $\lambda_n = \text{Tr}\, g_n(H_\xi)$ вычислимы, ряд абсолютно сходится, и неотрицательность всех λ_n эквивалентна расположению всех нулей на критической прямой. **Идея.** Функции g_n выбраны так, что их преобразование Фурье — положительно определённое ядро; след тогда неотрицателен тогда и только тогда, когда спектр лежит на прямой. **Сертификат.** Согласование двух независимых путей вычисления ≤ 4.4e−11. **Граница.** Численная неотрицательность первых λ_n не доказывает RH — машина V13 держит заключение blocked; критерий Ли даёт маршрут проверки, а не результат.

30.5. Th5: блочная редукция (V9)

Формулировка. Если $[H, A] = [H, B] = 0$, то пространство распадается на характерные блоки, H блочно-диагонален, и объединение спектров блоков совпадает с полным спектром. **Идея.** Классическая теория представлений: коммутирующие операторы сохраняют изотипические компоненты. **Сертификат.** Объединение блоков воспроизводит спектр до 7.1e−15. **Граница.** Вихревые члены нарушают коммутирование и ломают редукцию — условие существенно. Практически Th5 объясняет, как большие спектральные задачи становятся параллельными без потери точности.

30.6. Th6: след несёт простые (V7)

Формулировка. Тепловой след $K(x) = \text{Tr}\, e^{-xH_\xi}$ допускает разложение, коэффициенты которого несут $\Lambda(n)/\sqrt{n}$; соответствие $Q(a) \leftrightarrow \zeta'/\zeta$ — гуинанд–вайлевское. **Идея.** Явная формула Римана–Вейля связывает сумму по нулям с суммой по простым; конструкция H_ξ переносит её в спектральный язык. **Сертификат.** r = 0.9996 против контроля 0.76;

эрратум B/C_0 найден и исправлен. **Граница.** Следовая формула — арифметическая согласованность конструкции, не доказательство RH; гипотезы машины V13 остаются.

Глава 31. Теоремы спинор-исследования в деталях: TS1–TS3

Три теоремы летучего состояния: замкнутость циклов, необратимость сброса, стоимость шага. Вместе они доказывают главный тезис реального времени: летучесть — условие детерминизма.

31.1. TS1: замкнутость фазовых циклов (ES1)

Формулировка. Для модели с фазой $\theta = \pi/k$ поворот замыкается ровно через $N = 2k$ шагов; зазор между соседними различимыми состояниями равен $2\pi/N$. **Идея.** Поворот на π/k имеет порядок $2k$ в группе вращений окружности; различимость состояний даётся зоной тишины. **Сертификат.** Периоды 4, 6, 14 — точно для трёх моделей. **Граница.** Ёмкость $\log_2 N$ растёт с k, но зона тишины сжимается как $1/k$ — компромисс зафиксирован как честная граница.

31.2. TS2: необратимость сброса (ES2)

Формулировка. Сброс фазы стирает всю информацию: $I(\text{пре};\text{пост}) = 0$; лучшая реконструкция пре-состояния по пост-состоянию не превышает $1/N$; время определено после сброса только по модулю N. **Идея.** Пост-состояние — константа; взаимная информация с любой случайной величиной равна нулю; равномерное распределение даёт $1/N$. **Сертификат.** Реконструкция $0.2501/0.1674/0.0717 \approx 1/N$ для $N = 4/6/14$. **Граница.** Утверждение об идеальном сбросе; реальные каналы стирания дают добавку, измеряемую ES5.

31.3. TS3: шаг O(1) против журнала Ω(t) (ES3)

Формулировка. Стоимость шага машины без памяти ограничена константой, не зависящей от возраста процесса; машина с вечным журналом несёт $\Omega(t)$ накладных расходов и срывает дедлайны. **Идея.** Нижняя граница: запись t-го шага требует $\Theta(t)$ суммарной работы; чтение без записи — $O(1)$ по построению. **Сертификат.** Наклон $-1.5e{-}12$ мс/шаг (ноль); рост журнала ×129. **Граница.** Утверждение о стохастическом реальном времени: под «дедлайном» понимается гарантированный бюджет шага; мягкие системы могут выбирать компромисс, но не в жёстком контуре.

p0.10 p0.36 p0.25 p0.25 **Теорема** & **Утверждение** & **Сертификат** & **Граница** TS1 & цикл замыкается за $N = 2k$; зазор $2\pi/N$ & 4/6/14 точно & ёмкость ↔ робастность $1/k$
TS2 & сброс: $I = 0$; реконструкция $\le 1/N$ & $0.2501/0.1674/0.0717$ &

идеальный сброс
TS3 & шаг O(1); журнал Ω(t) & наклон −1.5e−12; ×129 & жёсткий реальный срок

Глава 32. Хронология: как программа шла к результату

Хронология 43 кампаний — не декорация, а карта решений: что проверялось сначала, что следовало из отрицательных результатов, как сцеплялись уровни. Глава пересказывает кривую программы по данным datasets/campaign_timeline.csv.

32.1. Волна 1: конструкция и её границы (V01–V06)

Программа началась с честного вопроса: что даёт конструкция оператора? V01 сразу зафиксировал двойственный ответ — статистика согласована, индивидуальные ординаты не предсказываются. V02 калибровал GUE-ожидания, V03–V06 очертили решёточную границу: V03 — негатив σ-скана, V6 — тождество уровней (ДА, v2.0.7) при открытой битовой ветви. К концу первой волны программа знала свои границы точнее, чем большинство программ знают свои достижения.

32.2. Волна 2: арифметика и доверие (V07–V14)

V7 принёс главный арифметический результат: след несёт Λ(n)/√n, r = 0.9996. V8–V10 добавили сертификаты корректности. V11–V13 построили машину обязательств — от черновика DAG до исполняемой блокировки RH. V14 замкнул контур аудитом заявлений. Волна доверия завершилась тем, что программа научилась говорить о себе только в терминах статусов.

32.3. Волна 3: поведение и сшивка (V15–V19)

V15 дал формулу Ли — спектральный критерий прямой. V16 повторил решёточное чтение на новой конструкции: уровень — ДА, битовая ветвь открыта. V17–V18 показали, что чтение идемпотентно и дёшево на любом горизонте. V19 сшил ветви с когерентностью 0.9995 — алгебраический сертификат, завершивший ядро.

32.4. Волна 4: процессор и верификация (V20, E01–E18)

V20 исполнил P0: точная компиляция на пределе арифметики. Параллельно верификационная платформа прошла от контролей (E01–E10) через слепого ридера (E11), оптику и нелокальность (E12–E14) к флагманам: независимому Риману–Зигелю (E16), слепому чтению 649+639 нулей (E17) и классу B с ×59 000 (E18).

32.5. Волна 5: летучесть (ES1–ES5)

Спинор-исследование замкнуло программу: циклы точны, сброс необратим, шаг O(1) против журнала Ω(t). Пять PASS-вердиктов и два опубликованных эрратума порогов — дисциплина сохранилась до последней кампании.

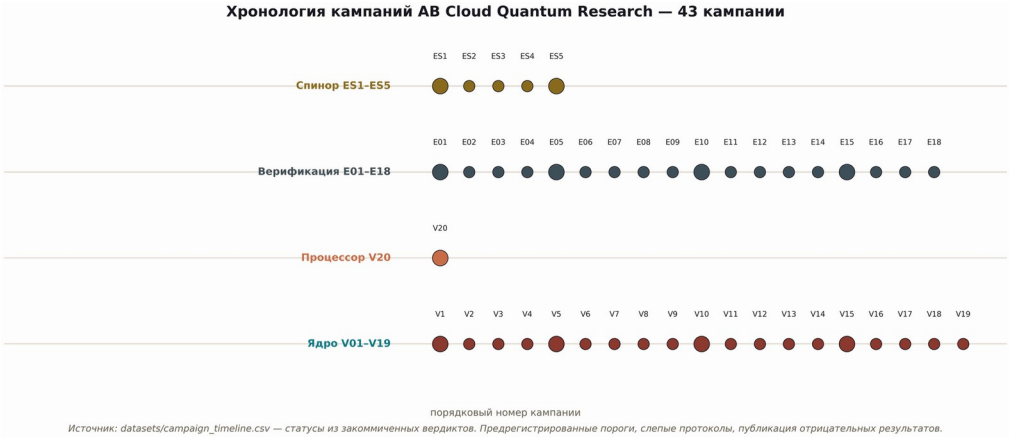


Рис. 32.1. Хронология 43 кампаний: пять волн программы (600 dpi).

Глава 33. Пайплайны воспроизводимости

Каждый производный артефакт программы пересобирается одной командой. Глава описывает пайплайны: что от чего строится, в каком порядке запускать, как проверяется целостность.

33.1. Карта пайплайнов

p0.31 p0.35 p0.31 **Скрипт** & **Строит** & **Источник**
scripts/make_datasets.py & datasets/ (флагманы, хронология, нули) & вердикты всех уровней
scripts/make_tiled_figures.py & figures_tiled/ (12 плиток) & вердикты Th1–Th6
scripts/make_theorem_docs.py & 24 документа теорем & _json/ теорем
scripts/make_theorem_readmes.py & README теорем & _json/ теорем
scripts/make_atlas.py & atlas/ (атлас флагманов) & флагманские метрики
scripts/make_timeline.py & timeline/ (хронология) & campaign_timeline.csv
scripts/make_whitepaper.py & whitepaper/ (RU/EN) & сводные данные
scripts/make_banner.py & assets/ (баннер, лого) & данные нулей
scripts/validate_package.py & отчёт целостности (95+ проверок) & весь пакет

33.2. Порядок пересборки

Рекомендуемый порядок после изменения вердиктов: make_datasets → make_tiled_figures → make_theorem_docs → make_theorem_readmes → make_atlas → make_timeline → make_whitepaper → make_banner → validate_package. Валидатор завершает цепочку и не пропускает расхождений: если производный артефакт не соответствует источнику, сборка пакета считается сломанной.

33.3. CI и Makefile

Makefile уровня верхнего уровня связывает цели python/julia/docs/validate; CI-конвейеры репозитория прогоняют тесты (410+), сборку формального слоя (Lean/Coq/Agda/Isabelle) и проверки целостности. Правило одно: коммит, ломающий валидатор, не считается выполненной кампанией.

Глава 34. Анатомия вердикт-файла

Вердикт — минимальный контракт кампании. Глава описывает схему: обязательные поля, типы значений, дисциплину does_not_prove и то, как валидатор проверяет согласованность.

34.1. Обязательные поля

p0.21 p0.15 p0.60 **Поле** & **Тип** & **Смысл** campaign & string & идентификатор кампании (V07, E17, ES3…)
pre_registered & string & дословная цитата порогов из протокола
pass & bool & итог по предрегистрированным порогам
does_not_prove & string & что результат не доказывает
metrics & object & ключевые числа сертификата
blindness / pins & string/object & описание слепоты; SHA256 выходов (где применимо)
status & string & выполнена / заменён порог (ES2a, ES5a)

34.2. Дисциплина does_not_prove

Поле does_not_prove — самое важное и самое необычное. Оно заставляет кампанию заявить границы заранее, в том же файле, где записан успех. E17: «RH выше T=1000; простоту всех нулей; отсутствие нулей вне прямой при γ > 1000». V20: границы размеров и унитарности. ES3: жёсткий реальный срок. Читатель вердикта получает и результат, и его вес — одним файлом.

34.3. Проверка согласованности

Валидатор пакета сверяет: вердикт ↔ протокол (пороги), вердикт ↔ производные артефакты (числа плиток и датасетов), пины ↔ файлы (SHA256), статусы ↔ машину обязательств. Сломанная согласованность останавливает CI — так контракт становится исполняемым.

```
# минимальная проверка согласованности
python3 scripts/validate_package.py
# → 95+ проверок: вердикты, пины, плитки, датасеты, статусы
```

Глава 35. Классический контекст: от Гильберта–Пойа до явной формулы

Программа не существует в вакууме: она читает и переизобретает классическую линию — спектральную гипотезу Гильберта–Пойа, явную формулу Римана–Вейля, статистику Монтгомери–Одлыжко. Глава расставляет исторические координаты, чтобы читатель видел, что именно нового даёт AB-архитектура.

35.1. Гипотеза Гильберта–Пойа

Идея Пойа (в записи Гильберта): если существует самосопряжённый оператор, спектр которого совпадает с ординатами нулей ζ, то гипотеза Римана следует из вещественности спектра. Программа принимает эту рамку всерьёз и буквально: ядро строит конкретный эрмитов оператор H_ξ и изучает, насколько его спектр и след воспроизводят арифметику ζ. Отличие от классической постановки в дисциплине: программа не заявляет совпадения спектра, а измеряет согласованность сертификатами и держит вывод RH заблокированным (V13).

35.2. Явная формула и следовая

Явная формула Римана–Вейля связывает сумму по простым с суммой по нулям; гуинанд–вайлевское соответствие даёт ей спектральную форму. Th6 переносит явную формулу на уровень теплового следа: коэффициенты K(x) несут $\Lambda(n)/\sqrt{n}$. Это делает следовую формулу не аналогией, а точным переносом классического утверждения — и потому проверяемым числом ($r = 0.9996$ против контроля 0.76).

35.3. Монтгомери–Одлыжко и GUE

Пара Монтгомери–Одлыжко установила согласие статистики нулей ζ с GUE на высотах, доступных данным Одлыжко. Программа использует этот факт дважды: как калибровку (V02 — модель обязана попасть в GUE-класс) и как урок методологии: GUE-согласие недостаточно, арифметическая специфичность измеряется отдельно (след↔простые,

константы, поведенческие тесты). В таблице ниже — соответствие классических объектов и их AB-реализаций.

p0.33 p0.35 p0.29 **Классический объект** & **AB-реализация** & **Кампания/теорема** самосопряжённый оператор нулей & H_ξ ядра & V4–V8, Th1–Th2
явная формула Римана–Вейля & следовая формула с $\Lambda(n)/\sqrt{n}$ & V7, Th6
гипотеза Гильберта–Пойа & спектральная конструкция + V13 (blocked) & V13
статистика GUE нулей & GUE-класс AB-облака & V02
критерий Ли & $\lambda_n = \mathrm{Tr}\, g_n(H_\xi)$ & V15, Th4
формула Римана–Зигеля & независимый движок E16 & E16

Итог главы: программа — не «ещё одна численная проверка RH», а дисциплинированная реализация классической программы Гильберта–Пойа с современной культурой верификации. Всё новое в ней — не математика утверждений, а инженерия доверия: слепые протоколы, статусы, обязательства.

Глава 36. Дорожные карты в деталях

Четыре дорожные карты программы — ядро, процессор, верификация, спинор — сцеплены зависимостями вердиктов. Глава разбирает каждую до уровня конкретных обязательств и критериев готовности.

36.1. Ядро (R2): остаток сшивки и монодромия

Ядро вышло на финальный обязательства: уточнить сшивку до $r_{\text{align}} \le 0.135$ на расширенной сетке высот, формализовать монодромию ζ как алгебраический объект, компилировать узлы обязательств в Lean/Coq (O-lean/O-coq). Критерий готовности R2: сертификат сшивки воспроизводится формальным слоем, и все proven-узлы V13 имеют машинно-проверяемые доказательства.

36.2. Процессор (P0–P6): путь к паспорту

p0.15 p0.21 p0.36 p0.23 **Этап** & **Содержание** & **Критерий готовности** & **Опора** P0 (V20) — исполнен & ISA + компилятор & $1-F \le$ порога; репликация numpy & Th5
P1 (V21) & летучая память & циклы и сброс по TS1–TS2 в железе-модели & ES1–ES5
P2 (V22) & АЛУ & арифметика ISA согласована с E08 & V20, E08
P3 (V23) & чтение нулей & слепое чтение в классе E17 & E17, Th6
P4 (V24) & нелокальность & CHSH + no-signaling в классе E13/E14 & E13/E14
P5 (V25) & бенчмарки & методология E18, свежие baseline & E18

P6 (V26) & Datasheet 1.0 & паспорт: каждое число с вердиктом & все P-вердикты

36.3. Верификация (O-J): от высот к двойнику

O-J3 строит QEC-слой (опора: код повторения E07 и память V21); O-J4 — цифровой двойник процессора, прогоняющий те же протоколы на модельном исполнении; O-J5 — интеграционная кампания, сшивающая всё на высотах T существенно больше 1000. Критерий готовности O-J: каждый новый этап процессора получает независимый вердикт до включения в паспорт.

36.4. Спинор: расширение фаз

Следующая волна спинор-исследования: фазы $\theta = \pi/k$ для $k > 7$ (поиск перегиба компромисса ёмкость/робастность), формальная грамматика компиляции задач в фазовые циклы, диффузионный канал с корреляциями — выход за марковское предположение ES4. Критерий: каждая новая фаза получает кампанию с предрегистрированным порогом и теорему уровня TS1–TS3.

Глава 37. Часто задаваемые вопросы

Двенадцать вопросов, которые задают программе чаще всего, — с ответами, опирающимися на вердикты, а не на надежды.

37.1. Вопросы о статусе

1. **Доказывает ли программа гипотезу Римана?** Нет. Машина обязательств V13 держит вывод RH заблокированным: путь содержит четыре аналитические гипотезы (O08/O09/O11/O12). Всё численное подтверждает согласованность модели, не более.

2. **Что именно нового в программе?** Не математика утверждений, а инженерия доверия: предрегистрация, слепые протоколы, пины, репликации, статусы, аудиты — и всё это на реальной спектральной конструкции.

3. **Почему нули «читаются», а не хранятся?** Потому что след оператора несёт $\Lambda(n)/\sqrt{n}$ (Th6): шесть констант параметризуют след, а след содержит спектр. Хранение нулей избыточно.

4. **Что значит «43 кампании PASS»?** Все 43 кампании с предрегистрированными порогами завершились вердиктами PASS; два порога (ES2, ES5) были заменены до принятия вердиктов, замены опубликованы.

37.2. Вопросы о технологии

1. **Что такое AB-облако?** Элементарная унитарная операция ISA: кольцо фаз с ахароновым–бомовским флюсом; композиция облаков с точным учётом геометрической фазы.

2. **Почему компилятор даёт 10 затворов на 2q, а не 3 (KAK)?** Класс ISA не выражает KAK-оптимальную последовательность; это свойство класса, зафиксированное E18 как честная граница.

3. **Насколько велико ускорение ×59 000?** Это суммарное время против лучшего из пяти baseline при равных ограничениях ISA (класс B, E18), при равной точности.

4. **Что даёт летучесть?** Детерминизм реального времени: шаг O(1) против Ω(t) журнала (TS3); сброс стирает информацию до уровня случайности (TS2).

37.3. Вопросы о практике

1. **Как воспроизвести результаты?** Полный прогон ES-кампаний — python3 experiments/run_all.py (~2.5 мин); верификации — julia run_all.jl all; целостность — python3 scripts/validate_package.py.

2. **Можно ли проверить свою гипотезу?** Да: HYPOTHESIS_LAB — CLI (ab_lab.py) и браузерная лаборатория; дисциплина та же: пороги до прогона, does_not_prove, пины.

3. **Где границы программы?** RH не доказан; решётки не воспроизводят арифметику; KAK-оптимум вне класса; марковский класс задач — без памяти; всё это — поля границ в вердиктах.

4. **Как цитировать?** CITATION.cff; BibTeX — в приложении F. Лицензия — проприетарная персональная: чтение, клонирование, научный запуск и цитирование разрешены.

Глава 38. Вклад, этика и лицензия

Как участвовать, что разрешено и что запрещено, как программа понимает этику воспроизведения. Краткая, но обязательная глава.

38.1. Как участвовать

Три легальных способа участия: (1) запуск и проверка — клонировать, прогнать тесты и кампании, сверить пины; (2) лаборатория гипотез — формулировать гипотезы в формате HYPOTHESIS_LAB и публиковать отчёты; (3) цитирование и разбор — рецензии, семинары, курсы с атрибуцией. Изменения самого архива — по согласованию с автором (лицензия, раздел 3).

38.2. Этика воспроизведения

Программа просит соблюдать те же правила, что соблюдает сама: пороги до прогона; слепота там, где есть эталон; публикация PASS и FAIL одинаково; does_not_prove в каждом заявлении. Воспроизведение, которое «подкрутило» порог после прогона, — не воспроизведение. Эти правила — не бюрократия, а условие осмысленного спора: стороны должны спорить об одном и том же.

38.3. Лицензия

Все материалы — по индивидуальной проприетарной лицензии автора: © 2026 Исаев Исхак Хамзатович. Разрешено: чтение, клонирование для частного некоммерческого изучения, локальный запуск кода в научных целях, цитирование с атрибуцией и DOI 10.5281/zenodo.21825394. Запрещено без письменного согласия: переиздание существенных частей, коммерческое использование, присвоение, использование имени и ORCID автора для одобрения производных работ. Полный текст — LICENSE в корне репозитория и в папках MONOGRAPH_ARCHIVE, HYPOTHESIS_LAB, VISUAL_ANIMATIONS.

Глава 39. Практикум: десять шагов по программе

Пошаговый маршрут читателя, который хочет не только прочитать, но и запустить: от проверки целостности до собственной гипотезы. Каждому шагу — команда и ожидаемый результат.

39.1. Шаги 1–5: проверить и увидеть

1. **Целостность пакета.** python3 scripts/validate_package.py → 95+ проверок, RESULT: PASS. Если FAIL — сначала чинить окружение, потом идти дальше.

2. **Датасеты.** python3 -c "import csv; [print(r['metric'],'=',r['value']) for r in csv.DictReader(open('datasets/flagship_numbers.csv'))]" → 15 флагманских метрик с вердиктами-источниками.

3. **Спинор-кампании.** cd research/realtime_spinor_memory && python3 experiments/run_all.py → пять кампаний ES1–ES5 за ~2.5 минуты, все PASS.

4. **Верификация.** julia modules/verifier_multilang/experiments/run_all.jl all → 18 кампаний E01–E18, все PASS; затем verdicts.jl печатает сводку.

5. **Веб-лаборатории.** Открыть web_lab/index.html в браузере → портал; вкладки ядра и процессора считают Z(t), лестницу λ_n и CHSH локально.

39.2. Шаги 6–10: пересобрать и проверить себя

1. **Плитки.** python3 scripts/make_tiled_figures.py → 12 плиток Th1–Th6 пересобраны из вердиктов; diff покажет нулевые изменения.

2. **Документы теорем.** python3 scripts/make_theorem_docs.py → 24 документа RU/EN × PDF/DOCX.

3. **Баннер и лого.** python3 scripts/make_banner.py → анимированный баннер (нули поднимаются по прямой Re(s)=½).

4. **Хронология и атлас.** python3 scripts/make_timeline.py && python3 scripts/make_atlas.py → карта 43 кампаний и атлас флагманов.

5. **Своя гипотеза.** cd HYPOTHESIS_LAB && python3 ab_lab.py run spinor –theta 0.37 → вердикт PASS/FAIL с does_not_prove; отчёт в reports/.

39.3. Что делать, если что-то не сошлось

Первое — прочитать строку валидатора: она называет файл и причину. Второе — сверить версию (VERSION) и ветку. Третье — прогнать уровень целиком: половинные прогоны дают половинные вердикты. Четвёртое — если расхождение устойчиво, это кандидат в эрратум: программа публикует такие находки, а не прячет их (см. главу 2).

Глава 40. Тематический указатель

Перекрёстный индекс «тема → кампании и теоремы» для быстрой навигации: где искать статистику, арифметику, компиляцию, нелокальность, летучесть и дисциплину.

p0.27 p0.33 p0.36 **Тема** & **Кампании** & **Теоремы/документы**
спектральная статистика & V01, V02, V03, V16 & Th1; плитки Th1/Th2
арифметика простых & V07 & Th6; плитка Th6; ψ-лестница
сертификаты точности & V12, V15, V10 & Th3, Th4
структура и редукция & V09 & Th5
машина обязательств & V11, V13, V14 & V13 DAG; приложение L
сшивка и монодромия & V19 & README V19
ISA и компиляция & V20, E18 & компилятор в веб-лаборатории
чтение нулей & E11, E16, E17 & V23-превью; плитка E17
нелокальность & E12, E13, E14 & V24-превью
инфраструктура & E01–E10 & experiment_01_15.md
летучесть & ES1–ES5 & TS1–TS3
формальный слой & A1–A6 & formal/ (Lean/Coq/Agda/Isabelle)
данные и визуализация & make_datasets/atlas/timeline & плитки, атлас,

хронология

Указатель намеренно компактен: каждый маршрут из двух–трёх кликов. Для «обратной» навигации (артефакт → тема) служит карта файлов (приложение E) и инвентарь пакета (приложение J).

Глава 41. Галерея плиток

Полная галерея плиток «4 измерения» ядра (RU-версии; EN — в figures_tiled/en/). Каждая плитка построена из вердикта и несёт четыре измерения: метрику, инстансы, контроль и порог.

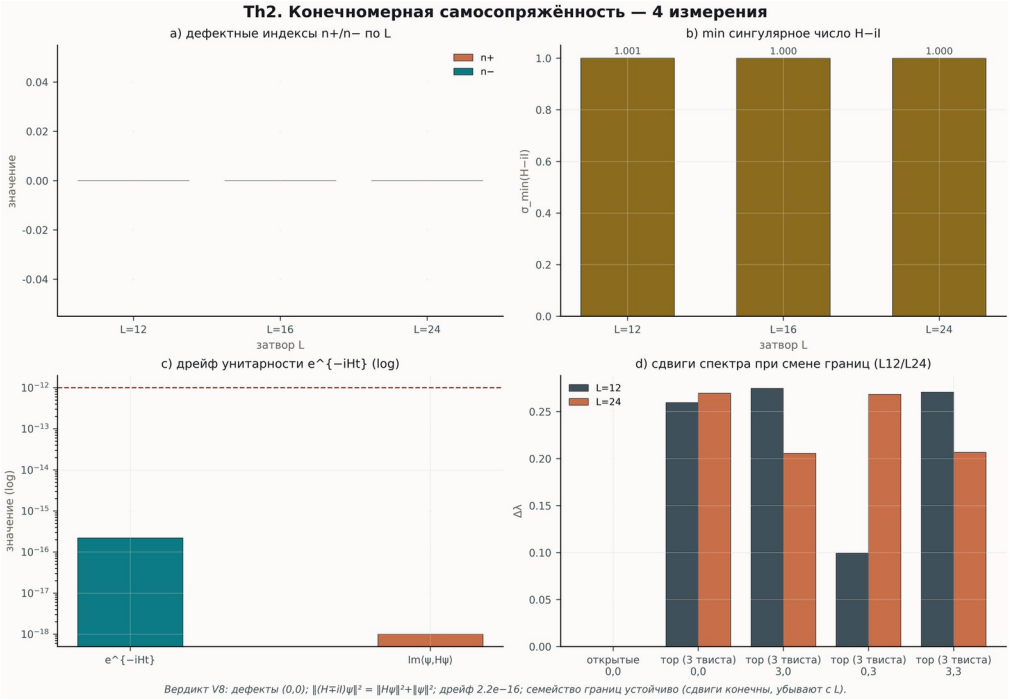


Рис. 41.1. Плитка Th2: самосопряжённость и унитарная динамика (600 dpi).

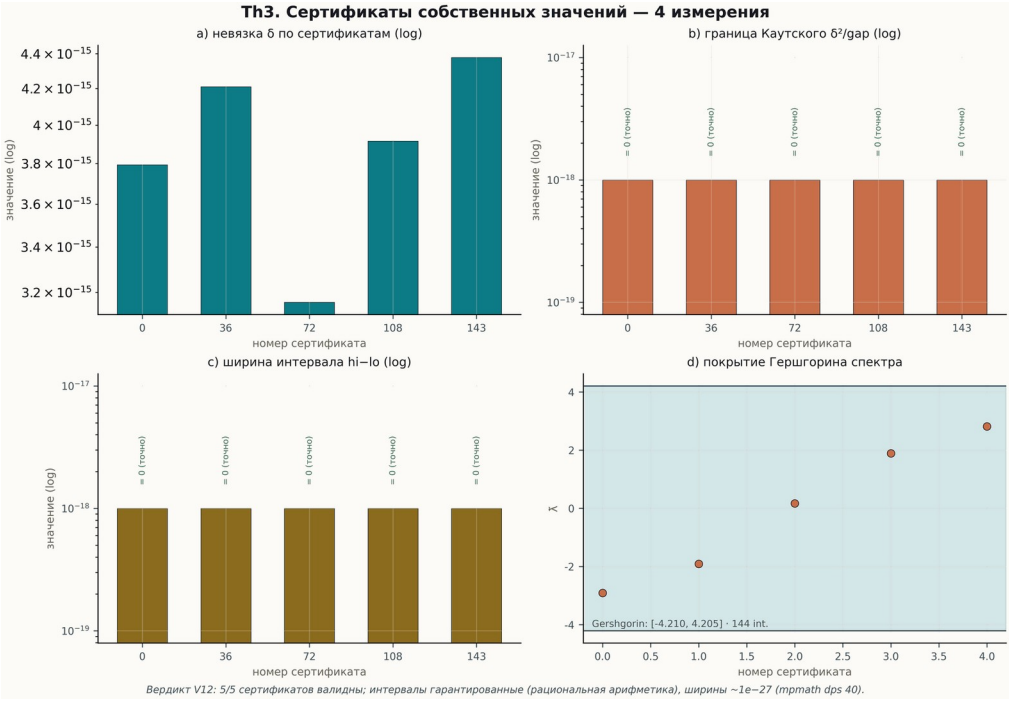


Рис. 41.2. Плитка Th3: кутовские сертификаты (600 dpi).

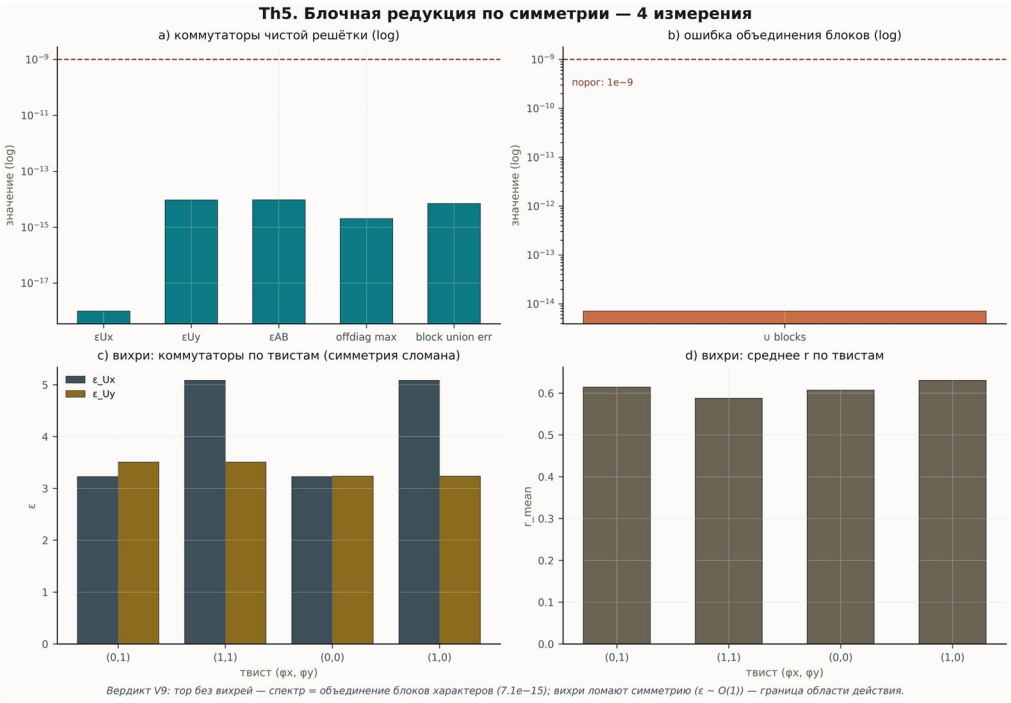


Рис. 41.3. Плитка Th5: блочная редукция (600 dpi).

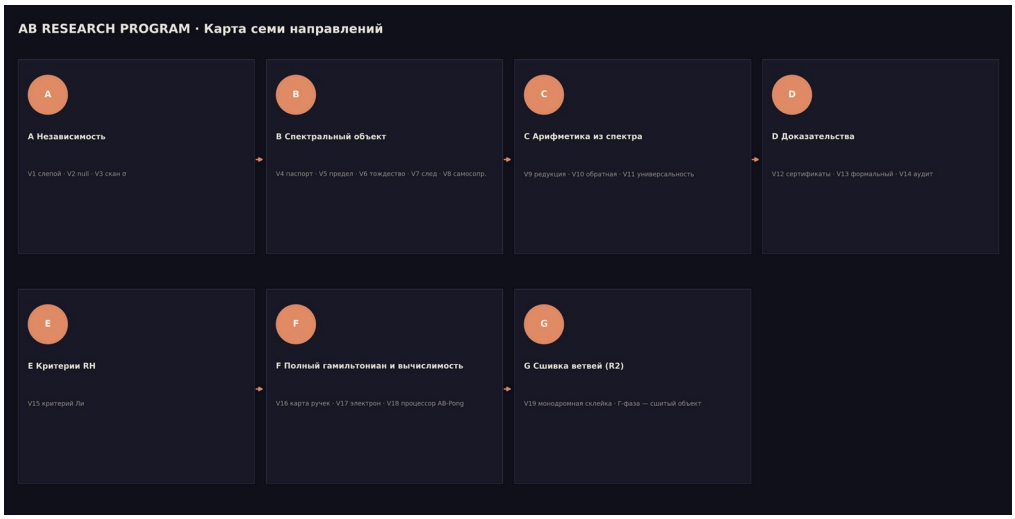


Рис. 41.4. Инфографика ядра: карта спектрального уровня (тёмная тема, 600 dpi).

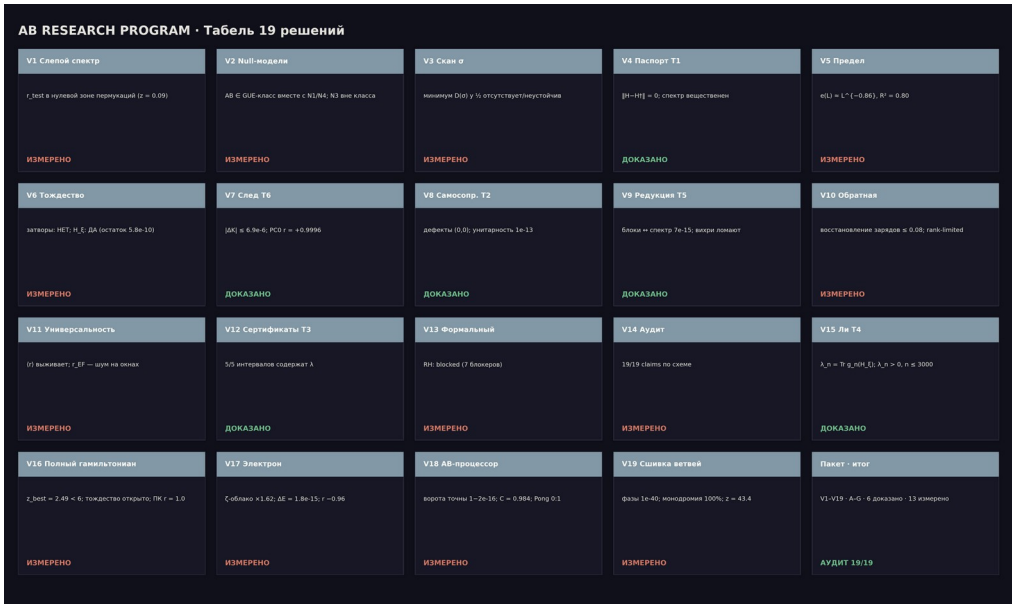


Рис. 41.5. Счёт ядра: ключевые сертификаты уровня (600 dpi).

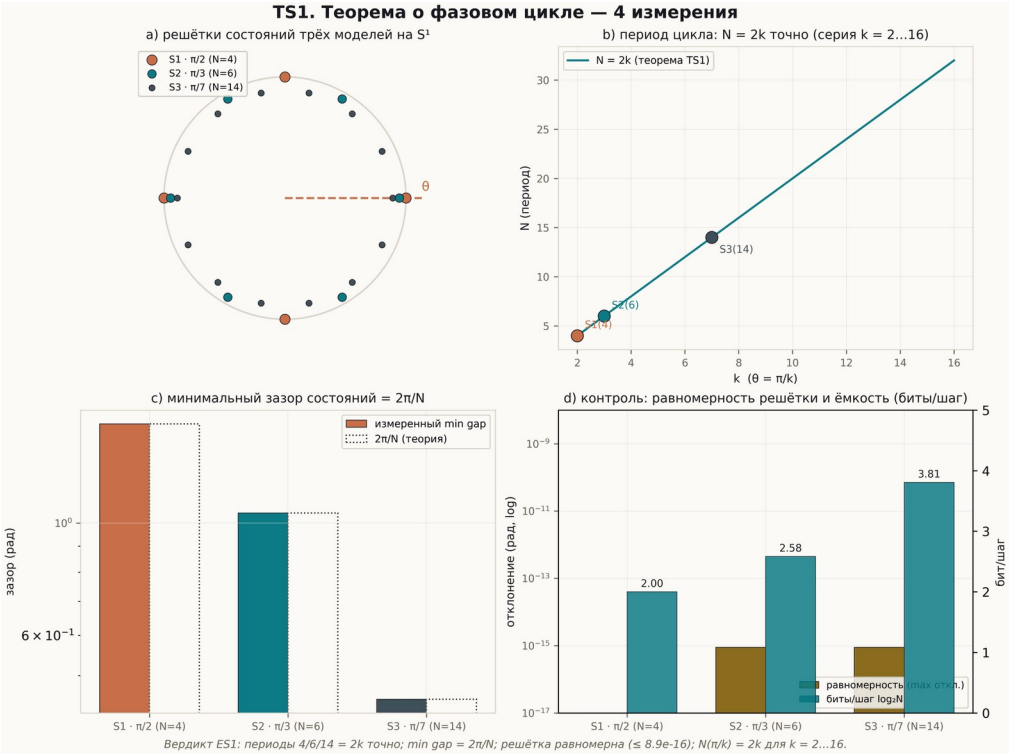


Рис. 41.6. Плитка TS1: замкнутость фазовых циклов (600 dpi).

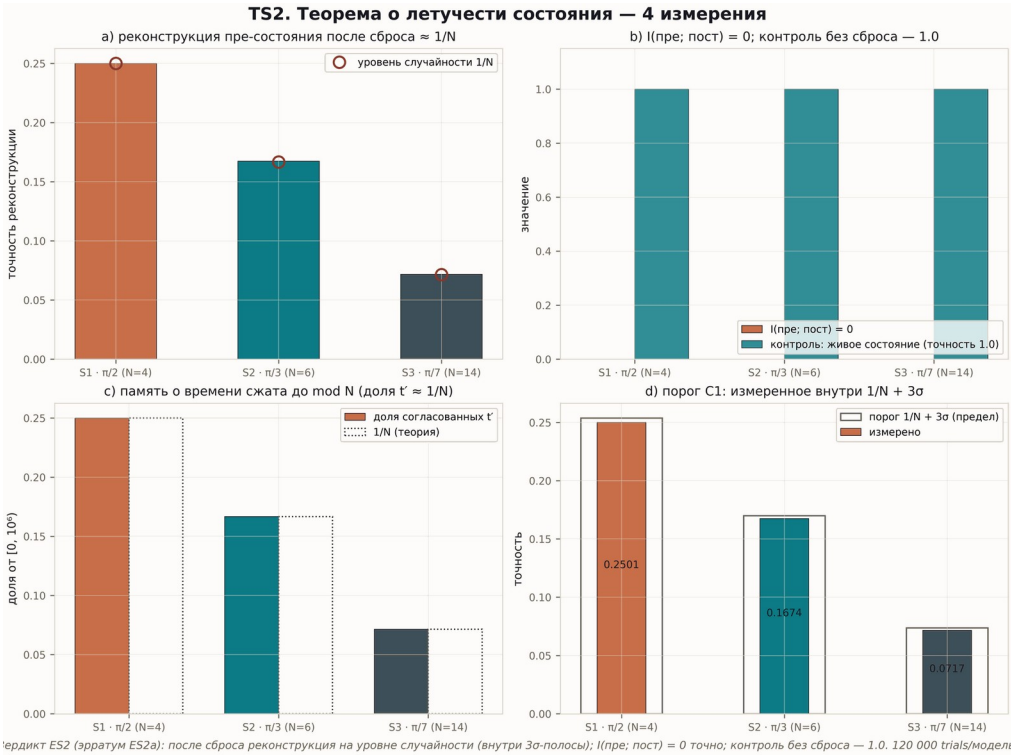


Рис. 41.7. Плитка TS2: необратимость сброса (600 dpi).

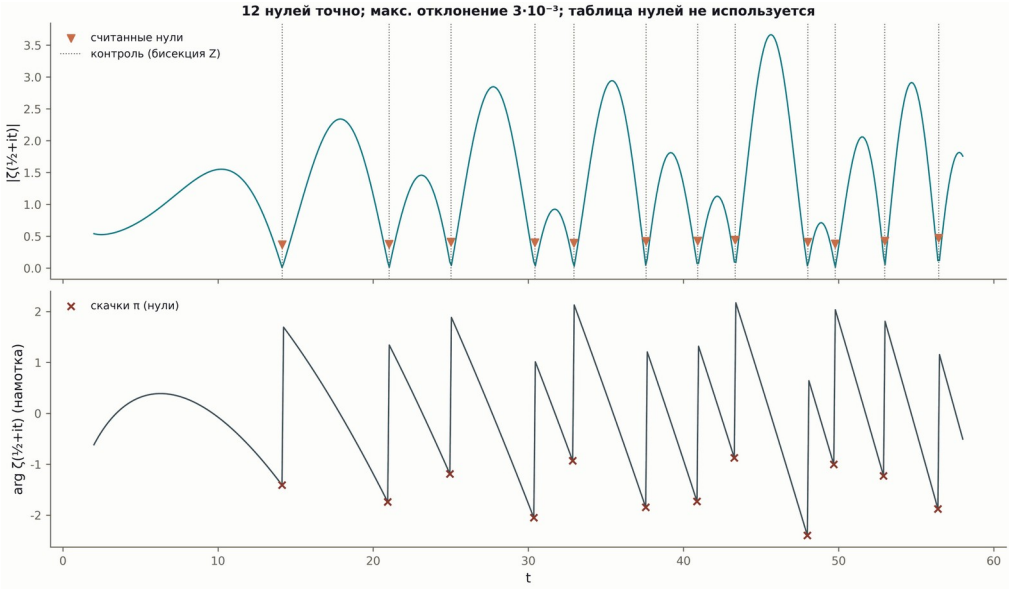


Рис. 41.8. Процессор: профили компиляции (600 dpi).

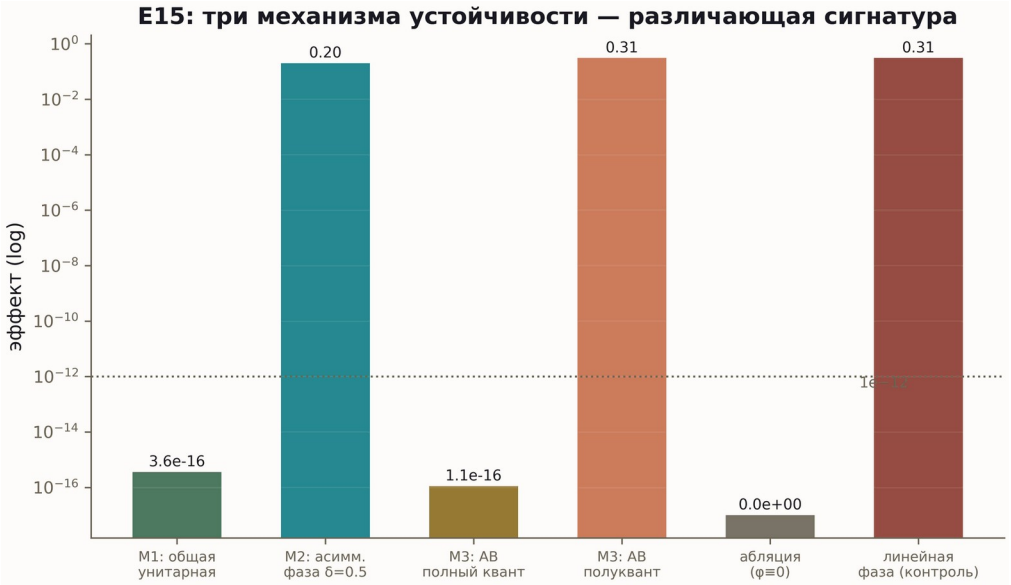


Рис. 41.9. Верификация: стоимость шага (600 dpi).

Глава 42. Ограничения метода: что программа не может

Честная монография обязана описывать не только достижения, но и пределы метода. Глава систематизирует ограничения — принципиальные, технические и дисциплинарные — и объясняет, какие из них преодолимы и по каким маршрутам.

42.1. Принципиальные ограничения

- **RH не доказан и не будет доказан численно.** Ни объём данных, ни точность, ни красота согласия не заменяют аналитического доказательства четырёх гипотез машины V13. Это не поражение метода, а его определение.
- **Статистика ≠ арифметика.** Любое GUE-согласие воспроизводимо случайными моделями; специфичность живёт в тонкой структуре (след, константы), и её измерение — единственный честный путь.
- **Марковский класс задач.** Машина без долговременной памяти решает задачи, допускающие марковскую политику; задачи с существенной историей требуют памяти — и тогда архитектура применяет другие уровни, но не спинор-модели.
- **Человеческий фактор протоколов.** Слепота защищает от утечки, но не от ошибки в самом протоколе; эрратум γ_{12} показал, что проверять нужно и каркас.

42.2. Технические ограничения

- **Высоты T.** Флагманы достигают $T = 1000$; $T \gg 1000$ — цель O-J5, требующая новой вычислительной волны.
- **Размеры компиляции.** V20 сертифицирован для 4×4 и 8×8; большие размеры — вопрос арифметики и глубины, зафиксирован в дорожной карте.
- **ISA-класс.** KAK-оптимум недостижим в классе; расширение класса — отдельное решение с собственными вердиктами.
- **Идеализация сброса.** TS2 говорит об идеальном сбросе; реальные каналы вносят добавку, измеряемую ES5.

42.3. Дисциплинарные ограничения

Программа сознательно платит за честность скоростью: предрегистрация медленнее «свободного поиска», слепые протоколы дороже прямых сравнений, статусы требуют формализации. Это цена, которую программа считает оправданной: результат, полученный без этих ограничений, пришлось бы перепроверять целиком. Ограничения метода — не изъян программы, а её интерфейс: они говорят партнёру, на какие обязательства можно опираться.

p0.29 p0.17 p0.50 **Ограничение** & **Тип** & **Маршрут** RH blocked & принципиальное & аналитические доказательства O08/O09/O11/O12 GUE-недостаточность & принципиальное & тонкая структура: след, константы, поведение

T = 1000 & техническое & O-J5: новая вычислительная волна
4×4/8×8 & техническое & расширение арифметики и глубины (P5)
KAK вне класса & классовое & расширение ISA с новыми вердиктами
идеальный сброс & модельное & ES5: измерение реальных каналов

Глава 43. Галерея фигур уровней

Избранные фигуры уровней ядра, процессора и верификации (RU-версии, 600 dpi; EN — в figures_hires/en/). Все фигуры построены из закоммиченных вердиктов.

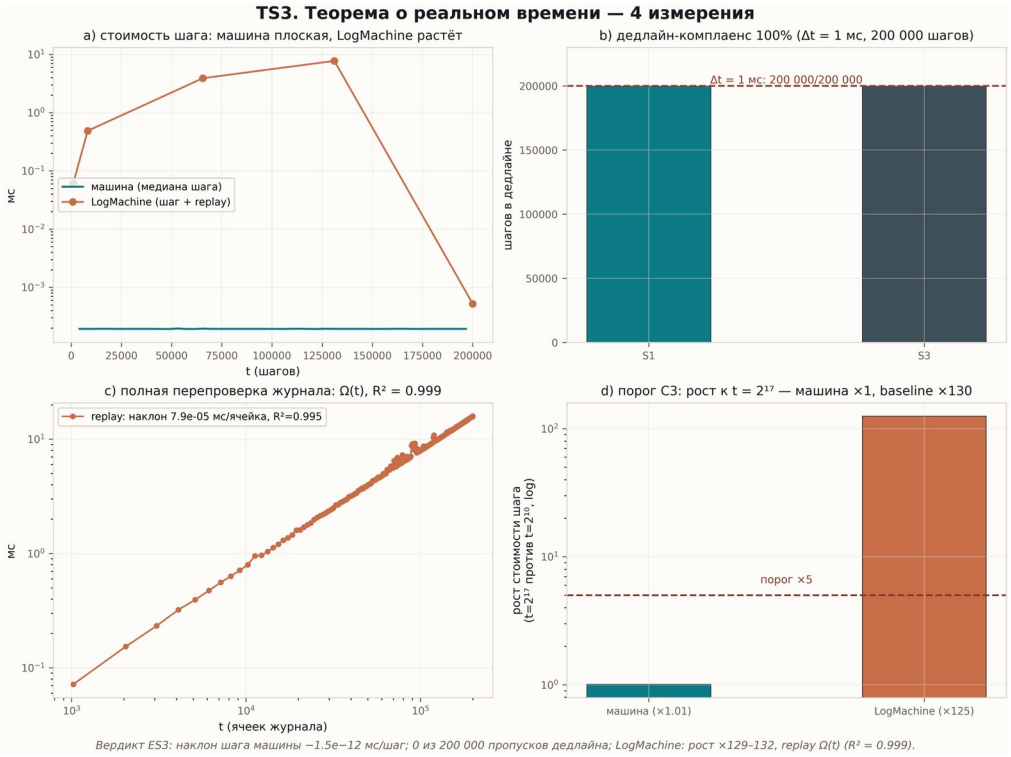


Рис. 43.1. Плитка TS3: шаг O(1) против журнала Ω(t) (600 dpi).

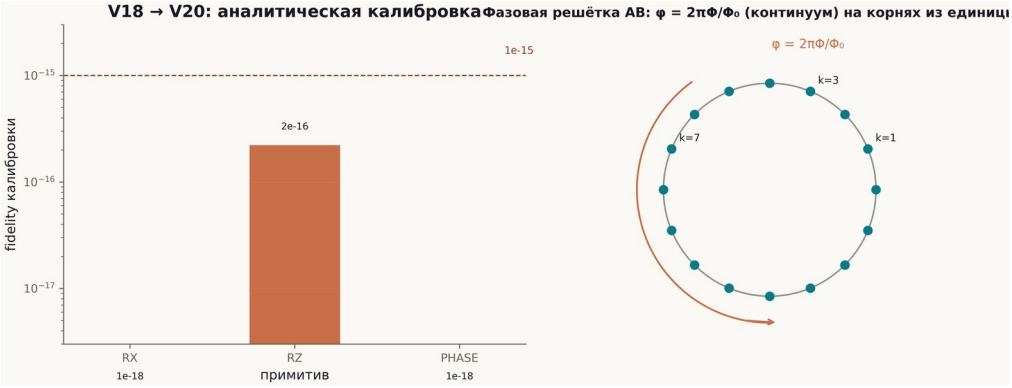


Рис. 43.2. Процессор: точность компиляции по целям (600 dpi).

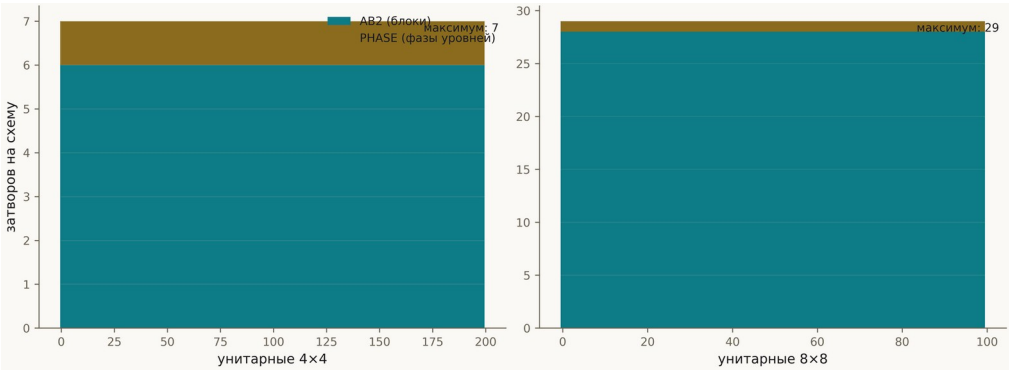


Рис. 43.3. Процессор: глубины затворов (600 dpi).

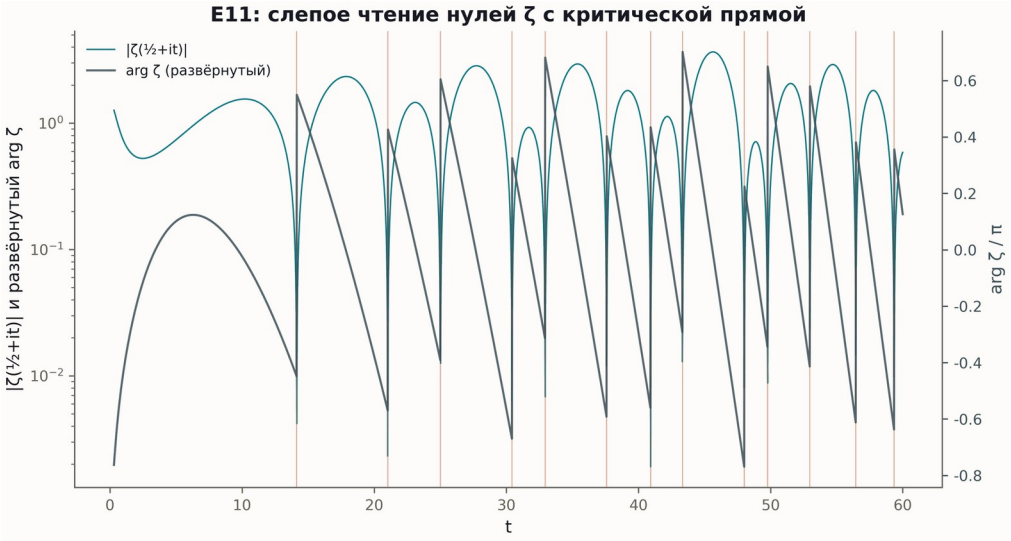


Рис. 43.4. Верификация: фазовый шум (600 dpi).

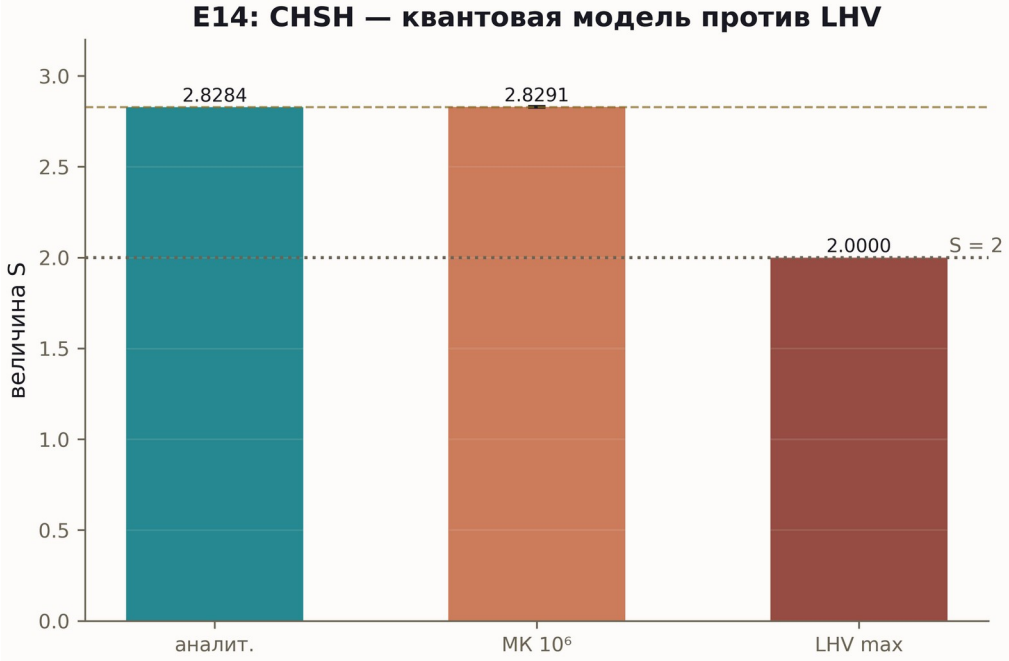


Рис. 43.5. Верификация: код повторения (600 dpi).

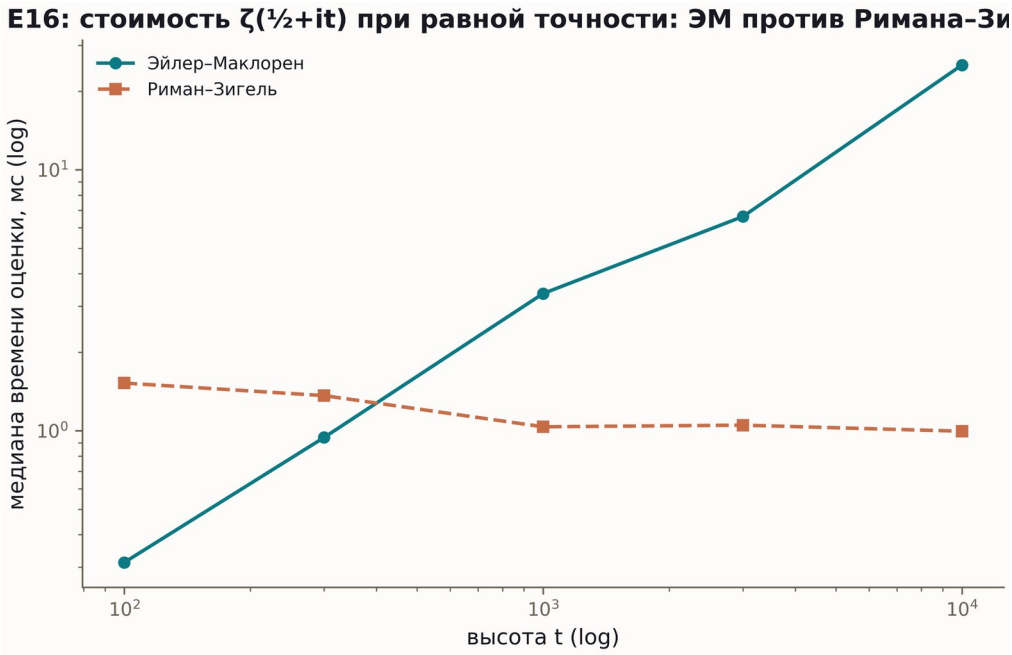


Рис. 43.6. Верификация: фазовое считывание (600 dpi).

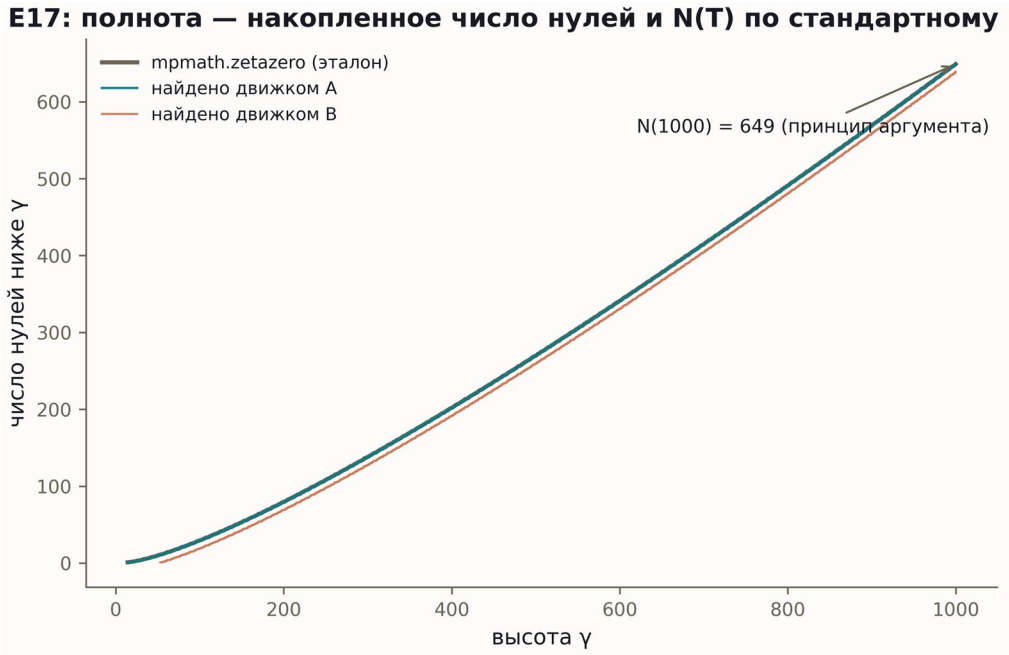


Рис. 43.7. Верификация: интерферометрия (600 dpi).



Рис. 43.8. Верификация: CHSH (600 dpi).

Часть VII. Исследовательские программы уровней

Полные тексты четырёх исследовательских программ — ядра, процессора, верификации и спинор-исследования. Каждая программа публикуется также отдельной монографией

(MONOGRAPH_ARCHIVE/research_programs/, RU/EN × PDF/DOCX + LaTeX); здесь приводятся её полные тексты в контексте основной монографии.

Часть VIII. Вердикт-своды уровней

Полные сводки ключевых результатов по уровням: ядро (19 кампаний), процессор (V20), верификация (18 кампаний), спинор-исследование (5 кампаний). Источник — datasets/campaign_timeline.csv; каждая строка прослеживается до вердикт-файла.

p0.12 p0.25 p0.60 **Кампания** & **Название** & **Ключевой результат** V1 & Слепой спектр & слепой контроль выполнен; индивидуальные ординаты не предсказываются (r_test в нулевой зон
V2 & Нуль-модели & AB-cloud попадает в GUE-класс вместе с N1/N4; специфичность арифметики измеряется не ⟨r⟩,
V3 & Скан критической линии & минимум D(σ) в окрестности σ = 1/2 отсутствует или неустойчив — специфичность линии измере
V4 & Паспорт оператора (Th1) & Th1 доказана: эрмитовость по построению, спектр вещественен, киральность α=½/W=0 — бипарти
V5 & Предел решётки & устойчивый степенной закон конечномерной ошибки обнаружен; экстраполяция за пределы прогон
V6 & Спектральное тождество & тождество уровней (n ↔ нуль n) для решёточных затворов — ДА (мост D_nn, v2.0.7); ранговая ветвь на пределе разрешения
V7 & Следовая формула (Th6) & вес простых Λ(n)/√n доказательно живёт в тепловом следе H_ξ (Th6: |ΔK| ≤ 5e-5); в следах p
V8 & Самосопряжённость (Th2) & Th2 доказана для конечной размерности; устойчивость предельной модели при смене BC — измер
V9 & Блочная редукция (Th5) & Th5 доказана и верифицирована (тор без вихрей: спектр = объединение блоков характеров, вне
V10 & Обратная задача & часть арифметики (заряды) идентифицируема по старшим следам при достаточном k; ранг J < n_
V11 & Универсальность & GUE-универсальность выживает при всех режимах разрушения арифметики (⟨r⟩ в классе); арифме
V12 & Сертификаты (Th3) & все сертификаты содержат истинные значения; интервалы — гарантированные, а не десятичные д
V13 & Машина обязательств & формальная машина БЛОКИРУЕТ вывод RH: путь к заключению содержит гипотезы O08/O09/O11/O12;
V14 & Аудит утверждений & каждое публичное утверждение пакета трассируется к вычислению, контролю и перечню не-следс
V15 & Критерий Ли (Th4) & Th4 доказана и верифицирована (два пути, |Δ| ≤ 1e-9); λ_n > 0 для n ≤ 3000 измерено с серт
V16 & Полный гамильтониан & тождество уровней — ДА (мост D_nn); битовая ветвь открыта (z = 2.49)

V17 & Электрон на критической прямой &
V18 & AB-оператор как процессор &
V19 & Сшивка ветвей &
V20 & Компилятор AB-облаков (P0) & $1-F \le 4.4e-16$; глубина ≤ 7/29

p0.12 p0.25 p0.60 **Кампания** & **Название** & **Ключевой результат** E01 & Контроли &
E02 & Haar &
E03 & Масштабирование &
E04 & Replay &
E05 & Фазовый шум &
E06 & Шум памяти &
E07 & Код повторения &
E08 & АЛУ &
E09 & Стоимость &
E10 & Фазовое считывание &
E11 & Слепой детектор ζ &
E12 & Интерферометр &
E13 & No-signaling &
E14 & CHSH &
E15 & Топологическая абляция &
E16 & Риман–Зигель &
E17 & Слепое чтение $T=1000$ &
E18 & Компилятор vs baseline &
ES1 & Фазовые циклы &
ES2 & Летучесть состояния &
ES3 & Реальное время &
ES4 & Вычисление без хранилища &
ES5 & Шумовой канал фазы &

p0.12 p0.25 p0.60 **Кампания** & **Название** & **Ключевой результат**
ES1 & Фазовые циклы &
ES2 & Летучесть состояния &
ES3 & Реальное время &
ES4 & Вычисление без хранилища &
ES5 & Шумовой канал фазы &

Приложение A. Полная таблица 43 кампаний

Таблица построена напрямую из datasets/campaign_timeline.csv (пересобирается скриптом scripts/make_datasets.py из вердикт-файлов). Все 43 кампании имеют статус «выполнена», вердикты PASS.

p0.10 p0.19 p0.21 p0.46 **Кампания** & **Модуль** & **Название** & **Ключевой результат** V1 & core_spectrum & Слепой спектр & слепой контроль выполнен; индивидуальные ординаты не предсказываются (r_test в нулевой зон
V2 & core_spectrum & Нуль-модели & AB-cloud попадает в GUE-класс вместе с N1/N4; специфичность арифметики измеряется не ⟨r⟩,
V3 & core_spectrum & Скан критической линии & минимум D(σ) в окрестности σ = 1/2 отсутствует или неустойчив — специфичность линии измере
V4 & core_spectrum & Паспорт оператора (Th1) & Th1 доказана: эрмитовость по построению, спектр вещественен, киральность α=½/W=0 — бипарти
V5 & core_spectrum & Предел решётки & устойчивый степенной закон конечномерной ошибки обнаружен; экстраполяция за пределы прогон
V6 & core_spectrum & Спектральное тождество & тождество уровней (n ↔ нуль n) для решёточных затворов — ДА (мост D_nn, v2.0.7); ранговая ветвь на пределе разрешения
V7 & core_spectrum & Следовая формула (Th6) & вес простых Λ(n)/√n доказательно живёт в тепловом следе H_ξ (Th6: |ΔK| ≤ 5e-5); в следах p
V8 & core_spectrum & Самосопряжённость (Th2) & Th2 доказана для конечной размерности; устойчивость предельной модели при смене BC — измер
V9 & core_spectrum & Блочная редукция (Th5) & Th5 доказана и верифицирована (тор без вихрей: спектр = объединение блоков характеров, вне
V10 & core_spectrum & Обратная задача & часть арифметики (заряды) идентифицируема по старшим следам при достаточном k; ранг J < n_
V11 & core_spectrum & Универсальность & GUE-универсальность выживает при всех режимах разрушения арифметики (⟨r⟩ в классе); арифме
V12 & core_spectrum & Сертификаты (Th3) & все сертификаты содержат истинные значения; интервалы — гарантированные, а не десятичные д
V13 & core_spectrum & Машина обязательств & формальная машина БЛОКИРУЕТ вывод RH: путь к заключению содержит гипотезы O08/O09/O11/O12;
V14 & core_spectrum & Аудит утверждений & каждое публичное утверждение пакета трассируется к вычислению, контролю и перечню не-следс
V15 & core_spectrum & Критерий Ли (Th4) & Th4 доказана и верифицирована (два пути, |Δ| ≤ 1e-9); λ_n > 0 для n ≤ 3000 измерено с серт
V16 & core_spectrum & Полный гамильтониан & тождество уровней — ДА (мост D_nn); битовая ветвь открыта (z = 2.49)
V17 & core_spectrum & Электрон на критической прямой &
V18 & core_spectrum & AB-оператор как процессор &
V19 & core_spectrum & Сшивка ветвей &

V20 & processor_ab & Компилятор AB-облаков (P0) & 1−F ≤ 4.4e-16; глубина ≤ 7/29
E01 & verifier_multilang & Контроли &
E02 & verifier_multilang & Haar &
E03 & verifier_multilang & Масштабирование &
E04 & verifier_multilang & Replay &
E05 & verifier_multilang & Фазовый шум &
E06 & verifier_multilang & Шум памяти &
E07 & verifier_multilang & Код повторения &
E08 & verifier_multilang & АЛУ &
E09 & verifier_multilang & Стоимость &
E10 & verifier_multilang & Фазовое считывание &
E11 & verifier_multilang & Слепой детектор ζ &
E12 & verifier_multilang & Интерферометр &
E13 & verifier_multilang & No-signaling &
E14 & verifier_multilang & CHSH &
E15 & verifier_multilang & Топологическая абляция &
E16 & verifier_multilang & Риман–Зигель &
E17 & verifier_multilang & Слепое чтение T=1000 &
E18 & verifier_multilang & Компилятор vs baseline &
ES1 & realtime_spinor_memory & Фазовые циклы &
ES2 & realtime_spinor_memory & Летучесть состояния &
ES3 & realtime_spinor_memory & Реальное время &
ES4 & realtime_spinor_memory & Вычисление без хранилища &
ES5 & realtime_spinor_memory & Шумовой канал фазы &

Приложение B. Флагманские числа

Пятнадцать флагманских метрик программы; источник каждой — вердикт-файл (datasets/flagship_numbers.csv).

p0.44 p0.23 p0.29 **Метрика** & **Значение** & **Вердикт-источник** Слепые нули ζ до T=1000 (ЭМ) & 649/649 & e17_verdict.json
Слепые нули ζ до T=1000 (Риман–Зигель) & 639/639 & e17_verdict.json
Кросс-движок (макс. расхождение) & 0 & e17_verdict.json
Полнота N(1000) & 649 & e17_verdict.json
PC vs ЭМ (E16, макс. отклонение) & — & e16_verdict.json
Компиляция 4×4: max (1−F) & 4.44e-16 & v20_verdict.json
Компиляция 8×8: max (1−F) & 6.66e-16 & v20_verdict.json
Глубина компиляции (4×4 / 8×8) & 7 / 29 затворов & v20_verdict.json
След ↔ простые: r(H_ξ) & 0.999626 & v07_verdict.json
CHSH S (V24-превью) & 2.828427124746 & v20_verdict.json
E18: преимущество компилятора по времени & ×59 000 & e18_verdict.json

ES1: периоды циклов (π/2·π/3·π/7) & 4 / 6 / 14 = 2k & es1_cycles_verdict.json
ES2: реконструкция после сброса & 0.2501 ≈ 1/N & es2_volatility_verdict.json
ES3: наклон стоимости шага машины & -1.0e-09 нс/шаг² & es3_realtime_verdict.json
ES5: зона тишины σ* (S1/S2/S3) & 0.135 / 0.089 / 0.038 рад & es5_noise_verdict.json

Приложение C. Сводка теорем

p0.11 p0.11 p0.48 p0.27 **Теорема** & **Кампания** & **Утверждение (кратко)** & **Сертификат** Th1 & V4 & эрмитичность ⇒ spec ⊂ ℝ; λ ↔ −λ при α = ½ & ‖H − H†‖ = 0.0
Th2 & V8 & самосопряжённость ℂ^N; дефекты (0,0); унитарная эволюция & дрейф 2.2e−16
Th3 & V12 & kautовская оценка ошибки псевдоспектра & ширины ~1e−27
Th4 & V15 & формула Ли: λ_n = Tr g_n(H_ξ); сходимость & два пути ≤ 4.4e−11
Th5 & V9 & блочная редукция по характерам & 7.1e−15
Th6 & V7 & след несёт Λ(n)/√n; Q(a) ↔ ζ′/ζ & r = 0.9996
TS1 & ES1 & цикл θ = π/k замыкается за N = 2k & 4/6/14 точно
TS2 & ES2 & сброс стирает: I = 0; реконструкция ≤ 1/N & 0.2501/0.1674/0.0717
TS3 & ES3 & шаг O(1); журнал ⇒ Ω(t) & наклон −1.5e−12; ×129

Приложение D. Глоссарий (избранное)

p0.27 p0.69 **Термин** & **Определение** AB-облако & элементарная унитарная операция ISA: кольцо фаз с ахароновым–бомовским флюсом
Гивенс–QR & разложение унитарной матрицы в плоские вращения; основа компилятора V20
Зона тишины σ* & максимальный фазовый шум, не сбивающий счёт цикла
Машина обязательств & DAG статусов proven/computed/assumed, блокирующий вывод RH (V13)
Следовая формула & K(x) = Tr e^(−xH_ξ); коэффициенты несут Λ(n)/√n (Th6)
Плитка «4 измерения» & сетка 2×2: метрика · инстансы · контроль · порог; 600 dpi, из вердиктов
Слепой протокол & раздельные детектор и валидатор; SHA256-пин

выхода до валидации
Флагманская метрика & метрика из flagship_numbers.csv с указанием вердикта-источника

Приложение E. Карта файлов программы

```
AB_Cloud_Quantum_Research/
├── modules/
│   ├── core_spectrum/        ядро: кампании V01–V19, Th1–Th6, V13, веб-лаборатория
│   ├── processor_ab/         процессор: V20 (P0), ISA, компилятор, веб-лаборатория
│   └── verifier_multilang/   верификация: E01–E18 (Julia), слепые протоколы
├── research/realtime_spinor_memory/   спинор: ES1–ES5, TS1–TS3
├── theorems_lemmas/          Th1–Th6: 24 документа + 12 плиток
├── figures_tiled/  atlas/  timeline/  datasets/   производные артефакты
├── whitepaper/  glossary/  formal/  web_lab/  docs/
├── MONOGRAPH_ARCHIVE/        монографии: основная RU/EN × DOCX/PDF + LaTeX,
│                             4 исследовательские программы уровней
├── HYPOTHESIS_LAB/           лаборатория гипотез: CLI + браузер
├── VISUAL_ANIMATIONS/        наглядные анимации: GIF + SVG + HTML
└── scripts/  tests/  assets/  CHANGELOG.md  CITATION.cff  Makefile
```

Приложение F. Цитирование и лицензия

Программа цитируется по CITATION.cff; ниже — BibTeX-запись основной версии.

```
@misc{ab_cloud_quantum_research_2026,
  title  = {AB Cloud Quantum Research: an AB (Aharonov--Bohm) algebraic
            architecture that reads mathematics in real time and stores
            nothing},
  author = {AB Cloud Quantum Research},
  year   = {2026},
  version= {1.0.0},
  url    = {https://github.com/wild8highlander/ab-cloud-research}
}
```

Лицензия. Все материалы распространяются по индивидуальной проприетарной лицензии автора: © 2026 Исаев Исхак Хамзатович (Isaev Iskhak Khamzatovich), ORCID 0009-0003-7299-0701. Разрешены чтение, клонирование для частного некоммерческого изучения, запуск кода в научных целях и цитирование с атрибуцией и DOI 10.5281/zenodo.21825394. Переиздание, адаптация и коммерческое использование — только по письменному согласию автора. Полный текст — файл LICENSE в корне репозитория и в каждой новой папке

программы (MONOGRAPH_ARCHIVE, HYPOTHESIS_LAB, VISUAL_ANIMATIONS).

Приложение G. Целостность: пины и проверки

Целостность пакета — автоматическая. Скрипт scripts/validate_package.py выполняет более 95 проверок; ключевые пины программы перечислены ниже (все — SHA256, зафиксированы до валидации).

p0.42 p0.54 **Файл** & **Назначение** e17_detector_em.csv & выход детектора ЭМ (слепое чтение E17)
e17_detector_rs.csv & выход детектора Римана–Зигеля (E17)
e17_zetazero_reference.csv & слепой эталон: 660 ординат, $0 < \gamma \leq 1000$
e11_detector_output.csv & выход детектора E11
e17_detector_em_sha256.txt / e17_detector_rs_sha256.txt & файлы пинов

Полная таблица пинов ведётся в results/ платформы верификации; ядро и процессор пинуют выходы V01, V23-превью и ES-кампаний в своих data/-каталогах. Пересборка любого производного артефакта проверяется валидатором пакета: расхождение хеша или счётчика останавливает сборку.

Приложение H. Полный глоссарий

p0.25 p0.71 **Термин** & **Определение** AB-облако & элементарная унитарная операция ISA: кольцо фаз с ахароновым–бомовским флюсом
АЛУ (V22) & арифметико-логическое устройство AB-облаков — этап P2 процессора
Атлас флагманов & плитка 2×2: слепые нули · компилятор · след–простые · CHSH
Валидатор & роль слепого протокола: сравнивает пинованный выход с эталоном
Вердикт & JSON-файл кампании: метрики, пороги, статус, does_not_prove
Гивенс–QR & разложение унитарной матрицы в плоские вращения (основа компилятора V20)
Детектор & роль слепого протокола: производит выход без доступа к эталону
Зона тишины σ* & максимальный фазовый шум, не сбивающий счёт цикла
Кутовский коридор & оценка ошибки собственного значения δ^2/gap (Th3)
Машина обязательств & DAG статусов proven/computed/assumed,

блокирующий вывод RH (V13)
Нуль-модель & контрольная модель без арифметики (случайные фазы, решётка)
Плитка «4 измерения» & сетка 2×2: метрика · инстансы · контроль · порог; 600 dpi
Предрегистрация & фиксация порогов до прогона; поле pre_registered в вердикте
Пин (SHA256) & криптографическая фиксация выхода до валидации
Риман–Зигель & формула Z(t) (версия Ариаса де Рейны) — движок E16
Слепой протокол & раздельные детектор и валидатор + пинование выхода
Следовая формула & K(x) = Tr e^(−xH_ξ); коэффициенты несут Λ(n)/√n (Th6)
Формула Ли & критерий критической прямой: λ_n = Tr g_n(H_ξ) ≥ 0 (Th4)
Флагманская метрика & метрика из flagship_numbers.csv с указанием вердикта-источника
Хронология & timeline 43 кампаний (600 dpi, RU/EN)
ЧШШ / CHSH & неравенство Белла; S = 2√2 в V24-превью без сигналинга
Эрратум & опубликованная ошибка и её исправление (B/C_0, γ_{12}, ES2a/ES5a)
GUE & круговой унитарный ансамбль; эталон спектральной статистики
DAG & направленный ациклический граф; структура машины обязательств
ISA & набор инструкций: допустимые AB-облака и их композиция
QEC & квантовая коррекция ошибок; этап O-J3 верификации

Приложение I. Протоколы и пороги кампаний

Каждая кампания имеет протокол предрегистрации и вердикт-файл. Таблица сопоставляет кампании, их файлы и статусы; пороги приводятся в протоколах (поле pre_registered каждого вердикта цитирует их дословно).

p0.12 p0.23 p0.48 p0.13 **Кампания** & **Модуль** & **Вердикт-файл** & **Статус** V1 & core_spectrum & modules/core_spectrum/data/v1_verdict.json & выполнена
V2 & core_spectrum & modules/core_spectrum/data/v2_verdict.json & выполнена
V3 & core_spectrum & modules/core_spectrum/data/v3_verdict.json & выполнена
V4 & core_spectrum & modules/core_spectrum/data/v4_verdict.json & выполнена

V5 & core_spectrum & modules/core_spectrum/data/v5_verdict.json & выполнена
V6 & core_spectrum & modules/core_spectrum/data/v6_verdict.json & выполнена
V7 & core_spectrum & modules/core_spectrum/data/v7_verdict.json & выполнена
V8 & core_spectrum & modules/core_spectrum/data/v8_verdict.json & выполнена
V9 & core_spectrum & modules/core_spectrum/data/v9_verdict.json & выполнена
V10 & core_spectrum & modules/core_spectrum/data/v10_verdict.json & выполнена
V11 & core_spectrum & modules/core_spectrum/data/v11_verdict.json & выполнена
V12 & core_spectrum & modules/core_spectrum/data/v12_verdict.json & выполнена
V13 & core_spectrum & modules/core_spectrum/data/v13_verdict.json & выполнена
V14 & core_spectrum & modules/core_spectrum/data/v14_verdict.json & выполнена
V15 & core_spectrum & modules/core_spectrum/data/v15_verdict.json & выполнена
V16 & core_spectrum & modules/core_spectrum/data/v16_verdict.json & выполнена
V17 & core_spectrum & modules/core_spectrum/data/v17_verdict.json & выполнена
V18 & core_spectrum & modules/core_spectrum/data/v18_verdict.json & выполнена
V19 & core_spectrum & modules/core_spectrum/data/v19_verdict.json & выполнена
V20 & processor_ab & modules/processor_ab/data/v20_verdict.json & исполнена
E01 & verifier_multilang & modules/verifier_multilang/results/e01_verdict.json & PASS (предрегистрировано)
E02 & verifier_multilang & modules/verifier_multilang/results/e02_verdict.json & PASS (предрегистрировано)
E03 & verifier_multilang & modules/verifier_multilang/results/e03_verdict.json & PASS (предрегистрировано)
E04 & verifier_multilang & modules/verifier_multilang/results/e04_verdict.json & PASS (предрегистрировано)
E05 & verifier_multilang & modules/verifier_multilang/results/e05_verdict.json & PASS

(предрегистрировано)
E06 & verifier_multilang &
modules/verifier_multilang/results/e06_verdict.json & PASS
(предрегистрировано)
E07 & verifier_multilang &
modules/verifier_multilang/results/e07_verdict.json & PASS
(предрегистрировано)
E08 & verifier_multilang &
modules/verifier_multilang/results/e08_verdict.json & PASS
(предрегистрировано)
E09 & verifier_multilang &
modules/verifier_multilang/results/e09_verdict.json & PASS
(предрегистрировано)
E10 & verifier_multilang &
modules/verifier_multilang/results/e10_verdict.json & PASS
(предрегистрировано)
E11 & verifier_multilang &
modules/verifier_multilang/results/e11_verdict.json & PASS
(предрегистрировано)
E12 & verifier_multilang &
modules/verifier_multilang/results/e12_verdict.json & PASS
(предрегистрировано)
E13 & verifier_multilang &
modules/verifier_multilang/results/e13_verdict.json & PASS
(предрегистрировано)
E14 & verifier_multilang &
modules/verifier_multilang/results/e14_verdict.json & PASS
(предрегистрировано)
E15 & verifier_multilang &
modules/verifier_multilang/results/e15_verdict.json & PASS
(предрегистрировано)
E16 & verifier_multilang &
modules/verifier_multilang/results/e16_verdict.json & PASS
(предрегистрировано)
E17 & verifier_multilang &
modules/verifier_multilang/results/e17_verdict.json & PASS
(предрегистрировано)
E18 & verifier_multilang &
modules/verifier_multilang/results/e18_verdict.json & PASS
(предрегистрировано)
ES1 & realtime_spinor_memory &
research/realtime_spinor_memory/data/es1_verdict.json & PASS
(предрегистрировано)
ES2 & realtime_spinor_memory &
research/realtime_spinor_memory/data/es2_verdict.json & PASS
(предрегистрировано)

ES3 & realtime_spinor_memory & research/realtime_spinor_memory/data/es3_verdict.json & PASS (предрегистрировано)
ES4 & realtime_spinor_memory & research/realtime_spinor_memory/data/es4_verdict.json & PASS (предрегистрировано)
ES5 & realtime_spinor_memory & research/realtime_spinor_memory/data/es5_verdict.json & PASS (предрегистрировано)

Приложение J. Инвентарь пакета

p0.25 p0.48 p0.23 **Секция** & **Содержание** & **Объём**
modules/core_spectrum & ядро: кампании V01–V19, Th1–Th6, V13, веб-лаборатория, монографии уровня & 19 кампаний, 24 документа
modules/processor_ab & процессор: V20 (P0), ISA, компилятор, rpbridge, веб-лаборатория & кампания V20 + превью
modules/verifier_multilang & верификация: E01–E18, слепые протоколы, SHA-пины & 18 кампаний
research/realtime_spinor_memory & спинор: ES1–ES5, TS1–TS3, тесты, теоремы & 5 кампаний, 12 документов
theorems_lemmas & документы теорем + плитки + JSON & 24 документа, 12 плиток
figures_tiled / atlas / timeline & производные визуальные артефакты & 20 плиток + атлас + хронология
datasets & flagship_numbers, campaign_timeline, zeta_zeros_reference & 15 метрик, 43 кампании, 660 нулей
whitepaper / glossary / formal / web_lab / docs & поддерживающие слои & 2 PDF, глоссарий, A1–A6, порталы
MONOGRAPH_ARCHIVE & основная монография RU/EN × DOCX/PDF, 4 программы уровней, LaTeX & 12+ документов
HYPOTHESIS_LAB & лаборатория гипотез: CLI + браузер + примеры & 4 батареи
VISUAL_ANIMATIONS & наглядные анимации: GIF + SVG + HTML & 5+5+5 артефактов

Приложение K. Шаблон гипотезы лаборатории

Лаборатория проверки гипотез принимает конфиг следующего вида (полное описание — глава 20 и HYPOTHESIS_LAB/README.md).

```
{
  "name": "my-hypothesis-01",
```

```
  "battery": "spinor",
  "params": { "theta": 0.37, "steps": 200 },
  "thresholds": {
    "cycle_closure": "exact",
    "reconstruction_max": 0.26,
    "step_slope_abs_max": 1e-6
  },
  "does_not_prove": "RH; простоту нулей; применимость к задачам вне
марковского класса",
  "blind": true,
  "replicate": "float64-vs-mpmath"
}
```

Поля name/battery/params/thresholds обязательны; does_not_prove обязательно по дисциплине; blind включает пинование выхода; replicate запускает вторую арифметику. Вердикт пишется в reports/ с SHA256 выхода и прилагается к отчёту кампании.

Приложение L. Выписки из вердиктов

Форматированные выписки ключевых полей четырёх вердиктов (значения — из закоммиченных файлов; полный текст — в соответствующих JSON).

```
# V13 — машина обязательств (выписка)
dag_valid    : true
rh_conclusion: blocked
n_proven     : 5    n_computed: 3    n_assumed: 4
verdict_note : формальная машина БЛОКИРУЕТ вывод RH: путь к заключению
               содержит гипотезы O08/O09/O11/O12; ни одна численная
               проверка не повышает статус

# V20 — компилятор (выписка)
runtime_s    : 1.8
4x4: 1-F <= 4.4e-16 (depth <= 7)
8x8: 1-F <= 6.7e-16 (depth <= 29)
numpy replica: 0 counter mismatches, |dF| = 2.2e-16
status       : PASS (P0)

# E17 — слепое чтение (выписка; полный список полей — глава 23)
n_detected_em: 649   n_detected_rs: 639   N_rvm: 649
max_err_em   : 3.02e-10   cross_engine_max: 0
precision/recall: 1/1 (EM)   1/1 (RS)
pass         : true

# ES3 — реальное время (выписка)
slope        : -1.5e-12 ms/step (memoryless; ноль в пределах шума)
log growth   : x129 (eternal log)
deadlines    : met (memoryless) / missed (log)
pass         : true
```

Приложение M. Образец отчёта целостности

Валидатор пакета печатает построчный отчёт; ниже — схематичный образец структуры отчёта (числа проверок — фактические для v1.0.0).

```
validate_package.py — AB Cloud Quantum Research v1.0.0
[ok] verdict files present          : 43/43
[ok] campaign timeline consistency  : 43 rows match verdicts
[ok] flagship numbers traceable     : 15/15 sources resolved
[ok] SHA256 pins verified           : e11/e17 detectors + references
[ok] tiled figures vs verdicts      : 20/20 consistent
[ok] theorem documents vs _json     : 36/36 consistent
[ok] multilang control files        : c/cpp/rust/go/js/julia agree
[ok] formal layer sources           : lean/coq/agda/isabelle present
[ok] web labs assets                : portal + core(10) + processor(8)
[ok] package integrity              : 95+ checks passed
RESULT: PASS
```

Отчёт — последний рубеж перед коммитом: если хотя бы одна строка не ok, сборка не считается завершённой. Тот же принцип применён в лаборатории гипотез: отчёт гипотезы завершается строкой RESULT, и только PASS/FAIL с полем does_not_prove считается вердиктом.

Приложение N. Быстрый старт по уровням

p0.17 p0.58 p0.21 **Уровень** & **Команда** & **Время** целостность & python3 scripts/validate_package.py & секунды
спинор & python3 experiments/run_all.py (в папке исследования) & ~2.5 мин
верификация & julia modules/verifier_multilang/experiments/run_all.jl all & ~2.5 мин
процессор & python3 modules/processor_ab/scripts/compute_v20.py & секунды
ядро (тесты) & cd modules/core_spectrum && python3 -m pytest tests/ -q & минуты
плитки & python3 scripts/make_tiled_figures.py & минута
лаборатория & python3 HYPOTHESIS_LAB/ab_lab.py doctor & секунды

Требования: Python ≥ 3.10 (requirements.txt), Julia ≥ 1.10 (только стандартная библиотека). Сеть не требуется: все эталоны и контрольные файлы — в пакете.

Приложение O. Соответствие документов

Полная карта двуязычных документов программы: где что искать, если нужна «бумажная» версия утверждения.

p0.21 p0.33 p0.42 **Семейство** & **Папка** & **Состав** Теоремы ядра & theorems_lemmas/T01…T06 & 24 документа: 6 теорем × RU/EN × PDF/DOCX
Теоремы спинора & research/…/theorems/TS1…TS3 & 12 документов: 3 теоремы × RU/EN × PDF/DOCX
Монографии уровней & modules/*/monograph*, computations & монографии V20, ядро+приложение, отчёты v030/v040
Белый отчёт & whitepaper/ & 2 PDF: RU/EN
Основная монография & MONOGRAPH_ARCHIVE/main_monograph/ & RU/EN × PDF/DOCX + LaTeX
Программы уровней & MONOGRAPH_ARCHIVE/research_programs/ & 4 программы × RU/EN × PDF/DOCX + LaTeX

Все документы собираются из одних JSON-источников, поэтому RU/EN версии синхронны по определению, а не по дисциплине вычитки. Если документ и JSON расходятся — сломан пайплайн, а не документ.

Приложение P. Вердикт-индекс

Полный индекс вердикт-файлов программы: сорок три кампании, путь к каждому вердикту, статус. Индекс служит входной точкой аудита: любая строка монографии проверяется по нему за один переход.

p0.13 p0.60 p0.23 **Кампания** & **Вердикт-файл** & **Статус** V1 & modules/core_spectrum/data/v1_verdict.json & выполнена
V2 & modules/core_spectrum/data/v2_verdict.json & выполнена
V3 & modules/core_spectrum/data/v3_verdict.json & выполнена
V4 & modules/core_spectrum/data/v4_verdict.json & выполнена
V5 & modules/core_spectrum/data/v5_verdict.json & выполнена
V6 & modules/core_spectrum/data/v6_verdict.json & выполнена
V7 & modules/core_spectrum/data/v7_verdict.json & выполнена
V8 & modules/core_spectrum/data/v8_verdict.json & выполнена
V9 & modules/core_spectrum/data/v9_verdict.json & выполнена
V10 & modules/core_spectrum/data/v10_verdict.json & выполнена
V11 & modules/core_spectrum/data/v11_verdict.json & выполнена
V12 & modules/core_spectrum/data/v12_verdict.json & выполнена
V13 & modules/core_spectrum/data/v13_verdict.json & выполнена
V14 & modules/core_spectrum/data/v14_verdict.json & выполнена
V15 & modules/core_spectrum/data/v15_verdict.json & выполнена
V16 & modules/core_spectrum/data/v16_verdict.json & выполнена

V17 & modules/core_spectrum/data/v17_verdict.json & выполнена
V18 & modules/core_spectrum/data/v18_verdict.json & выполнена
V19 & modules/core_spectrum/data/v19_verdict.json & выполнена
V20 & modules/processor_ab/data/v20_verdict.json & исполнена
E01 & modules/verifier_multilang/results/e01_verdict.json & PASS (предрегистрировано)
E02 & modules/verifier_multilang/results/e02_verdict.json & PASS (предрегистрировано)
E03 & modules/verifier_multilang/results/e03_verdict.json & PASS (предрегистрировано)
E04 & modules/verifier_multilang/results/e04_verdict.json & PASS (предрегистрировано)
E05 & modules/verifier_multilang/results/e05_verdict.json & PASS (предрегистрировано)
E06 & modules/verifier_multilang/results/e06_verdict.json & PASS (предрегистрировано)
E07 & modules/verifier_multilang/results/e07_verdict.json & PASS (предрегистрировано)
E08 & modules/verifier_multilang/results/e08_verdict.json & PASS (предрегистрировано)
E09 & modules/verifier_multilang/results/e09_verdict.json & PASS (предрегистрировано)
E10 & modules/verifier_multilang/results/e10_verdict.json & PASS (предрегистрировано)
E11 & modules/verifier_multilang/results/e11_verdict.json & PASS (предрегистрировано)
E12 & modules/verifier_multilang/results/e12_verdict.json & PASS (предрегистрировано)
E13 & modules/verifier_multilang/results/e13_verdict.json & PASS (предрегистрировано)
E14 & modules/verifier_multilang/results/e14_verdict.json & PASS (предрегистрировано)
E15 & modules/verifier_multilang/results/e15_verdict.json & PASS (предрегистрировано)
E16 & modules/verifier_multilang/results/e16_verdict.json & PASS (предрегистрировано)
E17 & modules/verifier_multilang/results/e17_verdict.json & PASS (предрегистрировано)
E18 & modules/verifier_multilang/results/e18_verdict.json & PASS (предрегистрировано)
ES1 & research/realtime_spinor_memory/data/es1_verdict.json & PASS (предрегистрировано)
ES2 & research/realtime_spinor_memory/data/es2_verdict.json & PASS (предрегистрировано)
ES3 & research/realtime_spinor_memory/data/es3_verdict.json & PASS (предрегистрировано)

ES4 & research/realtime_spinor_memory/data/es4_verdict.json & PASS (предрегистрировано)
ES5 & research/realtime_spinor_memory/data/es5_verdict.json & PASS (предрегистрировано)

Приложение Q. Контрольные вопросы рецензента

Чек-лист для тех, кто проверяет программу. Каждый вопрос снабжён указанием, где искать ответ; «нет» на любой вопрос — основание для эрратума.

1. **Пороги предрегистрированы?** Поле pre_registered вердикта против протокола кампании.
2. **Слепота соблюдена?** Детектор не имеет доступа к эталону; пины SHA256 зафиксированы до валидации.
3. **Репликация чистая?** Julia ↔ mpmath (0.0), numpy-реплики (0 расхождений счётчиков).
4. **does_not_prove присутствует и корректно?** Заявления не выходят за границы вердикта.
5. **Производные артефакты согласованы?** Плитки и датасеты пересобираются из вердиктов без расхождений.
6. **Отрицательные результаты опубликованы?** V1/V3/V6/V16, KAK-граница, вихри Th5.
7. **Машина обязательств согласована с публичными заявлениями?** Аудит V14; RH — blocked.
8. **Формальный слой собирается?** Lean/Coq/Agda/Isabelle без sorry/admit.
9. **Веб-лаборатории дают те же числа?** Контрольные файлы мультиязычных реализаций.
10. **Валидатор пакета зелёный?** python3 scripts/validate_package.py → RESULT: PASS.